Brandenburg University of Applied Sciences

IT Security
Computer Science and Media
Prof. Dr. habil. Oleg Lobachev
MSc. Florian Eich

Applied Graph Kernels for Schema-Aware In-Context Learning in NL2SQL

Bachelor Thesis

Winter Semester 2025

March 8, 2026

Mara Schulke – Matr-Nr. 20215853

Abstract

Current state-of-the-art NL2SQL systems largely rely on proprietary models for inference, which in turn poses serious data sovereignty, data privacy, and reliability concerns for their users as they require transmitting database schemas and contents across the wire. This applies especially to sensitive industries such as healthcare, finance, aerospace and defense. Recently, open-source models like OmniSQL emerged which show competitive NL2SQL performance on benchmarks and thus address the gap between research and production-ready systems that leverage open-source models for sensitive air-gapped environments.

This thesis introduces Natural, a sovereign NL2SQL pipeline built around the open-source OmniSQL models. The system consists of five components: schema-aware example selection using Wasserstein-Weisfeiler-Lehman kernels (σ), schema subsetting (ϕ), few-shot query projection (π), self-refinement (ρ), and consensus voting (ν). Natural can be deployed as a database extension or as an inference service without external dependencies.

Benchmark results on Spider and Bird demonstrate an 81.0% execution accuracy on Spider (dev), 81.4% on Spider (test) and 53.8% on Bird, representing +2.0pp, +2.4pp and +15.8pp improvements over the locally reproduced OmniSQL-7B-gguf baseline respectively. The ablation study reveals that in-context learning is the most impactful component. Adding example selection to the pipeline provides the largest execution accuracy improvements with +19.8pp on Bird. Other pipeline components contribute marginally to this outcome. Furthermore, synthetic examples outperformed reference training datasets by +5.4pp on Bird, which suggests that the structural similarity of examples and the database has a greater impact than the domain similarity. A 28.1pp gap between the officially published OmniSQL results and the locally reproduced measurements highlights reproduction challenges in LLM-based NL2SQL systems and limit the ability of this thesis to perform absolute performance comparisons and derive definite conclusions. The 7–16-second candidate latency of Natural on Spider enables real-world adoption for analytical workloads but is still limited for interactive use cases.

The results of this work demonstrate that pipeline composition yields substantial (relative) execution accuracy improvements within controlled environments. Natural thus builds a foundation for sovereign NL2SQL systems in privacy- and security-sensitive contexts, preventing reliance on proprietary models.

Contents

1 Introduction **1**
1.1 Problem Statement and Motivation . . . 1
1.2 Objectives of the Thesis . . . 1
1.3 Research Questions . . . 1
1.4 Thesis Structure . . . 2

2 Literature Review **3**
2.1 Foundations of Natural Language Interfaces to Databases . . . 3
2.2 Traditional NL2SQL Approaches . . . 3
2.2.1 Rule-based and Grammar-based Systems . . . 3
2.2.2 Graph Matching Methods . . . 4
2.2.3 Interactive Systems . . . 4
2.2.4 Query Synthesis . . . 4
2.2.5 Limitations of Traditional Approaches to NL2SQL . . . 4
2.3 Neural NL2SQL Approaches . . . 5
2.3.1 Early and Intermediate Neural Approaches . . . 5
2.3.2 Relation-Aware Transformer Approaches . . . 5
2.3.3 Comparative Analysis . . . 6
2.3.4 Limitations . . . 6
2.4 Pre-trained Language Models . . . 7
2.4.1 Early Pre-trained Language Model Adaptations . . . 7
2.4.2 Advanced Pre-trained Language Model Approaches . . . 7
2.4.3 Constrained Decoding and Ranking Techniques . . . 7
2.4.4 Advantages of PLM Approaches . . . 7
2.4.5 Limitations of PLM Approaches . . . 8
2.4.6 Comparison with Large Language Models . . . 8
2.5 Large Language Models . . . 8

2.5.1 In-Context Learning . . . 8
2.5.2 Self-Correction and Iterative Refinement . . . 9
2.5.3 Candidate Selection Frameworks . . . 9
2.5.4 Retrieval-Augmented Generation . . . 10
2.5.5 Specialized LLMs and Fine-tuning . . . 10
2.5.6 Limitations and Challenges . . . 10
2.6 Benchmarking . . . 11
2.6.1 Spider . . . 11
2.6.2 Bird . . . 11
2.6.3 Spider 2.0 . . . 11
2.7 Research Gaps . . . 12
2.7.1 Advanced Open-Source Approaches . . . 12
2.7.2 Deployment and Performance Gaps . . . 12
2.7.3 Ambiguity Resolution and Semantic Accuracy . . . 12
2.7.4 Evaluation and Benchmarking Gaps . . . 12
2.7.5 Thesis Placement . . . 12

3 Theoretical Foundations **13**
3.1 Relational Database Theory . . . 13
3.1.1 Relational Model Fundamentals . . . 13
3.1.2 Core Concepts . . . 13
3.1.3 SQL as a Declarative Query Language . . . 14
3.1.4 Normalization and Schema Design . . . 14
3.2 Machine Learning Fundamentals . . . 15
3.2.1 Neural Network Architecture . . . 15
3.2.2 Learning Approaches . . . 15
3.2.3 Transformer Architecture . . . 15
3.3 Large Language Models . . . 16
3.3.1 Feedforward and Residual Connections . . . 16
3.3.2 Positional Encoding . . . 16
3.3.3 Pre-training and Fine-tuning . . . 17
3.3.4 In-Context Learning . . . 17
3.3.5 Prompt Engineering . . . 17
3.3.6 Chain-of-Thought Reasoning . . . 17
3.3.7 Model Limitations . . . 17
3.4 Natural Language Processing . . . 18
3.4.1 Text Preprocessing and Tokenization . . . 18
3.4.2 Word Embeddings and Semantic Representations . . . 18
3.4.3 Semantic Similarity Metrics . . . 19
3.5 Graph Theory . . . 19
3.5.1 Graph Fundamentals . . . 19
3.5.2 Graph Kernels . . . 20
3.5.3 Weisfeiler-Lehman Algorithm . . . 20
3.5.4 Wasserstein Distance . . . 20
3.5.5 Wasserstein Weisfeiler-Leman Kernels . . . 20

4 System Design **21**
4.1 Initialization . . . 21
4.1.1 Embedding . . . 21
4.1.2 Schema Indexing . . . 22
4.2 Functions . . . 24
4.2.1 Example Selection – σ . . . 24
4.2.2 Schema Subsetting – ϕ . . . 24
4.2.3 Query Projection – π . . . 25
4.2.4 Self Refinement – ρ . . . 25
4.2.5 Voting – ν . . . 26

4.3 Composition – nq . . . 27

5 Implementation **28**
5.1 Architecture and Infrastructure . . . 28
5.1.1 Software Architecture . . . 28
5.1.2 Resource Management Strategy . . . 29
5.1.3 Technology Stack Decisions . . . 29
5.2 Pipeline Implementation . . . 30
5.2.1 Example Selection (σ) . . . 30
5.2.2 Schema Subsetting System (ϕ) . . . 30
5.2.3 Query Projection (π) . . . 31
5.2.4 Self-Refinement Mechanism (ρ) . . . 32
5.2.5 Consensus Voting System (ν) . . . 32
5.3 Supporting Systems and Optimizations . . . 33
5.3.1 Vector Database . . . 33
5.3.2 Embedding . . . 33
5.3.3 Wasserstein-Weisfeiler-Leman Kernels . . . 34
5.4 Benchmarking Infrastructure . . . 35
5.4.1 Execution-Based Evaluation System . . . 35
5.4.2 Cross-Dataset Validation . . . 36
5.4.3 Metric Computation . . . 36
5.5 Engineering Challenges and Lessons Learned . . . 36
5.5.1 Hardware Constraints and Development Velocity . . . 36
5.5.2 Technology Stack Trade-offs . . . 37
5.5.3 Lessons Learned and Future Recommendations . . . 37

6 Evaluation **39**
6.1 Methodology . . . 39
6.1.1 Experimentation Environment . . . 39
6.1.2 Benchmark Datasets . . . 39
6.1.3 Evaluation Metrics . . . 39
6.1.4 Baselines . . . 40
6.1.5 Vector Databases . . . 40
6.2 Benchmark Results . . . 41
6.2.1 Overview . . . 41
6.2.2 Baseline Performance Analysis . . . 42
6.2.3 Spider Results . . . 42
6.2.4 BIRD Results . . . 44
6.3 Performance Characteristics . . . 45
6.3.1 Error Rate Analysis . . . 46
6.3.2 Latency Analysis . . . 46
6.4 Performance Gap . . . 47
6.4.1 Magnitude and Characterization . . . 47
6.4.2 Potential Contributing Factors . . . 48
6.4.3 Implications for Evaluation Validity . . . 49
6.4.4 Summary . . . 50

7 Discussion **51**
7.1 Results . . . 51
7.1.1 Ablation . . . 51
7.1.2 Example Structure . . . 51
7.1.3 Limited Validity . . . 52
7.2 Research Questions . . . 52
7.2.1 RQ1: To what extent can open-source LLMs achieve competitive NL2SQL performance through pipeline composition and optimization? . . . 52

7.2.2 RQ2: How do NL2SQL pipeline components interact, and which configurations optimize the accuracy-latency tradeoff? . . . 53
7.3 Practical Implications . . . 56
7.3.1 Deployment Viability and User Experience . . . 56
7.3.2 Use Cases and Accessibility . . . 57
7.4 Threats to Validity . . . 58
7.4.1 Experimental Limitations . . . 58
7.4.2 Methodological Constraints . . . 58

8 Summary **60**
8.1 Research Objectives and Achievement . . . 60
8.2 Research Questions . . . 60
8.3 Future Work . . . 60
8.3.1 Reproducibility Research . . . 60
8.3.2 System Improvements . . . 61
8.4 Closing Remarks . . . 62

A Appendix **63**
A.1 Prompts . . . 63
A.1.1 Natural Inference Prompt . . . 63
A.1.2 Natural Refinement Prompt . . . 64
A.1.3 OmniSQL Inference Prompt . . . 65
A.2 Code . . . 65
A.2.1 Source Code Repository . . . 65
A.2.2 Vector Database API . . . 66
A.2.3 System Configuration . . . 67
A.3 Benchmark Results . . . 67
A.3.1 Spider . . . 68
A.3.2 Bird . . . 68

List of Figures

1 Paradigm shifts in NL2SQL research from rule-based systems to large language models, with representative systems and benchmarks. . . . 3
2 Natural (*Full*): All four inner steps execute once per iteration, producing candidates resolved by majority voting. . . . 21
3 Normalized SQL database schema with separate tables for users and contacts. . . . 22
4 Denormalized SQL database schema with contacts stored within an array column. . . . 22
5 Using JOIN on a normalized database schema in SQL. . . . 22
6 Using array access on a denormalized database schema in SQL. . . . 22
7 Graph representation of a normalized schema. . . . 23
8 Graph representation of a denormalized schema. . . . 23
9 Software architecture of Natural: dependency graph between the five Cargo crates. Arrows point from a dependent crate to the crate it depends on. . . . 28
10 Batch embedding data flow. Inputs are either flat sequences (`embed_seq`) or pre-chunked slices (`embed_chunked`); the chunked path preserves the original layout for seamless reconstruction. Batch processing reduces per-item embedding time from $\approx 200\,\text{ms}$ to $\approx 2\,\text{ms}$. . . . 34
11 Execution accuracy overview across all systems and benchmarks. The *Syn* configuration achieves the highest verified performance on Spider (dev) at 81.0% and Bird (dev) at 53.8%, while *Train* leads on Spider (test) at 81.4%. . . . 41
12 Exact match overview across all systems and benchmarks. ICL configurations demonstrate substantial improvements over baselines, with *Train* achieving 43.9% on Spider (test), the highest verified exact match rate across all benchmarks. . . . 43

13 Candidate latency across system configurations on SPIDER benchmarks. The *Zero-Shot* configuration shows doubled latency (14.0s on dev, 15.3s on test) compared to *Baseline* (7.3s) without notable performance gains, while ICL configurations incur additional overhead from example retrieval. 43
14 Execution accuracy improvement over the *Baseline* configuration across all benchmarks. The *Zero-Shot* configuration yields marginal gains ($\leq$1.6pp), while ICL-enabled configurations show substantial improvements peaking at +21.4pp (*Syn* on BIRD dev). The substantial contrast between SPIDER and BIRD gains highlights that ICL impact scales with query and schema complexity. 44
15 Error rates across system configurations. All configurations maintain error rates below 0.7 failures per hundred queries, indicating that failures are predominantly semantic rather than syntactic or runtime related. 45
16 Candidate latency across system configurations on the BIRD benchmark. Latencies are substantially higher than SPIDER equivalents, with *Syn* and *Ground* reaching 43.7s and 43.8s respectively, driven by the larger schemas, longer prompts and more complex query generation required by BIRD. 46
17 Latency vs. execution accuracy trade-off across all verified system configurations and benchmarks. Each point represents a (configuration, benchmark) pair. This figure highlights that moving from *Baseline* to ICL-enabled configurations increases $\mathbb{EA}$, but yields significant higher $\mathbb{CL}$, particularly on BIRD with a $\mathbb{CL}$of 43s. The *Zero-Shot* configuration represents an inefficient operating point with a comparatively high latency to accuracy. 47
18 Performance gap between measured OMNISQL-7B-gguf and reported OMNISQL-7B results across benchmarks. The Δ ranges from 2.6% on SPIDER (dev) to 28.1% on BIRD (dev), highlighting differences between local reproduction and published results. 48
19 Execution accuracy on SPIDER (dev). The *Syn* configuration achieves the highest execution accuracy among all tested systems at 81.0%, outperforming OmniSQL-7B-gguf by 2.0pp. 68
20 Execution accuracy on SPIDER (test). The *Train* configuration demonstrates the strongest performance among tested configurations, achieving 81.4% execution accuracy. 69
21 Exact match performance on SPIDER (dev). ICL configurations demonstrate substantial improvements over baseline, with *Ground* achieving 42.7% exact match compared to 30.3% for *Baseline*. 69
22 Exact match performance on SPIDER (test). The *Train* configuration achieves 43.9% exact match, representing a +11.4pp improvement over *Baseline*. 70
23 Execution accuracy on BIRD (dev). The *Syn* configuration achieves the highest verified execution accuracy at 53.8%, representing a +21.4pp improvement over the *Baseline* configuration. 70
24 Exact match performance on BIRD (dev). ICL-based configurations show dramatic improvements, with *Syn* achieving 18.8% compared to near-zero rates (0.03%) for systems without example retrieval, highlighting the strong impact of in-context learning on syntactic alignment for complex queries. 71
25 Exact match improvement over the *Baseline* configuration across all benchmarks. The *Train* configuration yields the largest $\mathbb{EM}$ improvement on SPIDER (+12.0pp on dev, +11.4pp on test), while *Syn* exceeds on BIRD (dev) with +18.5pp, confirming that diverse synthetic examples provide stronger syntactic alignment for complex queries. 71

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte Pair Encoding
CCG	Combinatory Categorial Grammar
CLI	Command Line Interface
COT	Chain of Thought
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DBMS	Database Management System
EA	Execution Accuracy
EM	Exact Match
ER	Error Rate
FFI	Foreign Function Interface
FFN	Feed-Forward Network
GELU	Gaussian Error Linear Unit
GGML	Georgi Gerganov Machine Learning
GGUF	GGML Universal Format
GNN	Graph Neural Network
GPU	Graphics Processing Unit
GPT	Generative Pre-trained Transformer
gRPC	gRPC Remote Procedure Call
ICL	In-Context Learning
ILP	Inductive Logic Programming
IO	Input/Output
JSON	JavaScript Object Notation
KV	Key-Value
LLM	Large Language Model
MB	Megabyte
ML	Machine Learning
NALIR	Natural Language Interface for Relational Databases
NL	Natural Language
NLIDB	Natural Language Interface to Databases
NLP	Natural Language Processing
NLQ	Natural Language Query
NL2SQL	Natural Language to SQL
OLAP	Online Analytical Processing
OOM	Out of Memory
PLM	Pre-trained Language Model
RAM	Random Access Memory
REST	Representational State Transfer
SOTA	State of the Art
SQL	Structured Query Language
SSD	Solid State Drive
T2SQL	Text to SQL
UI	User Interface
UX	User Experience
VRAM	Video Random Access Memory
WL	Weisfeiler-Lehman
WWL	Wasserstein-Weisfeiler-Lehman

System and Model Names

Name	Description
ASTReS	AST-based Retrieval and Synthesis
Chase	Chain-of-Thought Based Schema Exploration
Chill	Constructive Heuristic Induction for Language Learning
CodeS	Code-based SQL Models
Dail-SQL	Decomposed Prompting for Text-to-SQL
Din-SQL	Decomposed In-Context Learning for SQL
Grappa	Grammar-Augmented Pre-Training Approach
Graphix	Graph-based Schema Encoding
IRNet	Intermediate Representation Network
Krisp	Kernel-based Robust Interpretation for Semantic Parsing
Ladder	Language Access to Distributed Data with Error Recovery
Lunar	Language Understanding, Natural Access, Relational
Magic	Multi-step Attribute Generation for In-Context learning
Natural	Natural Language Understanding and Translation System
NatSQL	Natural SQL (Intermediate Representation)
OmniSQL	Omniscient SQL Generation Model
Precise	Practical Extensible Research Corpus for Information Semantics
Picard	Parsing Incrementally for Constrained Auto-Regressive Decoding
Rat-SQL	Relation-Aware Transformer for SQL
Resdsql	Ranking-Enhanced Encoding for Text-to-SQL
Ryansql	Recursively Yielding Annotation Network for SQL
Strug	Structure-Grounded Pre-training
SyntaxSQLNet	Syntax-based SQL Generation Network
TypeSQL	Type-aware SQL Generation
XiYan-SQL	Multi-Generator Ensemble SQL System

Benchmarks and Datasets

Name	Description
BIRD	Big Bench for Large-scale Database Grounded Text-to-SQL
Spider	Yale Semantic Parsing and Text-to-SQL Dataset
SynSQL	Synthetic SQL Dataset
WikiSQL	Wikipedia-based SQL Dataset

1 Introduction

The introduction outlines the problem statement, objectives, research questions, and the structure of the thesis.

1.1 Problem Statement and Motivation

Most Database Management Systems (DBMS) require specialized knowledge of the Structured Query Language (SQL) in order for users to query data. This represents a usability barrier for domain experts, data analysts and researchers. Recently Large Language Models (LLMs) have made natural language interfaces to databases (NLIDBs) substantially more feasible due to their language understanding and reasoning capabilities.

State-of-the-art (SOTA) NL2SQL systems, unfortunately, largely depend on proprietary models to achieve their accuracy. These models require transmitting database schemas and queries to external services, conflicting with data sovereignty requirements and security policies in regulated sectors such as healthcare, finance, and government technology, thus creating a gap between theoretical research performance and deployable systems. This thesis explores approaches to overcome these barriers, presenting a sovereign and open-source NL2SQL system which allows users in restricted sectors to interact with their data intuitively.

1.2 Objectives of the Thesis

This thesis addresses the challenges of database accessibility by:

1. Developing a portable and offline-only NL2SQL system that can translate Natural Language Queries (NLQs) into semantically accurate SQL queries using LLMs.
2. Evaluating the effectiveness and feasibility of different models and machine learning techniques to improve the performance of the system.
3. Benchmarking the performance of the implementation against common NL2SQL benchmarks.
4. Identifying use cases for real world scenarios that could deliver improvements to users.
5. Analyzing the shortcomings and limitations of the system and proposing solutions to overcome them.

1.3 Research Questions

This thesis investigates the applicability and competitiveness of local and open-source NL2SQL systems.

RQ1 — To what extent can open-source LLMs achieve competitive NL2SQL performance through pipeline composition and optimization?

SOTA systems combine multiple techniques like in-context learning, self-correction and candidate selection. This research question investigates whether the gap between proprietary systems and open-source models like OmniSQL can be closed by combining the models with additional NL2SQL techniques. Specifically:

- Can pipeline composition improve the performance of fine-tuned models?
- How does the performance of open-source NL2SQL systems compare to closed-source model baselines on standard benchmarks?
- What accuracy ceiling can be achieved within consumer hardware constraints?

RQ2 — How do NL2SQL pipeline components interact, and which configurations optimize the accuracy-latency tradeoff?

To resolve the dependency on proprietary systems for NL2SQL, a cost-effective local deployment must be achieved. An offline NL2SQL system must utilize compute efficiently as combining all available techniques may not be optimal for performance characteristics, as component redundancies, or components with unfavorable cost-benefit ratios harm the accuracy-latency tradeoff. This research question investigates the interactions of pipeline components and which configurations are favorable:

- Do independent components of NL2SQL systems provide independent benefits or do they interact synergistically?
- Which components contribute most to accuracy improvement, and which add latency without substantial benefit?
- How should pipelines be configured for different deployment scenarios (exploratory analysis vs production queries, simple vs complex databases)?
- What is the optimal tradeoff between accuracy and latency by use case?

1.4 Thesis Structure

This thesis follows a research and development methodology to implement a portable NL2SQL system that can be used in sovereign environments and be embedded into databases.

1. **Literature Review** — Analysis of existing research on NLIDB. This review establishes the ecosystem context and identifies current SOTA approaches, their benefits and shortcomings.
2. **Theoretical Foundations** — Introduces theoretical concepts required for understanding the problem statement and the proposed solution. The fundamentals introduced in this section are referenced in the system design and implementation.
3. **System Design** — Design of a system architecture that translates natural language to SQL using LLMs. The goal is to arrive at an architecture that implements robust natural language processing, schema-aware example selection and self refinement stages whilst maintaining high semantic accuracy.
4. **Implementation** — Implementation of a portable, standalone NL2SQL pipeline, according to the system design. Porting machine learning algorithms from the system design into a production grade implementation.
5. **Evaluation** — Benchmarking the implementation against standard evaluation datasets that evaluate the implementation's performance in multiple dimensions. The most relevant evaluation metrics are execution accuracy ($\mathbb{EA}$), exact match ($\mathbb{EM}$), error rate ($\mathbb{ER}$) and candidate latency ($\mathbb{CL}$).
6. **Discussion** — Analysis and interpretation of the evaluation results against the research goals. Interpreting the results recorded during the evaluation and determining the impact of approaches used. Addresses limitations of this thesis and the accompanying implementation.
7. **Summary** — Summarizes the core contributions and proposes directions for future research alongside possible applications.

2 Literature Review

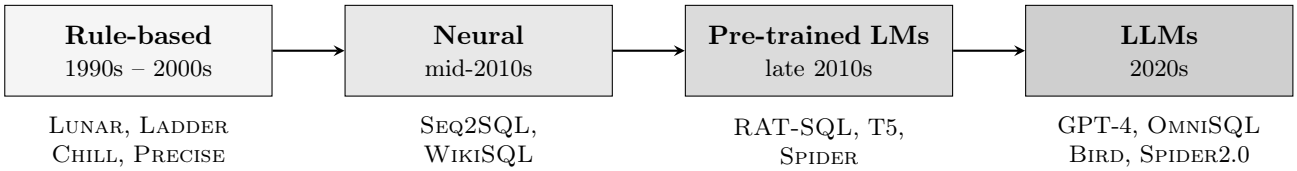

Figure 1: Paradigm shifts in NL2SQL research from rule-based systems to large language models, with representative systems and benchmarks.

A comprehensive literature review is performed to assess the research landscape on NL2SQL, also referred to as Text-to-SQL or T2SQL and NLIDBs. From the time their development accelerated in the late 1990s and early 2000s (Androutsopoulos, Ritchie, & Thanisch, 1995; Popescu, Etzioni, & Kautz, 2003; Tang & Mooney, 2001; Zelle & Mooney, 1996) until now, multiple larger paradigm shifts have occurred (Deng et al., 2020; F. Li & Jagadish, 2014; Yaghmazadeh, Wang, Dillig, & Dillig, 2017; Yu et al., 2020; Zhong, Xiong, & Socher, 2017). In particular, this thesis focuses on the recent advancements regarding LLMs and how they can be harnessed for effective NL2SQL systems (D. Gao et al., 2023; Lei et al., 2025; J. Li, Hui, Qu, et al., 2023; Rahaman, Zheng, Milani, Chiang, & Pottinger, 2024; Rajkumar, Li, & Bahdanau, 2022; B. Zhang et al., 2024). This literature review covers the foundational concepts, challenges, key advancements and research gaps associated with NL2SQL, establishing the context for this thesis and its research questions.

2.1 Foundations of Natural Language Interfaces to Databases

Earlier research on NLIDBs dates back over half a century, to the early 1970s. Two decades after the first major research systems were developed in this domain, Androutsopoulos, Ritchie, and Thanisch published an overview of NLIDBs and at the time, SOTA approaches (Androutsopoulos et al., 1995). Their work outlined key issues and challenges associated with NLIDBs, and compared them against existing solutions like formal query languages, form-based interfaces and graphical interfaces. These challenges like unobvious limits, linguistic ambiguities, semantic inaccuracy, tedious configuration etc. have shaped this field and remain relevant metrics today.

Early NLIDBs relied on traditional natural language processing (NLP) techniques to achieve natural language understanding capabilities. With Chill an inductive logic programming (ILP) approach was first introduced for NL2SQL systems, marking a key event for machine learning usage (Zelle & Mooney, 1996). In 2001 Tang and Mooney extended the approach of ILP parsing of NLQs multi-clause construction, yielding promising results in NLIDBs (Tang & Mooney, 2001).

Popescu et al. proposed a novel approach for implementing NLIDBs and outperformed the existing solutions, achieving 80% semantic accuracy (Popescu et al., 2003). The novelty of the Precise system lies in its natural language processing approach, its lexical mapping strategy, allowing Precise to identify questions it can, and can't answer, introducing the concept of semantically tractable questions, which results in an improved end user experience. Their approach was shown to be transferable and unbiased, able to handle unknown questions and maintain performance characteristics. Previous ILP-based approaches were suffering from a distribution drift of the questions asked.

This research era predominantly highlighted the tension between natural language, which can be ambiguous, semantically intractable or incomplete in meaning and formal languages like SQL which always have one deterministic and semantically tractable meaning in each statement.

2.2 Traditional NL2SQL Approaches

Prior to the widespread dominance of machine learning approaches for NLP a variety of traditional approaches have been explored in the field of NL2SQL.

2.2.1 Rule-based and Grammar-based Systems

Foundational research of NL2SQL systems mostly focused around applying rule engines that were tedious to set up, expensive to maintain and transfer across database systems. These rule engines mostly relied on

the systematic identification of linguistic patterns and were trying to template SQL from information that was derived from processing the NLQ (Codd, 1974; Hendrix, Sacerdoti, Sagalowicz, & Slocum, 1978; Woods, Kaplan, & Nash-Webber, 1972). These approaches mostly tried to translate NLQs into formal grammars which could then be deterministically mapped into a valid SQL query. These approaches have a limited number of language patterns they can process and require adaptation for new or unknown databases, query patterns etc. A potential upside of this class of NL2SQL systems is that they can confidently and reproducibly identify questions they can, and can't answer — thus leading to reliable and predictable user interfaces.

2.2.2 Graph Matching Methods

Reddy, Lapata, and Steedman applied at-the-time emerging graph matching research models to NLP, specifically to NLQ (Reddy et al., 2014). Graph matching was applied once the NLQ was parsed using a Combinatory Categorial Grammar (CCG) into a semantic graph which modeled the relationship between semantic entities. Subsequently, the graph could be matched against the actual graph derived from the database. This architecture led to better transferability across databases and domains as no fine-tuning per database was needed. Even though graph matching was an improvement over existing approaches, it quickly reached its limitations. Graph matching significantly relied on the CCG parser's accuracy, with parsing errors accounting for 10-25% of system failures depending on the dataset. Furthermore it struggled with both ambiguous language constructs and complex databases schemas which could not be matched (Reddy et al., 2014, p. 387).

2.2.3 Interactive Systems

In 2014 F. Li and Jagadish highlighted that a perfect translation of natural language into SQL was challenging as natural language is not made for query expressions and heavily relies on contextual information and clarifying questions in order to disambiguate in conversations. These learnings relate to early prior art from Montgomery and Codd which also made this observation — "natural language is not a natural query language" (Montgomery, 1972). NaLIR could accept logically complex English language sentences as input and translate them into SQL queries, including aggregation, nesting, joins etc. The innovation of NaLIR lies in its interactive communication that could detect potential misinterpretations and engage users to resolve ambiguities present in their NLQ without forcing them to entirely rephrase their query. NaLIR proved that choosing the right interaction model could offset system limitations — "In our system, we generate multiple possible interpretations for a NLQ and translate all of them in natural language for the user to choose from" (F. Li & Jagadish, 2014).

2.2.4 Query Synthesis

Yaghmazadeh et al. introduced SQLizer, which synthesizes SQL queries from natural language. This paper presents a query synthesis approach. SQLizer makes use of a three stage processing model: first generating a sketch of the query using semantic parsing, then using type-directed synthesis to complete the sketch and finally using automated repair if required. Yaghmazadeh, Wang, Dillig, and Dillig showed that alternating between repairing and synthesis yielded results that outperformed prevalent NL2SQL approaches like NaLIR. SQLizer is fully automated and database-agnostic, requiring no knowledge of the underlying schema. The authors evaluated SQLizer on 455 queries across three databases, where it ranked the correct query in the top 5 results for roughly 90% of the queries, representing a significant improvement over NaLIR (F. Li & Jagadish, 2014). This approach had shortcomings like yielding empty results, constrained language variation support and limited transferability (Yaghmazadeh et al., 2017, p.22-23).

2.2.5 Limitations of Traditional Approaches to NL2SQL

Many of the outlined approaches faced severe challenges when moving outside of research environments. Frequently systems performed comparatively well on research benchmarks that were often comprised of controlled question types and limited data variety. Ultimately no standard benchmarks existed for NL2SQL in this time, hence comparing multiple NL2SQL systems with each other was not easily feasible. Nonetheless, several fundamental challenges emerged:

1. **Limited language coverage** — Traditional NL2SQL systems were only able to process a small subset of natural language constructs they were programmed for (Hendrix et al., 1978; Kate & Mooney, 2006; Montgomery, 1972; Woods et al., 1972).

2. **Transferability** — Traditional approaches typically required manual configuration or training phases for each database they were deployed for, making cross domain usage expensive and time-consuming (Androutsopoulos et al., 1995; Woods et al., 1972).

3. **Brittleness** — Legacy systems did not handle synonyms, paraphrasing, or spelling errors. Manual adaptation and handling was needed to remediate. (Kate & Mooney, 2006; Yaghmazadeh et al., 2017).

4. **Poor scalability** — With potentially more complex underlying databases, traditional solutions often performed worse. Reddy, Lapata, and Steedman found, that with increasing schema complexity more compute was required to resolve the natural language query, thus harming scalability (Reddy et al., 2014).

These flaws of traditional NL2SQL approaches made it apparent, that a different class of approaches is needed, which increase transferability and reduce the brittleness since users are "unwilling to trade reliable and predictable user interfaces for intelligent but unreliable ones" according to Popescu et al. (2003). Whilst many approaches outlined tractable ways to increase user satisfaction and accuracy (e.g. through conversational approaches like Codd (1974)), NLIDBs were and are not considered to be a solved problem.

2.3 Neural NL2SQL Approaches

The limitations regarding brittleness, transferability and scalability of traditional approaches prompted the research community to explore neural network based NL2SQL systems. After substantial improvements in language understanding and accuracy were shown, a larger paradigm shift took place in research community in the mid-2010s.

2.3.1 Early and Intermediate Neural Approaches

Seq2SQL introduced in 2017 by Zhong et al. marked a breakthrough in the community. The authors of Seq2SQL framed NL2SQL as a reinforcement learning problem, introduced WikiSQL as training dataset and proved the viability of neural approaches.

With an execution accuracy of 59.4% Seq2SQL demonstrated its capabilities compared to previously existing NL2SQL systems, but showed that order-sensitivity of sequence-to-sequence neural networks harmed accuracy. Subsequent approaches addressed specific limitations: SQLNet eliminated order-sensitivity through a sketch-based generation approach which outperformed Seq2SQL by 7.7% on WikiSQL (dev) and by 6.4% on WikiSQL (test) (Xu, Liu, & Song, 2017). TypeSQL incorporated type-aware encoding of inputs, further improving performance by 4.7% above SQLNet while reducing the required training time by half (Yu, Li, Zhang, Zhang, & Radev, 2018).

SyntaxSQLNet improved the ability of NL2SQL systems on complex queries and marked the transitional period to Spider (see Section 2.6.1, Yu, Zhang, et al. (2018)). Spider is a comparatively large NL2SQL benchmark incorporating 10181 questions answer pairs across 138 domains. Subsequently, IRNet substantially improved the execution accuracy over SyntaxSQLNet by +28.4pp on Spider (dev) and by +19.5pp on Spider (test). IRNet introduced an intermediate representation (IR) format (SemQL) and approached NL2SQL with three-phase processing: schema linking, IR synthesis using SemQL, and deterministic SQL translation (Guo et al., 2019). Notably, graph neural networks (GNNs) were explored for neural network representation of database schemas. GNN outperformed SyntaxSQLNet and performed on par (Δ of +1.3pp) with IRNet, achieving 54.5% on Spider (dev). No evaluations were performed on Spider (test) (Bogin, Berant, & Gardner, 2019).

2.3.2 Relation-Aware Transformer Approaches

The release of RAT-SQL (Relation-Aware Transformer for SQL) by Wang, Shin, Liu, Polozov, and Richardson (2020) represented a leap for neural NL2SQL approaches. RAT-SQL diverged from earlier research through modeling the relationship between natural language and the database schema elements using relation-aware self-attention representing a novel approach for solving schema linking (Wang et al., 2020).

RAT-SQL's primary innovation was the ability to infer, understand and utilize the relationship between individual tokens in the natural language query and link it to the database schema. Thus allowing for reasoning capabilities on the actual database schema while generating the query. This yielded a 57.2% in exact match accuracy when being evaluated on the Spider benchmark, substantially outperforming comparative approaches like GNN, IRNet and IRNet V2 by 10.5%, 9.8% and 8.7% respectively. Although overall accuracy improved across all approaches when being paired with BERT (Bidirectional Encoder Representations from Transformers,

a popular pre-trained language model from Google) the Δ between the individual approaches remained steady. RAT-SQL outperformed SOTA approaches by 5% to 12.2% which demonstrated its capability advancement (Wang et al., 2020).

2.3.3 Comparative Analysis

Neural approaches progressively improved upon each other in multiple dimensions:

1. **Model Sophistication** — From sequence-to-sequence (Seq2SQL) to sketch-based and type-augmented approaches (SQLNet, TypeSQL), graph-based methods (GNN), and relation-aware self-attention (RAT-SQL) demonstrated increasing architectural sophistication in order to improve execution accuracy on benchmarks like Spider (Bogin et al., 2019; Wang et al., 2020; Xu et al., 2017; Yu, Li, et al., 2018; Yu, Yasunaga, et al., 2018; Zhong et al., 2017).

2. **Transferability** — The cross-domain generalization capabilities improved substantially, with RAT-SQL achieving the strongest performance on Spider's independent test set, thus preventing overfitting on training data (Wang et al., 2020).

3. **Robustness** — Structured decoding (SyntaxSQLNet), intermediate representations (IRNet), and relation-aware attention (RAT-SQL) increasingly enabled the robust handling of synonyms, spelling mistakes, and language variations, thus successfully addressing the limitations of traditional approaches (Guo et al., 2019; Wang et al., 2020; Yu, Yasunaga, et al., 2018).

4. **Query Complexity** — The performance on complex queries involving multiple tables, relying on complex aggregations, nested structures and joins improved dramatically. RAT-SQL marked the at-the-time capability ceiling on complex queries using neural approaches, outperforming IRNet by 10.5% (Guo et al., 2019; Wang et al., 2020).

5. **Schema Understanding** — TypeSQL, GNN and RAT-SQL each demonstrated enhanced schema linking capabilities, yielding the ability to accurately reason about user intent and traverse the database schema during query generation (Bogin et al., 2019; Wang et al., 2020; Yu, Li, et al., 2018).

2.3.4 Limitations

Despite the substantial accuracy, transferability and robustness improvements that were observed (Guo et al., 2019; Wang et al., 2020), neural NL2SQL approaches still suffered from serious shortcomings and unsolved challenges:

1. **Training Data** — Neural networks required substantially more training data than traditional systems which in turn required sophisticated efforts for data collection (Yu, Zhang, et al., 2018).

2. **Correctness** — The misalignment between neural networks and formal languages introduced frequent cases where models produced invalid SQL. Approaches like SyntaxSQLNet improved this by utilizing syntax trees during decoding but nonetheless syntactic correctness remained a challenge across multiple iterations of neural systems (Yu, Yasunaga, et al., 2018).

3. **Domain Language** — Despite the increased transferability of neural approaches, they still had a limited vocabulary and inter-domain understanding of terminology and relation between concepts. This yielded highly domain specific natural language to be challenging for these systems.

4. **Observability** — The black-box nature of neural networks made approaches relying on them, particularly the systems utilizing complex neural network architectures, hard to understand and explain in cases where they yielded undesirable output.

These limitations, combined with demonstrated performance improvements when paired with pre-trained language models (Wang et al., 2020), motivated further research into language model-based approaches.

2.4 Pre-trained Language Models

Combining specialized neural networks with pre-trained language models (PLMs) led to a pivotal point in the NL2SQL research field towards focusing on the application of PLMs for NL2SQL systems. Models like BERT or T5 offered performance improvements over specialized neural networks due to training on large natural language data, instead of smaller NL2SQL datasets — Spider2.0 which is a contemporary NL2SQL benchmark consists of just 632 real-world questions (Lei et al., 2025). PLM-based NL2SQL systems observed performance improvements through improved understanding of patterns and identification of semantic relationships.

2.4.1 Early Pre-trained Language Model Adaptations

? presented Grappa, a grammar-augmented pre-training approach in 2020 which was built on $\text{RoBERTa}_{\text{LARGE}}$, a derivative model of BERT (Yu et al., 2020). It was trained by generating synthetic training data using a Synchronous Context-Free Grammar which identifies patterns in NLQ as templates for synthesizing training data. Specialized pre-training allowed Grappa to establish a connection between natural language and database schema elements, showing improvements on existing approaches on benchmarks like Spider and WikiSQL (Yu et al., 2020).

StruG (Structure-Grounded Pre-Training) was subsequently introduced in 2020 by Deng et al. which improved model abilities for schema linking, separating the problem in three facets: column grounding, value grounding and column-value mapping. StruG achieves similar performance to Grappa while being cheaper to train (Deng et al., 2020).

2.4.2 Advanced Pre-trained Language Model Approaches

Choi, Shin, Kim, and Shin introduced Ryansql in 2020, which implemented a sketch-based approach for decomposing complex SQL generation into smaller problems. Ryansql transformed nested statements into top-level statements using the Statement Position Code technique, achieving 58.2% accuracy on Spider, a 3.2% improvement over prior best (Choi et al., 2020).

J. Li, Hui, Cheng, et al. introduced GRAPHIX-T5 in 2023, which combined T5 PLMs with graph-aware layers for NL2SQL tasks. GRAPHIX-T5 constructs a schema graph where nodes represent tables and columns and edges represent relationships between them, such as foreign keys or column association (J. Li, Hui, Cheng, et al., 2023). GRAPHIX-T5 outperformed standard T5 models, with $\text{GRAPHIX-T5}_{\text{LARGE}}$ showing a 6.6% increase in execution accuracy over T5_{LARGE}. When both GRAPHIX and baseline T5 models were combined with Picard, a constrained decoding mechanism, the absolute Δ between them jumped to 7.6%, reaching 81.0% on Spider dev (J. Li, Hui, Cheng, et al., 2023).

H. Li, Zhang, Li, and Chen proposed Resdsql in 2023, which decoupled schema linking and skeleton parsing. Resdsql employed a ranking approach to filter schema elements before passing them to the model for query generation, improving token efficiency and achieving leading performance on Spider, outperforming $\text{GRAPHIX-T5}_{\text{3B}}$-PICARD by 0.8% in execution accuracy. When combined with NatSQL (Gan et al., 2021) absolute improvement jumped to 3.1%.

2.4.3 Constrained Decoding and Ranking Techniques

A major challenge in NL2SQL research is ensuring model generated queries are syntactically valid. Scholak, Schucher, and Bahdanau released Picard in 2021, a constrained decoding mechanism which utilizes SQL grammar to incrementally parse generated SQL, rejecting invalid tokens. Picard significantly improved performance of PLMs like T5 and BERT, lifting them from mid-level to top-performing solutions on Spider. This addresses an issue associated with PLMs — while they outperform in natural language understanding, they often lack SQL grammar knowledge and tend to generate queries that are not executable due to unconstrained output space (Scholak et al., 2021).

2.4.4 Advantages of PLM Approaches

PLM approaches to NL2SQL tasks yielded performance improvements and primed the field towards LLMs. PLM approaches brought upsides:

1. **Compute Efficiency** — PLMs like RESDSQL achieve up to 84.1% on Spider whilst using fewer parameters than contemporary LLMs (H. Li et al., 2023).

2. **Transferability** — Approaches like GRAPPA and STRUG incorporate domain-specific understanding of natural language, table structures and SQL syntax during pre-training, addressing a primary issue with neural approaches (Deng et al., 2020; Yu et al., 2020).

3. **Vocabulary** — PLMs offer larger vocabulary due to vast training data, enabling them to handle variety of natural language patterns and addressing benchmark-overfitting of neural approaches.

2.4.5 Limitations of PLM Approaches

Although representing the best approaches at the time, PLMs introduce problems associated with their non-NL2SQL nature:

1. **Fine-tuning Requirements** — Most PLMs require substantial domain-specific fine-tuning, limiting adaptation to new domains. Furthermore when not using synthetic data generation like GRAPPA, annotated training datasets are needed (Yu et al., 2020).

2. **Wide Input & Output Space** — "Large pre-trained language models for textual data have an unconstrained output space; at each decoding step, they can produce any of 10,000s of sub-word tokens" (Scholak et al., 2021). GRAPHIX-T5 and PICARD proposed solutions (J. Li, Hui, Cheng, et al., 2023; Scholak et al., 2021).

3. **Limited Schema Awareness** — PLMs tend to incorporate limited schema awareness out of the box. RESDSQL and GRAPHIX-T5 tried to improve schema linking & awareness (H. Li et al., 2023; J. Li, Hui, Cheng, et al., 2023), nonetheless the non-specialized nature prevents NL2SQL being part of fundamental model architecture.

2.4.6 Comparison with Large Language Models

Research on applying PLMs for NL2SQL primed the field for a transition towards LLMs. While PLMs range from millions to a few billion parameters, prevalent LLMs such as GPT-3 and GPT-4 operate at larger scales, enabling in-context learning (ICL) techniques for improved transferability across domains (D. Gao et al., 2023). While PLMs require fine-tuning on domain-specific data (H. Li et al., 2023; J. Li, Hui, Cheng, et al., 2023), LLMs transfer cost to inference. Approaches like DINSQL show that with LLMs the engineering challenges around model instruction gained relevance while model training became less of a central problem (Pourreza & Rafiei, 2023).

2.5 Large Language Models

LLMs such as GPT-3, GPT-4, and Claude transformed the landscape of NL2SQL research. Early experiments with LLMs for NL2SQL tasks showed competitive capabilities against PLM approaches (D. Gao et al., 2023). Rajkumar et al. (2022) demonstrated that CODEX, a GPT-3-based model, without fine-tuning, could achieve competitive performance on SPIDER, outperforming existing approaches that required extensive training.

2.5.1 In-Context Learning

ICL, sometimes referred to as few-shot learning, is an approach leveraging the large context windows of LLMs by providing examples. The context windows of frontier LLMs can reach up to hundred thousands of tokens, enabling researchers to provide examples of accurate NL2SQL translation instead of applying parameter updates. ICL in NL2SQL refers to the process of providing the LLM with a small number of example pairs of NLQ to SQL mappings. This approach provides flexibility, as database contents, previous answers and large datasets can be dynamically utilized based on the question, rather than requiring retraining.

Example selection strategies were shown to have a substantial impact on ICL performance; D. Gao et al. (2023) evaluated example selection methods like Random, Question Similarity Selection, Masked Question Similarity Selection, and Query Similarity Selection and proposed a strategy to select, organize and present ICL examples to LLMs. DAIL-SQL utilizes both question and query similarity, masking domain-specific words and prioritizing examples that exceed a similarity threshold of τ (D. Gao et al., 2023, p. 5). DAIL-SQL encodes examples as question-SQL-pairs without the respective schema to improve token efficiency. Using a Code Representation Prompt for question and schema encoding enabled DAIL-SQL to achieve 86.6% execution accuracy on SPIDER. While GPT-4 achieves 72.3% execution accuracy zero-shot on SPIDER (D. Gao et al., 2023,

Table 1, p. 8), few-shot learning shows to improve model performance. One-shot boosts GPT-4's execution accuracy to 80.2%, a 7.9% increase, while five-shot reaches 82.4% (D. Gao et al., 2023, Table 2, p. 8). XiYan-SQL, Chase-SQL and Din-SQL all utilize variations of ICL to achieve their results (Y. Gao et al., 2025; Pourreza et al., 2024; Pourreza & Rafiei, 2023).

2.5.2 Self-Correction and Iterative Refinement

Pourreza and Rafiei (2023) proposed DIN-SQL as an approach to NLIDBs that rely on LLMs. DIN-SQL decomposes complex queries into sub-parts and utilizes ICL and self-correction during the generation phase. Compared to DAIL-SQL, which relies on example selection, DIN-SQL focuses on a refinement loop that allows the model to self-correct errors, repairing schema linking, syntactic or semantic errors by reviewing its output against the schema, user input and database errors. DIN-SQL achieves 85.3% execution accuracy on Spider, surpassed by DAIL-SQL by 1.3% (D. Gao et al., 2023; Pourreza & Rafiei, 2023). Pourreza and Rafiei found that generic self-correction which assumes that the query contains errors, lowers execution accuracy by 4.2% on Spider compared to gentle self-correction, which assumes nothing about the validity of the query. Smaller models perform better with generic and larger models with gentle correction (Pourreza & Rafiei, 2023).

Building upon DIN-SQL's self-correction module, Askari, Poelitz, and Tang (2024) proposed MAGIC (Multi-Agent Guideline for In-Context Text-to-SQL) which automates self-correction prompt engineering through a set of specialized agents (Askari et al., 2024). MAGIC consists of a manager agent, a correction agent and a feedback agent that collaboratively refine LLM instructions during the refinement loop. MAGIC derives common failure patterns of the initial query generation phase from training data, allowing it to spot common mistakes at generation time. The autogenerated guidelines from MAGIC yield 85.6% execution accuracy on Spider (dev) which represented a 5.31% improvement over DIN-SQL's human written correction guidelines (Askari et al., 2024). While DIN-SQL and MAGIC focus on automated self-correction in single-turn settings, H. Zhang, Cao, Xu, Chen, and Yu (2024) introduced Chain-of-Editions (CoE-SQL), which addresses challenges of multi-turn conversational NLIDBs. Interactive information seeking from the user has shown to be an effective way to drive accuracy and improve user satisfaction (F. Li & Jagadish, 2014). Rather than approaching each query independently, CoE-SQL recognizes that successive SQL queries in a conversational context usually require only small and incremental modifications of previous queries (F. Li & Jagadish, 2014; Montgomery, 1972; H. Zhang et al., 2024).

2.5.3 Candidate Selection Frameworks

Candidate selection strategies have shown performance improvements on challenging benchmarks, acknowledging the difficulty of generating perfect SQL in one attempt even with capable LLMs and self-correction mechanisms.

Pourreza et al. (2024) introduced CHASE-SQL in 2024, a framework that leverages multiple reasoning paths to generate query candidates. CHASE-SQL harnesses three generation strategies: a divide-and-conquer approach which breaks down complex NLQs into sub tasks, a Chain of Thought (COT) based generation approach which inspects execution plans of SQL queries and a schema-aware generation of synthetic examples for ICL (Pourreza et al., 2024). For candidate selection CHASE harnesses a fine-tuned LLM that can do binary selection of candidates. CHASE-SQL yields 73% execution accuracy on Bird and 87.6% on Spider (Pourreza et al., 2024). Conceptually similar work has been done by Y. Gao et al. with XiYan-SQL which is architected as multi-generation ensemble strategy with better schema representation. XiYan-SQL integrated ICL alongside supervised fine-tuning approaches to generate query candidates (Y. Gao et al., 2025). A key contribution of Y. Gao et al. (2025) is their M-Schema representation of database schemas, which improves the model's schema awareness and reduces schema linking errors. The query generation stage utilizes both a fine-tuned SQL generation model and ICL strategies. Following the query generation stage a self-correction stage is utilized to correct common errors. Lastly a selection model is used to choose the most accurate candidate (Y. Gao et al., 2025). XiYan-SQL achieved 89.65% execution accuracy on Spider and 73.34% on Bird (Y. Gao et al., 2025).

Both CHASE and XiYan-SQL show that using distinct models for candidate generation and candidate selection yield best-in-class execution accuracy, combining the strengths of LLMs for language understanding with specialized model architectures for candidate ranking.

2.5.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) has emerged as a paradigm for enhancing LLMs and thus NL2SQL systems by integrating external knowledge retrieval with the generative capabilities of LLMs. The most prevalent form of RAG in NL2SQL is the encoding of the database schema into the LLM prompt. Y. Gao et al. (2025) proposed the M-Schema format, a semi-structured text-based schema description. D. Gao et al. (2023) proposed a Code Representation Prompt encoding raw SQL construction statements. Y. Gao et al. (2025) provided an ablation study for the M-Schema which yielded questionable results. While the M-Schema format yielded best results when XiYan-SQL was combined with GPT-4o or Claude 3.5 Sonnet, it performed worse than alternatives on DeepSeek and Gemini models (Y. Gao et al., 2025).

Resdsql was one of the earlier approaches to explore subsetting database schemas prior to encoding (H. Li et al., 2023). Shen et al. (2024) introduced ASTReS which dynamically retrieves database schemas and uses abstract syntax trees (ASTs) to select optimal few-shot examples for ICL. By pruning the ASTs down to the most relevant subset, ASTReS achieved 86.6% execution accuracy on Spider at time of publication, indicating that schema subsetting is a sensible optimization mechanism. ASTReS was combined with GRAPHIX-T5 to achieve this result (Shen et al., 2024).

This technique becomes especially relevant for large enterprise schemas with hundreds of tables and columns and recent benchmarks like Spider2.0. ASTReS demonstrated that efficient schema retrieval enables smaller models like GRAPHIX-T5 to achieve competitive performance against LLM-based approaches like DAIL-SQL (D. Gao et al., 2023; Shen et al., 2024).

2.5.5 Specialized LLMs and Fine-tuning

While general-purpose LLMs demonstrated strong natural language capabilities, researchers explored the potential of fine-tuned LLMs for NL2SQL tasks, which offer a tradeoff between natural language understanding and SQL generation.

To resolve the prevalent dependency on proprietary APIs for inference, H. Li et al. (2024) introduced CodeS in 2024, a series of open-source language models dedicated for NL2SQL tasks with parameter sizes ranging from 1B to 15B. H. Li et al. (2024) showed that CodeS-7B achieves 85.4% execution accuracy on Spider, outperforming both fine-tuning approaches like Redsql and GRAPHIX-T5-PICARD as well as prompting based methods DIN-SQL+GPT-4 and DAIL-SQL+GPT-4 (H. Li et al., 2024). The same tendency was observed on Bird with CodeS-15B achieving 60.37%, compared to 57.41% for DAIL-SQL+GPT-4 and 55.90% for DIN-SQL+GPT-4 (H. Li et al., 2024). While more recent approaches like XiYan-SQL and CHASE both outperform CodeS, CHASE relies on proprietary models and XiYan-SQL doesn't disclose what base models were used.

H. Li et al. published a follow-up paper in 2025 which introduced the next-generation OmniSQL models with 7B, 14B and 32B sizes, trained using synthetic data generation. The models achieve execution accuracy improvements on both Spider and Bird; the 7B model reaches 88.9% on Spider and 66.1% on Bird, a 5%+ improvement over comparable alternatives (H. Li et al., 2025). These results were achieved without combining OmniSQL with advanced ICL, self-correction, RAG or candidate-selection techniques, indicating that higher accuracy scores are possible.

2.5.6 Limitations and Challenges

Despite the advancements of LLM-based NL2SQL systems, a set of limitations and challenges emerged that prevent real-world adoption:

1. **Hallucination** — LLM-based NL2SQL systems suffer from incorrect, but syntactically correct SQL queries. They produce subtle semantic errors like reversing the order of aggregations or using the wrong aggregation method are hard to catch. This is contrary to traditional approaches which fail visibly on unknown questions and databases. As noted by Floratou et al., "achieving Enterprise-Grade NL2SQL is still far from being resolved" even with SOTA models, complex real-world database schemas and ambiguous queries remain challenging (Floratou et al., 2024; Lei et al., 2025).

2. **Resource Requirements** — SOTA models require significant resources during inference, making them expensive to deploy at scale. While smaller LLMs exist, they typically show reduced performance on complex queries, creating a trade-off between accuracy and resource efficiency. Even fine-tuned LLMs like OmniSQL degrade in performance with shrinking parameter sizes, but maintain higher parameter efficiency over general purpose LLMs.

3. **Data Privacy and Security Implications** — Using LLMs through proprietary APIs imposes data privacy and security threats as sensitive data may be transferred to respective model vendors. Recent open-source model developments like CodeS and OmniSQL mitigate this problem partially, but using local inference with LLMs increases hardware requirements.

4. **Ambiguity Handling in Complex Scenarios** — LLMs continue to struggle with ambiguity resolution in NLQs, especially on complex database schemas. This becomes particularly prominent in enterprise databases where similar columns exist across multiple tables, or domain-specific terminology which might have multiple possible interpretations. Approaches like multi-path reasoning and candidate selection show improvements in execution accuracy, but often increase inference time and complexity (Y. Gao et al., 2025; Pourreza et al., 2024).

5. **Competitive PLM Approaches** — Approaches like Resdsql and GRAPHIX-T5 achieved competitive execution accuracy while using fewer parameters than LLMs (H. Li et al., 2023). Despite reducing inference-time requirements, these approaches required domain-specific fine-tuning which limited flexibility and introduces training costs. Using PLMs allows frontloading the computational effort and can thus reduce ongoing costs in compute-constrained environments. Yet, current research shifted towards LLMs due to their transferability and natural language capabilities.

2.6 Benchmarking

In 2018 Spider was introduced as a standardized benchmark for NL2SQL systems and was recently followed by Bird in 2023 and Spider 2.0 in 2025. These benchmarks allow cross system comparisons within the NL2SQL research community and act as a goal post for performance improvements.

2.6.1 Spider

Spider, introduced by Yu, Zhang, et al. in 2018, has become the de facto standard benchmark for evaluating complex and cross-domain Text-to-SQL systems. It consists of 10,181 questions and 5,693 unique SQL queries spanning 200 databases across 138 domains. Spider was split in training and test sets which contain different databases in order to prevent overfitting models from succeeding. Spider evaluates both exact match accuracy and execution accuracy, with contemporary SOTA systems achieving approximately 85-90% execution accuracy as of 2026 (Y. Gao et al., 2025; H. Li et al., 2025; Pourreza et al., 2024; Yu, Zhang, et al., 2018).

2.6.2 Bird

The Bird benchmark, released in 2023, addresses the gap between academic benchmarks and real-world applications by focusing on large-scale databases with actual data content (J. Li, Hui, Qu, et al., 2023). Bird contains 12,751 text-to-SQL pairs and 95 databases with a total size of 33.4 GB across 37 professional domains. Bird emphasizes challenges related to dirty database contents, external knowledge between natural language questions and database values, and SQL efficiency in massive databases. Even SOTA LLMs like GPT-4 achieve only 54.89% execution accuracy on Bird, compared to human performance of 92.96%, highlighting the significant challenges posed by real-world database scenarios on NLIDBs (J. Li, Hui, Qu, et al., 2023).

2.6.3 Spider 2.0

Spider 2.0 which was introduced by Lei et al. in 2025 represents the most recent advancements of benchmarks for NL2SQL systems. Spider 2.0 is much smaller, with only 632 real-world problems which were derived from enterprise database use cases. Spider 2.0 moves towards testing the deep understanding of the database, requiring models to understand metadata, SQL dialect documentation and project-level codebases (Lei et al., 2025). The tasks contained in Spider 2.0 often demand multiple complex SQL queries often exceeding the 100-line mark. Lei et al. highlights the gap between research and production environments, as advanced NL2SQL systems achieve only 21.3% on Spider2.0 compared to 91.2% on Spider (Lei et al., 2025; Yu, Zhang, et al., 2018).

2.7 Research Gaps

The literature shows that significant progress has been made in the development of practically feasible NL2SQL systems. Despite these advancements, several critical research gaps remain open which limit the practical deployment of NL2SQL systems.

2.7.1 Advanced Open-Source Approaches

The research community primarily focused on developing the most performant NL2SQL systems, paying less attention to fully open-source solutions. Thus, two areas are under-researched:

1. **Limited Exploration of Technique Synergies** — Research primarily focuses on techniques like candidate selection, RAG, and self-correction in isolation, using primarily proprietary LLMs. A gap remains in understanding how these techniques can be optimally combined, especially with open-source models.
2. **Efficiency-Accuracy Tradeoffs** — Whilst recent research efforts focus primarily on achieving a SOTA execution accuracy metric, the relationship of computational requirements and performance gains remains unclear.

2.7.2 Deployment and Performance Gaps

Despite impressive academic results, significant gaps remain in transitioning NL2SQL systems from research environments to real-world deployments:

1. **Database Integration** — While existing research has often focused on standalone systems, little attention has been given to the integration complexities between NL2SQL capabilities and DBMS.
2. **Latency Considerations** — Most research papers evaluate models based on their accuracy metrics alone without factoring in the latency and throughput characteristics of their solutions, even though responsiveness is an important metric for user experience.
3. **Privacy and Security** — While open-source models address some privacy concerns by enabling a local deployment of LLMs, research gaps remain regarding NL2SQL systems which respect database access controls and security policies.

2.7.3 Ambiguity Resolution and Semantic Accuracy

Fundamental questions remain open regarding effective ambiguity resolution. Current LLM-based systems struggle to identify queries containing ambiguities that they can't confidently resolve. Current research and benchmarks does not focus sufficiently on ambiguity detection and resolution.

2.7.4 Evaluation and Benchmarking Gaps

Two primary gaps remain in the evaluation and benchmarking infrastructure of NL2SQL systems:

1. **Performance Metrics** — Current benchmarks often don't capture latency or throughput of NL2SQL systems at all, allowing for solutions to achieve SOTA scores that require significantly more resources than their predecessors.
2. **User Experience Metrics** — Contemporary benchmarks focus on execution accuracy without assessing potential user satisfaction, trust, and overall experience. Although execution accuracy is a big factor for the trustworthiness of a NL2SQL system, it is not the only component as indicated by F. Li and Jagadish in 2014.

2.7.5 Thesis Placement

The under-researched aspects regarding the performance gap between proprietary and open-source models as well as the integration, privacy and security concerns mentioned above, motivate the implementation of Natural. Specifically this thesis researches the ability of schema-aware example selection for in-context learning using graph-kernels to improve the semantic accuracy of offline only NL2SQL systems. By providing a production-ready implementation of a Natural, this work further bridges the gaps between theoretical advancements and their practical deployability, whilst exploring performance and portability boundaries.

3 Theoretical Foundations

This section introduces theoretical concepts needed for understanding NATURAL's design and implementation sections. It covers relational database theory, machine learning fundamentals, large language models, natural language processing, embeddings and graph theory.

These foundations support the system's core functions: example selection (σ), schema subsetting (ϕ), query projection (π), self-refinement (ρ), and voting (ν).

3.1 Relational Database Theory

The relational database theory provides a mathematical framework for formalizing data layout in database management systems (DBMS), enabling efficient storage and retrieval mechanisms. This theory helps to understand how natural language queries need to be translated into database operations and what challenges arise during this translation.

3.1.1 Relational Model Fundamentals

The relational model was introduced by Codd in 1970 and forms the theoretical foundation for most modern database systems (Codd, 1970, 1974). A relational database organizes data in relations (i.e., tables), where each relation consists of tuples (i.e., rows) containing attributes (i.e., columns) of specific domains (i.e., data types and constraints). The mathematical origins of this model are rooted in set theory and first-order predicate logic (Codd, 1970; Date, 2003).

The relational model abstracts data storage and physical implementation details by design, allowing interactions with data through logical structures (relations, tuples, attributes etc.) rather than exposing the underlying storage mechanism. This abstraction is fundamental for NL2SQL systems, as natural language queries can be mapped into queries on these structures regardless of underlying physical storage implementation, allowing for transferability of NL2SQL systems (e.g., across different database deployments).

3.1.2 Core Concepts

The core concepts of the relational model that are relevant for this thesis include:

- **Relations** — Mathematical sets of tuples representing entities (e.g., employees, products) or relationships (e.g., enrollment, purchases). Each relation has a fixed structure.
- **Tuples** — Tuples represent rows of data, grouping several attributes into one logical unit (e.g., one purchase containing the amount, date, and shop). All tuples in the same relation share the same structure.
- **Attributes** — Frequently referred to as columns, representing a dimension of a tuple (e.g., customers have names). Attributes are represented in a fixed domain.
- **Domains** — Sets of allowed values for attributes, mathematically formalizing data types and constraints.
- **Schemas** — Structural definitions specifying relations, their attribute names, domains, and integrity constraints.
- **Primary Keys** — Uniquely identify tuples within a relation. These ensure entity integrity and referential relationships.
- **Foreign Keys** — Reference primary keys in other relations, allowing databases to maintain referential integrity and enabling semantically correct complex queries across multiple tables through joins.

These concepts form the semantic foundation that NLIDBs must navigate when translating user queries into formal database operations. As shown in the literature review, schema linking is a crucial problem of NLIDBs, establishing an understanding of which relations a natural language query refers to is the first step of formalizing a database operation for retrieval.

3.1.3 SQL as a Declarative Query Language

Structured Query Language (SQL) is a de facto standard for interfacing with relational databases. It is a declarative language to describe database operations like selection, insertion, updating and deletion of database contents. SQL's design is influenced by relational algebra and tuple relational calculus. It enables complex data retrieval through a readable query syntax incorporating relational algebra operations like:

1. **Selection** (σ) — Filtering tuples based on specified conditions
2. **Projection** (π) — Extracting specific attributes from relations
3. **Joins** ($\bowtie$) — Combining data from multiple relations based on common attributes
4. **Aggregation** — Computing summary statistics over groups of tuples

The basic syntax of SQL queries follows a logical pattern that reflects these theoretical operations:

```
SELECT attributes      -- Project attributes
FROM relations         -- Specify relations to use
WHERE conditions       -- Selection of tuples
GROUP BY attributes    -- Grouping for aggregation
HAVING conditions      -- Group-level filtering
ORDER BY attributes    -- Ordering of results
```

There are further operations like various types of `JOIN`s, `OFFSET`, `LIMIT`, `WITH` that are heavily used in SQL which are excluded for readability and simplicity.

The nature of SQL creates a challenge for NL2SQL systems; natural language queries express (often fuzzy) user intent in terms of desired output (e.g., "Give me 5 great movies") while SQL formalizes an explicit and discrete way of how to retrieve this information from the database:

```
SELECT m.title, AVG(r.score) as rating
FROM movies m
JOIN ratings r ON m.movie_id = r.movie_id
GROUP BY m.movie_id, m.title
HAVING AVG(r.score) >= 8
ORDER BY rating DESC
LIMIT 5
```

This semantic gap between natural language and formal queries represents the core challenge that this thesis addresses. SQL's syntax allows complex queries to be built from simpler components through nesting and combination of operations. NL2SQL systems must fill in the individual query components and how to compose these to semantically accurate and syntactically correct query statements. This is important when dealing with complex natural language queries that may require multi-clause SQL statements involving subqueries, multiple joins, and aggregation functions to answer.

3.1.4 Normalization and Schema Design

Database schemas are usually normalized to eliminate redundancy and maintain data integrity. The normal forms (1NF, 2NF, 3NF, BCNF) provide design guidelines and rules for relational database design to reduce duplication, inconsistencies and anomalies (Date, 2003).

Understanding normalization forms is crucial for NL2SQL systems to handle the accompanying challenges. Specifically, normalized schemas often contain multiple tables to store a single piece of information, which requires NLIDBs to understand relationships and generate appropriate joins. Humans typically think about data in denormalized, conceptual terms, which means that NL2SQL systems must overcome this layout difference as there is no 1:1 mapping of concepts to tables.

The tension between normalized database design and intuitive natural language use represents a challenge that influences the architecture and design decisions explored in subsequent sections of this thesis.

3.2 Machine Learning Fundamentals

Machine learning represents the algorithmic foundation for SOTA NL2SQL systems, enabling systems to learn how natural language queries translate into SQL. Understanding these foundations is essential for implementing and optimizing LLM-based NL2SQL approaches.

3.2.1 Neural Network Architecture

Neural networks are the computational concept behind contemporary NL2SQL approaches, representing complex capabilities through compositions of simpler operations. A neural network is comprised of interconnected computational units, called neurons, organized in layers, where each connection has an associated and learnable weight and bias parameters. The computation paradigm of neural network outputs is called forward propagation, which is applying the weights (W) and biases (b) to the input parameter (a) and transforming it using a (non-linear) activation function f:

$$a^{l+1} = f(Wa^l + b) \tag{1}$$

where $W \in \mathbb{R}^{m \times n}$ is the weight matrix, $a \in \mathbb{R}^n$ is the input vector, $b \in \mathbb{R}^m$ is the bias vector, and f is an activation function (e.g. ReLU, sigmoid, or tanh). Using this function repeatedly propagates information through the neural network.

The important architectures for natural language processing networks include:

- **Feedforward Networks** — Which can process fixed-size inputs through successive linear transformations and activations, this architecture is particularly suitable for classification and regression problems within NL2SQL systems.
- **Recurrent Networks (RNNs/LSTMs)** — Which can handle sequential data of variable-length by maintaining hidden states that capture dependencies, enabling processing of natural language sequences of arbitrary length.
- **Embedding Layers** — Which map discrete tokens (e.g. words, characters, or subwords) to a dense vector representation, thus providing the foundation for subsequent neural language processing.

The ability of neural networks to learn hierarchical representations through multiple layers makes them well-suited for the complex translation required in NL2SQL systems.

3.2.2 Learning Approaches

There are three approaches to training a machine learning model, also known as the process of learning:

1. **Supervised Learning** – The process of training a model using a labeled dataset of input and output pairs. This learning approach is commonly used in models used for NL2SQL which were trained on question and answer pairs (natural language question and corresponding SQL query). This learning approach aims to minimise the loss between predictions and ground truth.
2. **Unsupervised Learning** – Includes learning patterns that work with unlabeled datasets and use approaches like clustering, language modeling and heuristics to infer structure and yield predictions. Unsupervised learning is appropriate when there is no clear answer and the domain provides large datasets that are not feasible to label manually (e.g., dataset gathered through webscraping the internet).
3. **Reinforcement Learning** – The process of learning through reward signals, commonly known as the reward function. Applied to NL2SQL systems this could transfer to training a model's SQL generation capabilities based on execution feedback. Historically this learning pattern is not widely used in NL2SQL approaches.

3.2.3 Transformer Architecture

The transformer architecture, introduced by Vaswani et al. in 2017, introduced the model architecture found in modern LLMs. Unlike earlier sequence to sequence models, transformers are capable of processing entire sequences of inputs in parallel by using a self-attention mechanism. This enabled more efficient training on massive datasets and a better representation of complex dependencies within inputs.

3.2.3.1 Attention

Attention allows models to selectively focus on parts of the input by learning to weight different sections of the input sequence based on its relevance to the current computation. While RNNs processed sequences sequentially, attention enables direct modeling of relationships between any positions in a sequence, which enabled the transformer architecture to become prevalent for natural language processing. Attention can be computed by retrieving the weighted sum of values V based on the compatibility between queries Q and keys K:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

where d_k is the dimensionality of the key vectors, used for scaling to prevent softmax saturation which would incur small gradients. The softmax operation converts compatibility scores into a probability distribution, determining how much attention to pay to each value. Attention proved to be a foundational advancement in NLP and therefore recent NL2SQL research. Transformers extended the concept of attention into self-attention and multi-head attention patterns that underpin today's LLMs.

3.2.3.2 Multi-Head Attention

Multi-head attention is an extension of the attention architecture, computing it in parallel on different dimensions which allows models to capture different semantic meanings in the input at the same time:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{3}$$

where each *head* computes:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{4}$$

Each head can specialize in capturing a different type of semantic meaning. For example one head might focus on syntactic structure (e.g., linking SQL keywords to their arguments), while others might capture semantic relationships between words (e.g., list all x where y). This mechanism is essential for the complex tasks which require capturing multiple different semantic relationships between words.

3.3 Large Language Models

LLMs are an advancement built on the transformer architecture for natural language processing, demonstrating SOTA capabilities in understanding, reasoning and text generation. These models form the core of most modern NL2SQL systems like XiYan-SQL, DAIL and others. LLMs have a wide language understanding as they have been trained on large text corpora, are exposed to different domains of training data and have an SQL capabilities as they have been exposed to it during pre-training. This shifted the approach while developing NL2SQL systems to focus on ICL learning methods instead of training dedicated models from scratch.

3.3.1 Feedforward and Residual Connections

Every layer in a transformer combines a multi-head attention with position-wise feedforward networks:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \tag{5}$$

Residual connections and layer normalization stabilize training of complex networks:

$$\text{Output} = x + \text{Sublayer}(\text{LayerNorm}(x)) \tag{6}$$

where Sublayer represents either multi-head attention or the feedforward network. These components enable transformers to scale to billions of parameters while maintaining training stability.

3.3.2 Positional Encoding

Since transformers process sequences in parallel, positional encodings inject information about token positions. Original transformers use sinusoidal functions, while modern LLMs often employ learned positional embeddings or relative position encodings. This positional information is important for NL2SQL systems, where the order of generated SQL clauses (e.g., `SELECT` must come before `WHERE`) carries semantic meaning.

3.3.3 Pre-training and Fine-tuning

LLMs are trained in a two-fold process that separates the training of general language understanding from task-specific capabilities.

3.3.3.1 Pre-training

During pre-training LLMs are exposed to vast amounts of unlabeled text gathered from the internet, books etc. Using self-supervised objectives, no manual labeling of the datasets is required. The most common pre-training objective is causal language modeling, where a model learns to predict the next token given a previous context:

$$\mathcal{L}_{\text{LM}} = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t}; \theta) \tag{7}$$

This objective ingrains general language patterns, common knowledge, reasoning capabilities, and even programming knowledge into models when code is included in the training datasets. Models like GPT, LLaMA, and Qwen are pre-trained using this objective, aiming to predict coherent continuations of text sequences.

3.3.3.2 Fine-tuning

Fine-tuning is the process of adapting a pre-trained model, e.g., for solving NL2SQL tasks, by continuing model training on a labeled dataset (e.g., natural language question to expected sql query). During fine-tuning, the model can learn to map natural language questions to expected SQL queries when being presented with a question and database schema.

Fine-tuning usually requires smaller datasets (thousands to hundreds of thousands of examples) compared to pre-training (billions to trillions of tokens), largely relying on the natural language understanding capabilities acquired during the pre-training phase.

Models like OmniSQL demonstrate that smaller fine-tuned models can outperform larger LLMs for NL2SQL tasks using a fraction of the parameters (H. Li et al., 2025).

3.3.4 In-Context Learning

ICL is the process of teaching a model how to perform a certain task by providing examples in its prompt, without updating models parameters. Thus ICL and fine-tuning differ by their timing: runtime vs learning.

This capability emerged with sufficiently large models which have large enough context windows to fit in examples of solutions for similar problems. This method was shown to be very effective by D. Gao et al. in 2023, improving the performance of general purpose LLMs as well as fine-tuned LLMs when being presented with relevant examples (D. Gao et al., 2023).

3.3.5 Prompt Engineering

Prompt engineering is the process of structuring the input to LLMs in a way where they adhere to desired behaviours. For NL2SQL systems previous research has found diverging effectiveness between different example, schema and question presentation mechanisms (D. Gao et al., 2023; H. Li et al., 2025).

3.3.6 Chain-of-Thought Reasoning

Prompting or training models to output their COT encourages models to generate reasoning traces before producing their final answer to a question or input. For NL2SQL, this usually involves generating explanations of query logic, analysing the input database schema, and planning the query structure before generating a query. Fine-tuned models like OmniSQL are trained to produce COT output during generation in markdown which enables debugging model behaviour and understanding the query generation approach.

3.3.7 Model Limitations

LLMs and the transformer architecture suffer from two primary architectural limitations.

3.3.7.1 Hallucination

As LLMs are predicting sequences of output tokens, they have no underlying understanding of the actual problem domain and solution space they are presented with. Thus in NL2SQL systems, LLMs might generate invalid SQL queries which might reference non existent tables, columns or relationships, output invalid syntax or generate promising but semantically invalid queries.

These flaws are inherent to a text oriented processing model and are non recoverable, although using candidate validation and self-refinement invalid queries can be recovered. Detecting semantic invalidity is a research problem on its own.

3.3.7.2 Context Window Limitations

As the underlying transformer architecture is still bound to finite context windows, extensive database schemas or long lists of examples can exceed the window, thus causing the query generation to fail.

3.4 Natural Language Processing

Natural language processing (NLP) provides the foundations for working with natural language inputs in computer science and mathematics. It allows for text understanding and text representation, text processing and semantic feature extraction, capturing meaning and structure. For NL2SQL systems NLP enables semantic understanding of natural language queries and semantic search using linguistic similarity. Using NLP and text embeddings reference datasets can be used for semantic search.

3.4.1 Text Preprocessing and Tokenization

Before neural networks can process text, the raw text must be converted into structured representations. This transformation is called tokenization, which refers to the technique of segmenting text into discrete units called tokens.

3.4.1.1 Tokenization Approaches

Tokens are short character chains that allow representing text efficiently as a series of tokens by referring to the tokenizers vocabulary. Therefore NLP systems mostly employ subword tokenization. Modern tokenization approaches balance vocabulary size with representational efficiency. Common approaches include **Byte-Pair Encoding (BPE)**, **WordPiece** and **SentencePiece**.

These subword tokenization approaches are valuable for NL2SQL because database schemas often contain domain-specific terminology, compound words, and technical terminology that may not appear in common vocabularies. Subword approaches handle unseen words by decomposing them into meaningful tokens.

3.4.1.2 Tokenization in NL2SQL Contexts

For NL2SQL systems, tokenization must be capable of both encoding NL queries and SQL queries. SQL presents a challenge in NLP as it relies on a structured syntax and usage of special characters (e.g., punctuation, underscores, SQL operators) which are less commonly found in natural language.

3.4.2 Word Embeddings and Semantic Representations

Word embeddings are the vector space representation of tokens where semantic meaning is captured through geometric properties. As word embeddings are ordinary vectors in a high-dimensional vector space, common vector operations can still be performed.

3.4.2.1 Embedding Functions

An embedding function maps tokens from a discrete vocabulary V to dense vectors in $\mathbb{R}^d$:

$$e_w : V \to \mathbb{R}^d \tag{8}$$

where d is the embedding dimensionality (usually between 256-8192 for modern models). The key property of the vector space $\mathbb{R}^d$ is that semantically similar words occupy nearby regions in this vector space, hence vector operations and proximity in the vector space allows for computation of semantic properties such as semantic

distance. Early embedding methods like Word2Vec and GloVe learned static representations where each word has a single vector regardless of its surrounding context. Modern contextual embeddings, generate different vectors for the same word based on its context, thus capturing meaning more accurately.

3.4.2.2 Sentence Embeddings

While word embeddings are useful for determining the relationships between words, they are limited in length and complexity. NL2SQL systems require representations of entire questions and SQL queries in order to discover answers to similar questions asked in the past. Sentence embeddings aggregate the token-level meaning into fixed-size vectors in the same vector space $\mathbb{R}^d$, thus representing complete sentences or texts in one vector.

$$e_s : (v_0, .., v_n) \to \mathbb{R}^d \tag{9}$$

Different pooling strategies emerged, like mean pooling, max pooling, using a CLS token or learned aggregation. Natural uses an embedding model (Qwen3-Embedding-4B) trained specifically for generating high-quality sentence representations for subsequent searching.

3.4.3 Semantic Similarity Metrics

Quantifying the similarity between two natural language sentences is essential for example selection in NL2SQL. This requires that similarity metrics in the vector space align with natural understanding of semantic relation.

3.4.3.1 Cosine Similarity

Cosine similarity is a measurement using the angle between two vectors, to capture their directional similarity while normalizing for magnitude:

$$s_{\cos} = \frac{u \cdot v}{\|u\|\|v\|} = \frac{\sum_{i=1}^{n} u_i v_i}{\sqrt{\sum_{i=1}^{n} u_i^2}\sqrt{\sum_{i=1}^{n} v_i^2}} \tag{10}$$

Thus cosine similarity ranges from -1 which indicates opposite directions to 1 representing same direction, with 0 indicating orthogonality of the input vectors. This metric is widely used in NLP as it is independent of vector magnitude, efficient to compute and well-suited for high-dimensional vector spaces.

3.4.3.2 Cosine Distance

Cosine distance converts similarity into a distance metric:

$$d_{\cos}(u, v) = 1 - \mathrm{s}_{\cos}(u, v) \tag{11}$$

Thus cosine distance ranges from 0 for identical direction to 2 for opposite direction input vectors. This satisfies properties desirable for distance metrics while preserving the angular relationship of the input vectors u and v.

3.5 Graph Theory

Graph theory is a mathematical domain for working with structures and relationships. It is relevant for databases and NL2SQL systems as modeling relationships in databases maps intuitively to graphs. This section introduces the theoretical framework of graphs, graph kernels and graph similarity which are required foundations for the example selection algorithm of Natural.

3.5.1 Graph Fundamentals

A graph is noted as $G = (V, E)$ in its simplest form and consists of vertices V and edges $E \subseteq V \times V$ which are connections between vertices. Labeled graphs are an extension to this concept which associate labels with vertices and edges, encoding semantic information such as vertex types, attributes, or constraints.

3.5.2 Graph Kernels

Graph kernels are functions $k : \mathcal{G} \times \mathcal{G} \to \mathbb{R}$ which are measuring the structural similarity between two graphs:

$$k(G_1, G_2) = \langle \phi(G_1), \phi(G_2) \rangle \tag{12}$$

where $\phi : \mathcal{G} \to \mathcal{H}$ maps graphs to a feature space $\mathcal{H}$, which allows measuring similarity in the feature space, rather than using raw graph matching to compute the similarity and $G_i \in \mathcal{G}$. Thus, on a high level, graph kernels may be used to identify common super structures in graphs.

3.5.3 Weisfeiler-Lehman Algorithm

The Weisfeiler-Lehman (WL) algorithm is a graph kernel algorithm which iteratively refines node labels based on the neighborhood structure in the graph (Togninalli, Ghisu, Llinares-López, Rieck, & Borgwardt, 2019):

1. **Sorting** — Represent vertices as a sorted list of neighbors (L_v)
2. **Compression** — Compute the hash of L_v for every vertex ($h(L_v)$)
3. **Relabeling** — Relabel every vertex v with $h(L_v)$ as its new node label

This algorithm is repeated for w iterations (typically $w = 3 - 5$, due to diminishing returns). After the iterative vertex label computation, labels encode the w-hop neighborhood structure. Subsequently a label histogram can be used as the graph-level feature vector, enabling efficient structural comparison.

3.5.4 Wasserstein Distance

The Wasserstein distance measures the minimum cost of transforming one probability distribution into another (also known as earth mover's distance). Given two distributions μ and ν over space $\mathcal{X}$ with metric d:

$$W_p(\mu, \nu) = \left(\inf_{\gamma \in \Gamma(\mu,\nu)} \int_{\mathcal{X} \times \mathcal{X}} d(x, y)^p \, d\gamma(x, y) \right)^{1/p} \tag{13}$$

where $\Gamma(\mu, \nu)$ is the set of couplings of μ and ν. Unlike other distance metrics of probability distributions like the Kullback-Leibler divergence, the Wasserstein distance accounts for the underlying metric space structure.

3.5.5 Wasserstein Weisfeiler-Leman Kernels

WWL kernels combine the labels computed using WL with Wasserstein distance to measure graph distance (Togninalli et al., 2019):

1. Apply w WL iterations to both graphs, producing node label distributions at each iteration
2. Compute Wasserstein distance between graphs using Hamming distance (for the categorical features) or Euclidean distance (for continuous features) as the ground metric
3. Aggregate distances into a similarity score using a Laplacian kernel:
   $k_{\text{WWL}}(G_1, G_2) = \exp(-\lambda D_W))$

The resulting graph kernel captures both local neighborhood structures, using the WL algorithm, and global distributional properties, using the Wasserstein distance (Togninalli et al., 2019). For database schemas, WWL kernels can capture similar structural patterns (e.g., table structures, possible joins, constraints) while remaining robust to minor structure variations. This makes them well suited for NL2SQL example selection algorithms that factor in schema similarity.

4 System Design

This section outlines the architecture of the NL2SQL system NATURAL, addressing limitations and research gaps identified in the literature review. NATURAL is a pipeline that transforms NLQs into SQL queries consisting of five components:

1. **Example Selection** σ – Identifying semantically and structurally similar example queries using cosine similarity and schema distance.
2. **Schema Subsetting** ϕ – Reducing schema complexity by subsetting to relevant tables and relationships.
3. **Query Projection** π – Using ICL with a fine-tuned model to project NLQs to SQL statements.
4. **Self Refinement** ρ – Refinement of generated SQL statements through execution feedback and error analysis using the underlying model.
5. **Voting** ν – Self consensus voting mechanism to choose the most likely result from multiple generation attempts.

NATURAL processes queries using the following execution flow:

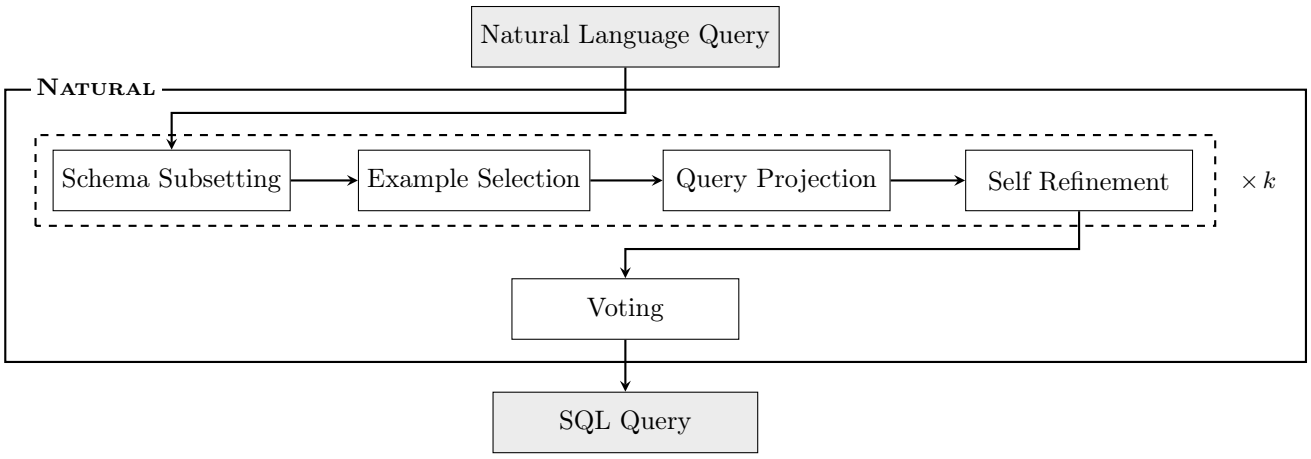

Figure 2: NATURAL (*Full*): All four inner steps execute once per iteration, producing candidates resolved by majority voting.

This design builds upon the ICL architecture from DAIL-SQL (D. Gao et al., 2023) and OmniSQL (H. Li et al., 2025) but incorporates a new schema-aware example selection technique which uses graph kernels for measuring structural similarity between databases.

4.1 Initialization

Before processing queries, NATURAL requires initialization with reference data to construct an embedding space $v \in \mathcal{V}$ and schema distance index $d \in \mathcal{D}$. This preprocessing phase analyzes datasets and previous user interactions to enable schema-aware example selection.

The initialization process consists of two components: (1) embedding generation for semantic similarity computation, and (2) schema indexing for structural similarity measurement. These components support the example selection function σ.

4.1.1 Embedding

NATURAL constructs an embedding space v to enable semantic similarity search (D. Gao et al., 2023). It uses cosine similarity to identify relevant examples based on natural language queries and SQL statements, allowing efficient retrieval from large collections of question-answer pairs.

Therefore, a set of training samples can be embedded $\mathcal{T} = \{(q_1, \omega_1), ..., (q_n, \omega_n)\}$ into an embedding space v, where each q_i is a natural language question and ω_i is the corresponding SQL query.

The embedding space v can be formally defined as:

$$v = \{ \ (q, \iota(q), \omega, \iota(\omega)) \mid (q, \omega) \in \mathcal{T} \ \}$$

where ι is the embedding function that maps text to vector representations.

4.1.2 Schema Indexing

To choose relevant samples for ICL, the SQL queries chosen as examples should use structurally similar database schemas to minimize the structural difference between the selected samples and the ground truth query for a natural language question.

For example, for ω_{ground}, the question "Give me all contacts for the user with the id 10" might look different depending on the database schema, thus selecting samples based only on the similarity of the natural language question will yield inferior sample quality.

```
CREATE TABLE users (
  id TEXT PRIMARY KEY
);

CREATE TABLE contacts (
  user_id TEXT NOT NULL,
  name TEXT NOT NULL,
  FOREIGN KEY (user_id)
   REFERENCES users(id),
  PRIMARY KEY (user_id, name)
);
```

Figure 3: Normalized SQL database schema with separate tables for users and contacts.

```
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  contacts TEXT[] NOT NULL
);
```

Figure 4: Denormalized SQL database schema with contacts stored within an array column.

Given the database schemas in Figures 3 and 4, the respective definitions of ω_{ground} would be 5 and 6.

```
SELECT contacts.name
FROM users
JOIN contacts
  ON contacts.user_id = users.id
WHERE users.id = 10;
```

Figure 5: Using JOIN on a normalized database schema in SQL.

```
SELECT contacts
FROM users
WHERE id = 10;
```

Figure 6: Using array access on a denormalized database schema in SQL.

Figures 5 and 6 show the structural similarity of the database schema is crucial to the relevance of an example.

4.1.2.1 Graph Representation

To determine structural similarity of database schemas, this thesis proposes using graph representation for the representation of database structures. A graph representation allows leveraging established methods for measuring graph similarity such as Wasserstein-Weisfeiler-Lehman graph kernels (Togninalli et al., 2019). Given a database schema s the corresponding graph mapping $G_s = (V, E, \ell, w)$ is defined in Tables 1 and 2.

The structural similarity of two databases is defined as the Wasserstein distance of their graphs. The graph representation captures essential topology through node and edge labeling, prioritizing constraints over

Node Type	Description	Label $\ell(v)$
$v \in V_t$	Tables	1
$v \in V_c$	Generic columns	2
$v \in V_c$	Primary keys	3
$v \in V_c$	Foreign keys	4
$v \in V_c$	Text columns	5
$v \in V_c$	Numeric columns	6
$v \in V_c$	Datetime columns	7
$v \in V_c$	Boolean columns	8
$v \in V_c$	Other columns	9

Table 1: Node types and semantic labels where $V = V_t \cup V_c$ where V_t represents table nodes and V_c represents column nodes

Edge Type	Description	Weight $w(e)$
$e \in E_{tc}$	Ownership	0.5
$e \in E_{fk}$	FK references	0.9
$e \in E_{rel}$	Relationships	1.0

Table 2: Edge types and weights where $E = E_{tc} \cup E_{fk} \cup E_{rel}$

types. This hierarchical labeling ensures that structural relationships take precedence over exact content, while omitting table names, column names and domain terminology for schema-agnostic comparison. The graph representation of the database schemas introduced in 3 and 4 are therefore:

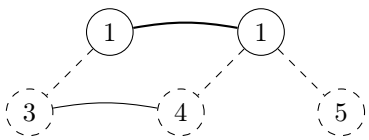

Figure 7: Graph representation of a normalized schema.

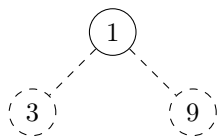

Figure 8: Graph representation of a denormalized schema.

Legend: ○ table ◌ column **-** foreign key – reference - - table-column

The Δ of the order of the graphs 7 and 8 further highlight the structural difference between the two schemas.

4.1.2.2 Distance Measurement

To measure the distance between the graphs shown in Figures 7 and 8, the Wasserstein Weisfeiler-Lehman (WWL) graph kernel method is used (Togninalli et al., 2019). WWL kernels compute structural similarity by combining discrete Weisfeiler-Lehman graph features with continuous optimal transport theory. The method first extracts WL features from the labeled graph structure, then computes the Wasserstein distance between the resulting feature distributions. For two graphs G and G' with semantic node labels, the distance is computed as:

$$D_W^f(G, G') = W_1(f(G), f(G')) \tag{14}$$

The graph representation of the normalized schema G_{norm} (see Figure 7) with 5 nodes and 5 edges and denormalized schema G_{denorm} (see Figure 8) with 3 nodes and 2 edges show significant topological differences. The WWL distance measures the reduced graph complexity (i.e., fewer nodes and edges) and the loss of relational structure (i.e., elimination of foreign key relationships), which results in a higher distance measurement despite modeling logically equivalent data: $D_W^f(G_{norm}, G_{denorm}) \approx 0.78$.

4.1.2.3 Distance Index

The distance index d contains precomputed distances between observed database schemas, which enables efficient retrieval during example selection. This index is constructed as a set of schema-distance tuples $\{(sm_1, di_1), \ldots, (sm_i, di_i)\}$ where sm_i is a database schema name and di_i denotes its WWL distance to the current schema.

4.2 Functions

The following sets are used to introduce the functions σ, ϕ, π, ρ, and ν.

1. $\mathcal{Q}$ – The set of all possible natural language queries.
2. $\mathcal{S}$ – The set of all possible database schemas.
3. $\mathcal{E}$ – The set of all possible execution functions for SQL validation.
4. $\mathcal{V}$ – The set of all possible embedding spaces.
5. $\mathcal{D}$ – The set of all possible distance indices over observed databases.
6. $\mathcal{C}$ – The set of all possible candidate sets $\mathcal{C}'$.
7. $\mathcal{Q}_{\mathcal{S}}$ – The set of all valid queries over the database schema S.

4.2.1 Example Selection – σ

The selection function σ retrieves relevant examples from the embedding space v based on semantic similarity to the input query. This function implements the example selection mechanism that enables few-shot learning for SQL generation.

$$\sigma : \mathcal{Q} \times \mathcal{C} \times \mathcal{S} \times \mathcal{V} \times \mathcal{D} \to \mathcal{C}'$$

For a specific input instance, we write:

$$\sigma(q, C, s, v, d) = \{(q_1, s_1, \omega_1, d_1), \ldots, (q_k, s_k, \omega_k, d_k)\}$$

where $q \in \mathcal{Q}$ is the input NLQ, $C \in \mathcal{C}$ is the current set of generated query candidates, $s \in \mathcal{S}$ is the database schema, $v \in \mathcal{V}$ is the embedding space, $d \in \mathcal{D}$ is the distance index, and the result is a candidate set $\mathcal{C}' \in \mathcal{C}$ containing k tuples of natural language queries, schemas, corresponding SQL queries and their combined distance. The function uses cosine similarity in the embedding space to identify examples that are semantically closest to the input query q, considering natural language representations and schema distance as described in Section 4.1.1. This approach ensures examples are weighted by the following properties:

1. **Question similarity** – Semantic similarity between the input query and observed natural language queries in the embedding space v, measured using cosine distance between their embeddings.
2. **SQL similarity** – Structural similarity between the current best candidate $\hat{\omega} = \nu(C)$ and candidate SQL queries using code representations, enabling pattern recognition across different database domains. On the first iteration $C = \emptyset$ and this term is omitted.
3. **Schema similarity** – Schema compatibility measured through the distance index d, ensuring selected examples operate on analogous database structures with similar table relationships and column types.

The exact weighting of these can be adjusted through three constants, w_q, w_s, and w_d.

4.2.2 Schema Subsetting – ϕ

The subsetting function ϕ reduces the database schema to include tables, columns, and relationships that are relevant to the candidates C. This schema pruning mechanism reduces the complexity of the SQL generation task and improves token efficiency by focusing on relevant schema elements.

$$\phi : \mathcal{C} \times \mathcal{S} \to \mathcal{S}'$$

For a specific input instance, we write:

$$\phi(C, s) = s'$$

Algorithm 1 σ - Example Selection

Require: $q \in \mathcal{Q}$, $C \in \mathcal{C}$, $s \in \mathcal{S}$, $v \in \mathcal{V}$, $d \in \mathcal{D}$
Require: $k \in \mathbb{N}$, $k \geq 1$ ▷ Number of examples to select
$\hat{\omega} \leftarrow \nu(C)$ if $C \neq \emptyset$, else $\perp$ ▷ Best current candidate, if any
$candidates \leftarrow \emptyset$ ▷ Initialize candidate set
for $(q_i, s_i, \omega_i) \in v$ **do** ▷ For each example
  $sim_q \leftarrow cosine(\iota(q), \iota(q_i))$ ▷ Calculate question similarity
  $sim_s \leftarrow cosine(\iota(\hat{\omega}), \iota(\omega_i))$ if $\hat{\omega} \neq \perp$, else 0 ▷ SQL similarity
  $sim_d \leftarrow d(s, s_i)$ ▷ Calculate schema distance
  $score \leftarrow w_q \cdot sim_q + w_s \cdot sim_s + w_d \cdot sim_d$
  $candidates \leftarrow candidates \cup \{(q_i, s_i, \omega_i, score)\}$
end for
return $top_k(candidates)$ ▷ Return k highest scoring examples

where $C \in \mathcal{C}$ is the set of generated query candidates so far, $s \in \mathcal{S}$ is the full database schema, and $s' \subseteq s$ is the reduced schema subset containing only relevant tables and columns. The function analyzes the generated candidates in C to identify which schema elements are referenced across them. On the first iteration $C = \emptyset$ and the full schema s is returned unchanged. This enables focusing on relevant portions of potentially large and complex database schemas, improving accuracy and computational efficiency and preventing exceeding limited context windows.

Algorithm 2 ϕ - Schema Subsetting

Require: $C \in \mathcal{C}$, $s \in \mathcal{S}$
if $C = \emptyset$ **then return** s
end if ▷ No candidates yet: return full schema
$s' \leftarrow \emptyset$ ▷ Empty schema subset
for $table \in s$ **do**
  if $\exists\, r \in references(C, table)$ **then** ▷ Any generated candidate references the table
    $s' \leftarrow s' \cup \{table\}$ ▷ Extend subsetted schema
  end if
end for
return s' ▷ Return subsetted schema

4.2.3 Query Projection – π

The projection function π represents the translation mechanism that converts natural language queries into SQL statements using the selected examples and database schema. This function encapsulates LLM inference that generates candidate SQL queries based on the provided context.

$$\pi : \mathcal{Q} \times \mathcal{S} \times \mathcal{C}' \to \Omega_{\mathcal{S}}$$

For a specific input instance, we write:

$$\pi(q, s, c) = \omega$$

where $q \in \mathcal{Q}$ is the input natural language query, $s \in \mathcal{S}$ is the database schema, $c \in \mathcal{C}'$ is the set of selected examples, and $\omega \in \Omega_{\mathcal{S}}$ is the generated SQL query. The function operates by constructing a prompt combining the natural language query, relevant schema information, and selected examples in a format optimized for LLM inference. The model generates a SQL query capturing the semantic intent of the natural language input while adhering to the constraints of the database schema.

4.2.4 Self Refinement – ρ

The refinement function ρ implements a self-correction mechanism to iteratively improve generated SQL queries by identifying and correcting syntax errors, semantic inconsistencies, and execution failures. This function

Algorithm 3 π - Query Projection

Require: $q \in \mathcal{Q}$, $s \in \mathcal{S}$, $c \in \mathcal{C}'$
Require: $\tau \in \mathbb{R}$, $\tau > 0$ ▷ Relevance threshold
$c_{rel} \leftarrow \{ (q_i, s_i, \omega_i, d_i) \in c : d_i < \tau \}$ ▷ Filter by relevance
$ex \leftarrow \{ fmt(q_i, \omega_i, d_i) \mid (q_i, s_i, \omega_i, d_i) \in c_{rel} \}$ ▷ Format examples
$prompt \leftarrow prompt(q, s, ex)$ ▷ Construct prompt
$out \leftarrow model(prompt)$ ▷ Model inference
$\Omega_{cand} \leftarrow extract_{sql}(out)$ ▷ Extract SQL candidates
for $\omega_{raw} \in \Omega_{cand}$ **do** ▷ Validate candidates
  if $\omega_{raw} \in \Omega_{\mathcal{S}}$ **then** ▷ Check syntactic validity
    return ω_{raw} ▷ Return first valid query
  end if
end for

enables learning from mistakes and producing higher-quality outputs through feedback loops.

$$\rho : \mathcal{Q} \times \mathcal{S} \times \mathcal{E} \times \Omega_{\mathcal{S}} \to \Omega_{\mathcal{S}}$$

For a specific input instance, we write:

$$\rho(q, s, e, \omega_{raw}) = \omega_{refined}$$

where $q \in \mathcal{Q}$ is the original natural language query, $s \in \mathcal{S}$ is the database schema, $e \in \mathcal{E}$ is the execution function for validation, $\omega_{raw} \in \Omega_{\mathcal{S}}$ is the previously generated SQL query and $\omega_{refined} \in \Omega_{\mathcal{S}}$ is the improved SQL query. The function operates by executing the candidate query against the database, analyzing any errors or unexpected results, and prompting the language model to generate an improved version based on identified issues. This process continues until a valid, executable query is produced or a maximum number of attempts is reached.

Algorithm 4 ρ - Self Refinement

Require: $q \in \mathcal{Q}$, $s \in \mathcal{S}$, $e \in \mathcal{E}$, $\omega_{raw} \in \Omega_{\mathcal{S}}$
$exec \leftarrow e(s, \omega_{raw})$ ▷ Verify execution output
$prompt \leftarrow prompt_{refine}(q, s, \omega_{raw}, exec)$ ▷ Construct refinement prompt
$out \leftarrow model(prompt)$ ▷ Generate refinement output
$\Omega_{ref} \leftarrow extract_{sql}(out)$ ▷ Extract refined candidates
for $\omega_{ref} \in \Omega_{ref}$ **do** ▷ Validate refined queries
  if $\omega_{ref} \in \Omega_{\mathcal{S}}$ **then** ▷ Check syntactic validity
    return ω_{ref} ▷ Return first valid refinement
  end if
end for

4.2.5 Voting – ν

The voting function ν implements a consensus mechanism that selects the most reliable SQL query from multiple candidates generated through the pipeline. This function enhances robustness by leveraging result distribution of multiple attempts to identify the most likely correct answer.

$$\nu : \mathcal{C} \times \mathcal{E} \to \Omega_{\mathcal{S}}$$

For a specific input instance, we write:

$$\nu(C, e) = \omega_{consensus}$$

where $C \in \mathcal{C}$ is a set of candidate SQL queries, $e \in \mathcal{E}$ is the execution function for validation, and $\omega_{consensus} \in \Omega_{\mathcal{S}}$ is the selected consensus query. The voting function ν implements a majority voting algorithm similar to

OmniSQL (H. Li et al., 2025), where the most frequent result across multiple attempts is deemed the most likely correct answer. The function applies frequency-based selection among valid candidates. In cases where no clear majority exists, the function may apply additional heuristics such as query complexity or execution performance to determine the final query candidate $\omega_{consensus}$.

Algorithm 5 ν - Consensus Voting

Require: $C \in \mathcal{C}$, $e \in \mathcal{E}$

```
results ← {}                                        ▷ Map from result sets to candidate lists
for ω ∈ C do                                        ▷ Execute each candidate
    result ← e(ω)                                   ▷ Execute query
    results[result] ← results[result] ∪ {ω}
end for
dist ← {results[r] : r ∈ results}                   ▷ Get all result groups
dist ← sort(dist, |group| ↦ |group|)                ▷ Sort by group size (largest first)
return top_1(top_1(dist))                           ▷ Return largest group
```

4.3 Composition – nq

The nq function composes the system by iteratively subsetting the schema, selecting relevant examples and generating query candidates, self-refining results through execution feedback, and yielding the most likely candidate ω through majority voting.

$$nq : \mathcal{Q} \times \mathcal{S} \times \mathcal{E} \times \mathcal{V} \times \mathcal{D} \to \Omega_{\mathcal{S}}$$

For a specific input instance, we write:

$$nq(q, s, e, v, d) = \omega$$

where $q \in \mathcal{Q}$ is the input natural language query, $s \in \mathcal{S}$ is the database schema, $e \in \mathcal{E}$ is the execution function for validation, $v \in \mathcal{V}$ is the embedding space, $d \in \mathcal{D}$ is the distance index, and $\omega \in \Omega_{\mathcal{S}}$ is the generated SQL query. The nq function implements Algorithm 6.

Algorithm 6 nq - Natural Query Pipeline

Require: $q \in \mathcal{Q}$, $s \in \mathcal{S}$, $e \in \mathcal{E}$, $v \in \mathcal{V}$, $d \in \mathcal{D}$

Require: $k \in \mathbb{N}$, $k \geq 1$ ▷ Number of candidates to generate

```
while |C| < k do                                    ▷ Generate k candidates
    s′ ← φ(C, s)                                    ▷ Subset schema using generated candidates
    C′ ← σ(q, C, s, v, d)                           ▷ Select examples using current candidates
    ω ← π(C′, q, s′)                                ▷ Generate candidate query
    if e(ω, s′) then                                ▷ Validate syntactic correctness
        C ← C ∪ {ω}                                 ▷ Add valid candidate
    end if
    ω′ ← ρ(q, s′, e, ω)                             ▷ Attempt self-correction
    if e(ω′, s′) then                               ▷ Validate corrected query
        C ← C ∪ {ω′}                                ▷ Add corrected candidate
    end if
end while
return ν(C, e)                                      ▷ Select consensus result
```

5 Implementation

This section discusses the implementation phase of NATURAL, translating the system design outlined in Section 4 into a working system. Engineering challenges encountered, performance bottlenecks identified, and optimization strategies implemented to achieve deployment viability on consumer hardware are discussed and examined. Furthermore, the section covers software architecture, infrastructure, and implementation of each pipeline component.

5.1 Architecture and Infrastructure

The architecture is split into inference, sampling, and evaluation. The runtime code focuses on algorithm implementations outlined in Section 4, sampling code focuses on indexing examples and computing distance indices, and evaluation code runs the NATURAL pipeline on benchmarks. This subsection outlines the scope of each component, the design rationale, and technology stack decisions.

5.1.1 Software Architecture

NATURAL consists of five software components:

1. `natural-models` – Handling model execution on GPUs for inference and embedding.
2. `natural-graphs` – Graph library for representing database schemas as graphs and computing graph similarities and distances.
3. `natural-inference` – Core pipeline implementation using other software components during inference.
4. `natural-sampling` – Sampling setup for indexing samples and computing graph distance indices.
5. `natural-benchmark` – Benchmarking setup to run experiments, evaluate system accuracy, and compile statistics.

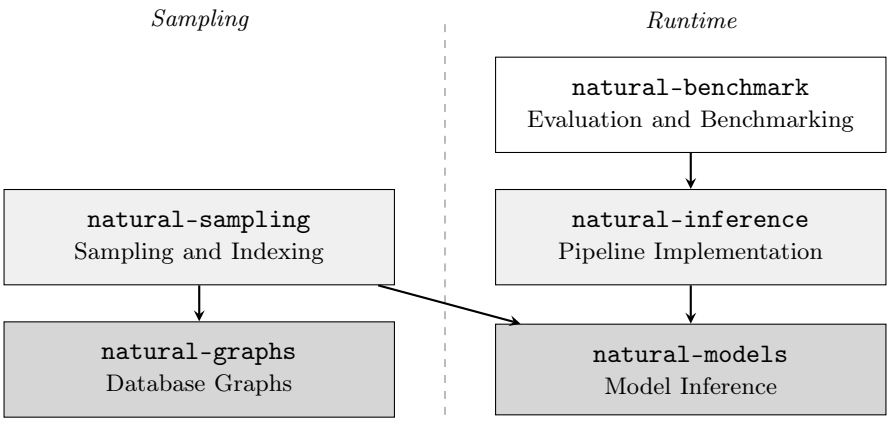

Figure 9: Software architecture of NATURAL: dependency graph between the five Cargo crates. Arrows point from a dependent crate to the crate it depends on.

This separation emerged as the split between inference, sampling, and evaluation code required shared fundamentals like model execution and graph representation which improves testability and maintainability. Separating the inference, sampling, and evaluation code yields a smaller footprint when embedding inference code into databases. Challenges with this multi-component architecture are tied to dependency management and build complexity although these can be mitigated through `cargo`'s workspace support.

5.1.2 Resource Management Strategy

Given the 24GB of VRAM capacity of the RTX 3090, not all model sizes can be used. While 14B model variants theoretically work, the need for an embedding model typically exhausts VRAM, except when using steep quantization formats. OmniSQL 7B models show an average performance degradation of only 0.3% compared to their 14B counterparts by H. Li et al. in 2025. On the spider test dataset the 7B model outperformed its 14B counterpart by 0.6%. Due to unclear performance gains of the 14B model, the significant increase in inference time and limited VRAM, this thesis focuses on the 7B variant of OmniSQL.

As an embedding model is required for semantic search of samples in the pipeline, a companion model is required to embed the sample datasets (SynSQL, Bird and Spider) during sampling and the user provided NLQ during inference. The Qwen3 embedding models ranked first on the MTEB multilingual leaderboard. The Q8_0 quantized version of the 8B model consumes 8GB of VRAM which cannot fit into 24GB when accounting for KV-Cache requirements. Instead of choosing a heavily quantized version of the 8B variant (i.e., Q4_KM or below), the 4B model with Q8_0 shows similar performance on simple use cases while offering a smaller memory footprint at 2GB (Y. Zhang et al., 2025).

As models have significant loading times, the system loads models globally at startup and passes references using atomic reference counting to ensure models are not loaded twice, can be reused between inference calls and GPU memory is not exhausted which would lead to a crash.

5.1.3 Technology Stack Decisions

The choice of programming language, inference frameworks, and libraries determines system performance, deployment characteristics, and development velocity. Each decision was evaluated with available hardware resources, database integration capabilities, and research objectives in mind.

5.1.3.1 Language and Ecosystem

The language and ecosystem choice significantly impacts the development of any system. Viable languages for implementing NLP and ML-heavy systems are Python, R, Julia, Rust, C / C++ and Java and other general purpose languages. While interpreted languages like Python, R and Julia tend to be easier to use for rapid prototyping and validation due to their flexible type systems and scientific ecosystem, they come with drawbacks regarding deployability, speed, and robustness compared to compiled and strongly typed languages. Languages like Java, C / C++ and Rust offer stability at runtime, better interoperability into other programs (e.g., database extensions) and resource utilization.

R and Julia are less adopted than Python, machine learning frameworks, and NLP. Java requires the Java Virtual Machine (JVM) at runtime which yields a higher overhead than C / C++ and Rust while offering little advantage over interpreted languages like Python. Thus, Python, C / C++, and Rust are viable languages for Natural. C and C++ are prone to crashes and memory safety issues whereas Rust resolves most of these downsides while maintaining similar performance, memory management and portability characteristics.

Given that performance plays a critical role when developing machine learning systems with limited hardware access, the path to production deployments of NL2SQL is easier and the language ecosystem offers bindings for notable scientific libraries, Rust offers value for ML systems at the cost of development speed.

5.1.3.2 Inference Framework

For local model inference, two frameworks emerged: llama.cpp (FFI) and Candle (Rust-native). While Rust-native frameworks offer better portability, llama.cpp provided a better balance between performance, ecosystem maturity and implementation simplicity. llama.cpp provides out-of-the-box optimizations with comprehensive GGUF quantization support, yielding approximately 25-30 tokens per second on consumer RTX 3090 hardware. It offers high-level abstractions for model loading, tokenization and sampling. The GGML ecosystem and advanced quantization schemes (like `Q4_K_M`, `Q8_0`) justified the FFI complexity. Candle, even though it offered comparable inference speeds, required extensive low-level implementation work and proved incompatibility with scenarios where the binary is embedded into a database (e.g., PostgreSQL). The combination of manual tensor management, convoluted quantization support, and deployment constraints made llama.cpp a better alternative for production-ready systems.

5.1.3.3 Similarity Search Framework

For lightweight similarity search, SQLite with its `sqlite-vec` extension is a sensible option. For smaller data loads introducing the complexity of `faiss` and vector search implementations like `qdrant` outweighs their speed benefits. The `sqlite-vec` extension is a "fast enough" vector search implementation, allowing reduced complexity and compatibility with graphical database interfaces that support SQLite to inspect the embedding space.

5.2 Pipeline Implementation

This section discusses the implementation and integration of each component (σ, ϕ, π, ρ, ν) into NATURAL.

5.2.1 Example Selection (σ)

The example selection algorithm is used to select a set of examples from the vector database for ICL and is implemented in the `natural-sampling` crate.

5.2.1.1 Distance Computation

The algorithm implemented in `natural-sampling::selection` computes three distinct distance metrics. The `selection` module computes semantic similarity through cosine distance of the question embedding. Structural SQL similarity is computed using the cosine distance of the SQL embedding, and schema distance using WWL kernel distance from the `natural-graphs` component. The embeddings are computed using the Qwen3-Embedding-4B model using the embedding implementation in `natural-sampling`.

5.2.1.2 Weighting Strategy and Performance Characteristics

The three dimensions of similarity (question, query, structural) are combined into one distance metric through three weights: w_q for question distance, w_s for SQL distance, w_d for schema-level distance. The selection algorithm yields a maximum of n candidates (e.g., `TOP_K = 256`) to limit overhead whilst maintaining good selection quality. The cosine similarity is computed using `sqlite-vec`. The exact weights can be optimized independently depending on the deployment scenario.

5.2.1.3 Implementation Architecture

The `Selector` struct contains the embedding computation logic for the selection algorithm. Via `Selector::new` the code can compute the embeddings and use them to run example selection.

The `selection` algorithm follows a two-stage approach: initial candidate retrieval through vector search (using the `Vector` struct and SQLite) and reweighting using the weights and a precomputed WWL distance index from `natural-graphs`.

5.2.2 Schema Subsetting System (ϕ)

The schema subsetting system is implemented in `natural-inference::pipeline::subsetting` in the `SchemaSubsetter` struct. The core algorithm is implemented within the `optimize` function which performs execution based validation to determine which tables are required for the query candidates.

5.2.2.1 Query Validation

Query validation employs a trial-and-error approach where, for each query candidate and table in the schema, an in-memory SQLite database is created using `Connection::open_in_memory()`. The connection is initialized using the schema with all tables except the current one. Once the connection is ready, `SchemaSubsetter` prepares every query candidate against this reduced schema. If preparation fails, the table removed is required for the query, thus it gets added to the list of subsetted tables. This process is repeated for all query-table combinations to build a minimal schema containing only essential tables for execution of the query candidates provided to the subsetting algorithm.

5.2.2.2 Performance and Trade-off Characteristics

As a new in-memory database for each table-query combination is created, this leads to $O(queries \times tables)$ complexity. However, this execution-based approach provides superior correctness guarantees since heuristic approaches cannot determine whether candidates will execute successfully in database environments. Using a SQLite in-memory instance both the correctness of query candidates and the referenced set of tables can be known prior to query execution.

5.2.2.3 Pipeline Integration

`SchemaSubsetter` is used prior to prompting the model using ICL to ensure model context is used efficiently and attention is given to relevant parts of the schema. The `optimize` method returns a `Schema` object containing only tables identified as required through execution testing which is processed by subsequent pipeline stages like generation and refinement.

5.2.3 Query Projection (π)

The ICL module implements the query projection algorithm described in Section 4.2.3. It is implemented in the `ICLGenerator` struct and wraps the `SqlModel` struct from `natural-models`. Using llama.cpp it runs model inference using a prompt optimized for ICL with OmniSQL.

5.2.3.1 In-Context Learning

`ICLGenerator::generate` implements few-shot learning with relevance filtering which removes examples with a similarity of less than 0.5. Thus only semantically or structurally similar examples are provided to the model. The prompt is constructed based on an adapted version of the OmniSQL format (H. Li et al., 2025): task overview, sql schema, filtered examples with similarity scores, explicit instructions, and target question. The biggest differentiation from H. Li et al.'s prompt is the example section including similarity. For SQL query presentation the code representation prompt format is used, inspired by DAIL-SQL (D. Gao et al., 2023). Every example includes the question, similarity score, and formatted SQL query.

5.2.3.2 Prompt Engineering Strategy

OmniSQL prompt was used as base for Natural's prompt with a code representation prompt. Natural uses more explicit instructions (see Appendix A.1.1) compared to DAIL-SQL. Key differentiations include the encoding of similarity scores to give the model the ability to weight examples.

This prompt steers the model towards precision and COT reasoning. It addresses LLM issues like verbosity, over-complexity or missing query constraints, and hallucination and accuracy concerns identified in the literature review where LLMs generate plausible but incorrect SQL queries.

5.2.3.3 Model Integration and Performance

`SqlModel` is wrapped around `llama-cpp-2` bindings and loaded globally at startup to avoid 30-60 second initialization times per query. Using the `prompt` method from `natural-models`, tokenization, inference, and decoding with configurable PromptParams for context size and generation limits is handled automatically.

5.2.3.4 Output Processing and Validation

As OmniSQL was finetuned to output thoughts and predictions using markdown, a postprocessing module is needed, and a module to identify whether a SQL query is syntactically valid.

To extract candidates, all generated responses are post-processed through a markdown parser (`pulldown-cmark`) which parses the model output and looks for code-block fence characters ```. All candidates are processed in reversed order and the first valid query is returned as a candidate. Reversing processing order is needed as the model outputs thoughts top-to-bottom as a markdown document with the answer usually being output at the end as OmniSQL is trained to use COT. This approach ensures returned query candidates are executable and contain valid SQL. Natural addresses the limitation of LLMs when it comes to generating perfect SQL queries by implementing recovery and refinement mechanisms in the subsequent pipeline.

5.2.4 Self-Refinement Mechanism (ρ)

The self-refinement mechanism described in Section 4.2.4 is implemented in `natural-inference::pipeline::refinement`. This implementation corresponds to the ρ function, providing error correction through execution feedback and iterative improvement of the generated SQL queries.

5.2.4.1 Error Correction

The `Refinement::optimize` method takes a `RawQueryCandidate` and attempts to improve it through reprompting the underlying model. Contrary to the ICL generation which uses few-shot examples, the refinement process focuses on single-query self-refinement with explicit instructions on how to fix errors, thus allowing the model to perform quality assurance on its own output.

5.2.4.2 Correction Prompt

The refinement prompt uses the same structured format as the ICL generation but with a few key differences: the prompt explicitly instructs the model to "spot any errors in the SQL query, correct them" and provides the original question, schema context, and the candidate query that needs improvement (see Appendix A.1.2).

5.2.4.3 Model Resource Management

Given that both the query projection and the self-refinement mechanism require an LLM with SQL generation capabilities, the same underlying `SqlModel` is used from the shared pipeline context rather than loading two separate models into memory. This significantly reduces the VRAM requirements but potentially limits the refinement capabilities compared to having a dedicated quality assurance model trained on different data. Due to the limited available hardware the effects of having a separate, distinct refinement model could not be verified. Y. Gao et al. have implemented multi model generation pipelines in 2025 and achieved promising results.

5.2.4.4 Output Processing and Validation

Similar to query projection, the output of the self-refinement component are using the same `Markdown` parser to extract the SQL query. The system validates each extracted candidate by trying to parse it using `QueryCandidate::try_from` and returns the first syntactically valid refinement starting from the bottom. This ensures that refinement produces executable SQL whilst falling back gracefully if refinement fails.

5.2.4.5 Pipeline Integration

The self-refinement module is integrated into the main pipeline after the ICL generation and processes its unvalidated `RawQueryCandidate` output. Notably both the unrefined and the refined query candidates are kept in the candidate set for subsequent majority voting.

5.2.5 Consensus Voting System (ν)

The `natural-inference::pipeline::voting` implements the majority function ν – this implementation closely follows the design and algorithm outlined in Section 4.2.5. The function `voting` providing a result-based self-consensus mechanism which takes in the set of candidates that were predicted during the generation phase and returns the most likely query.

H. Li et al. have shown in 2025 that self-consensus can significantly improve the accuracy of models that are already showing state-of-the-art performance. The result-based self-consensus mechanism is executing every query candidate, verifies that it works on the database, and loads its results. By partitioning the available candidates into buckets based on their hashed result, queries can be deemed semantically equivalent if they end up in the same bucket. After every query has been executed and partitioned, the algorithm groups the buckets in a hash map `buckets` with the result hash as key and bucket as value. The heuristic method employed is steering the pipeline to agree with itself, if multiple generation attempts yielded the same result, these generation attempts are more likely to be accurate than others.

5.2.5.1 Error Handling and Fallback

Queries that fail execution are contained in an `errors` `HashSet` rather than being included in voting buckets. If all candidates fail execution, the voting function fails with an error that contains all collected execution errors to provide diagnostic information to the calling side.

5.2.5.2 Limitations and Trade-offs

The result-based partitioning approach has limitations with regard to subtle errors that differ by few rows, as these would be treated as completely different result buckets. Furthermore executing every query candidate against the real database comes along with a significant performance penalty for expensive queries where the voting step might incur load onto the database, use significant amounts of memory for loading in all data into memory and slow down the voting as the entire result needs to be hashed. This system lacks model uncertainty calibration for SQL confidence scoring which could be used as another dimension for partitioning.

Another more advanced optimization could be to group by result-schema and row count, which still needs to execute the candidate queries partially but does not load the actual data from the database into memory. For the scope of this thesis and small real-world application scenarios, the cost of this stage is negligible compared to the llm inference done in previous steps.

5.2.5.3 Pipeline Integration

The voting mechanism represents the last step of the pipeline function and selects the final consensus from all generated and refined candidates, and ultimately returns the output of the Natural system.

5.3 Supporting Systems and Optimizations

The supporting systems of Natural are important tools to simplify the actual pipeline development, employing performance optimizations and enable rapid development of new approaches and hypotheses. The primary support systems are `Vector` – a vector database abstraction on top of SQLite, `wwl` – a library that implements Wasserstein-Weisfeiler-Leman kernels (Togninalli et al., 2019) and the `natural-sampling` module to compare and semantically search the embedding space.

5.3.1 Vector Database

The vector database implementation `Vector` is used both during sampling to construct the vector db and runtime to run the example selection algorithm. Using SQLite and `sqlite-vec` to implement cosine search in text corpora of up to a few million examples dramatically simplified the development of Natural as no manual performance optimizations had to be implemented. SQLite has a second architectural benefit besides relatively good performance, which is the portability characteristics of having all examples indexed in a single file on disk. This enables swapping out the respectively used example by using another `.vector` file on disk. See Appendix A.2.2 for reference.

5.3.1.1 Database Schema

As the `.vector` file is still a regular SQLite database, alongside indexing tables using embeddings (`float[4096]`), regular database tables can be maintained to save data derived at sampling time. `Vector` hosts the database schema described in Appendix A.2.2.1. Which maintains the precomputed distance indices and the database graphs for example selection. Thus at pipeline runtime the database indices and graph representations can be reconstructed through a cheap selection from `Vector` even though they are not indexed through embeddings.

The schema of the `samples` table largely enables the example selection at runtime, it maintains embeddings of the sql query and natural language question respectively. Thus, after cosine similarity search, the original query or question can be reconstructed as well as the graph layout of the underlying database can be accessed through the `database` table. The `database` and `database_indices` table contain `JSON` columns respectively for serializing complex graph and graph index structures to the database.

5.3.2 Embedding

The embedding functionalities in `natural-sampling` abstract batch embedding of questions and SQL queries using the GPU so that the runtime and sampling code can efficiently compute and use embeddings.

5.3.2.1 Architecture

The primary type for embedding text is the `SemanticString` struct which holds strings alongside their embedding vectors. Furthermore this struct offers `embed_seq` and `embed_chunked` methods which can be used for batch computation of embeddings.

5.3.2.2 Batch Computation

During sampling, offloading data to the GPUs memory is an expensive operation. When sampling hundreds of thousands or millions of examples, incurring the IO roundtrip from CPU to GPU for computing a single embedding is a performance bottleneck that increases the sampling time from hours to days. Batch processing addresses this by minimising the IO roundtrips needed between the CPU and GPU before computing embeddings and better utilise the available computing power.

Using the `embed_seq` method, a sequence of strings can be embedded and a sequence of `SemanticStrings` is returned. Furthermore through the `embed_chunked` method, chunks of unrelated data can be embedded in a single operation. The method maintains the original chunk layout which makes it possible to reconstruct the input semantic seamlessly. This optimization is used to embed the questions and sql queries alongside while maintaining clear separation in the embedding output.

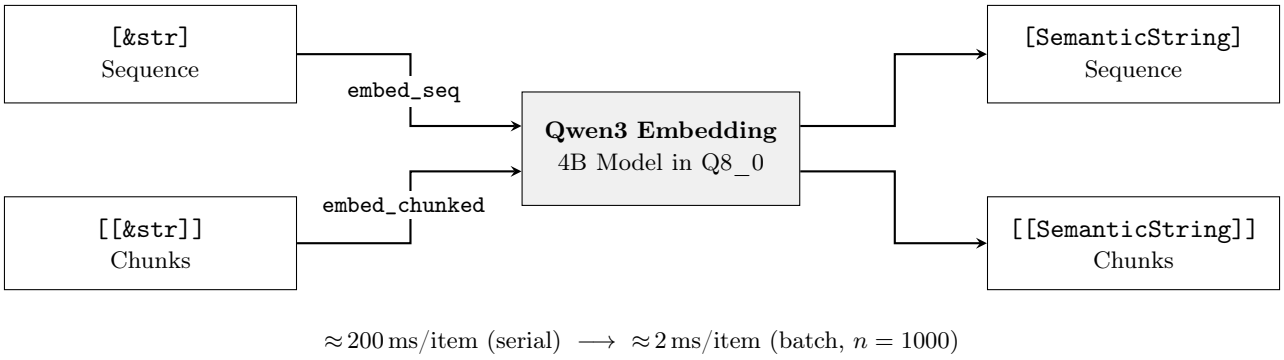

Figure 10: Batch embedding data flow. Inputs are either flat sequences (`embed_seq`) or pre-chunked slices (`embed_chunked`); the chunked path preserves the original layout for seamless reconstruction. Batch processing reduces per-item embedding time from $\approx 200\,\text{ms}$ to $\approx 2\,\text{ms}$.

5.3.3 Wasserstein-Weisfeiler-Leman Kernels

WWL kernels are used to construct the distance index $d \in \mathcal{D}$ for the σ function (see Section 4.2.1) by computing schema distance through graph-based distance metrics. The underlying WWL implementation used by Natural is the reference implementation of WWL kernels from Togninalli et al., which is implemented in Python. Thus Natural cannot directly use the WWL implementation as Rust and Python FFI is not directly possible. In order to utilize the Python libraries and ecosystem (e.g., optimal transport) and the existing implementation of the WWL kernels, writing a thin layer of Rust bindings around Togninalli et al.'s implementation was deemed most sensible. Using `pyo3` calling Python code is made possible and lets Natural hook into existing ecosystems where needed. Due to the design laid out in Section 4 the WWL implementation is only needed during sampling time and not required at runtime as all distances are precomputed and stored in a distance index, keeping the runtime portability characteristics of using Rust that were discussed above.

5.3.3.1 Rust-Python Integration Architecture

The WWL kernel library used by Natural is implemented using `pyo3` around the existing Python `wwl` library. The Rust bindings are usable through the `wwl` crate. The `WWLKernel` struct encapsulates the Python module reference and provides type-safe interfaces for both categorical and continuous propagation modes through respective methods. Using an existing graph library (`petgraph`), the complexity of the Rust bindings could be kept minimal, but required conversion from Rust's `petgraph` graph representation to Python's `igraph` graph representation.

5.3.3.2 Schema Graph Representation

The `natural-graphs` library implements the schema graph representation discussed in Section 4.1.2.1 by parsing SQL statements using the `sqlparser` library and constructing an undirected `petgraph` graph instance. Nodes represent tables and columns and edges capture foreign key relationships and column ownership. Node labeling is used to encode schema constraints hierarchically as outlined in Section 4.1.2.1.

5.3.3.3 Propagation Mechanisms

The `wwl` crate supports both categorical and continuous Weisfeiler-Leman propagation schemes. Categorical propagation operates on discrete node labels through iterative label refinement, suitable for constraint-based schema comparison. Continuous propagation utilizes node features encoded in matrices, enabling similarity computation based on quantitative schema properties like column cardinalities or data type frequencies (Togninalli et al., 2019). As Natural projects the database schema into a graph representation with categorical node labels, the categorical propagation scheme is used in `natural-graphs` to determine schema distance. The `wwl` bindings allow configuration of the kernel through the `KernelConfig` and `DistanceConfig` structs, which provide control over the iteration count (defaulting to 3), sinkhorn approximation settings etc.

5.3.3.4 Distance Computation and Caching

The pairwise Wasserstein distance is computed between the current database schema and each sample database during sampling phase. The distance index is a `HashMap` of a sample database name to its distance to the current database Natural is running on. Caching of this computation is an efficient strategy to minimise runtime inference time as the distance index is static as long as database schemas are not mutated. Natural is storing JSON-serialized distance indices in `Vector` for fast distance lookups at runtime. This design prevents the computational cost of WWL distance computations from affecting inference, reducing runtime schema comparison to constant-time lookups for all previously indexed database combinations that were known during sampling time. Given that the set of sample database and target databases are usually fixed, the distance lookup becomes negligible in terms of computational cost.

5.3.3.5 Integration with Example Selection

The WWL kernel is integrated in the sampling phase, where distance indices are computed and cached. Furthermore during example selection (σ) the cached distance index is loaded from `Vector` for distance lookups.

The distance index is used for weighting the schema compatibility in the selection algorithm (see Section 4.2.1) to balance structural similarity against question-level semantic relevance for improved ICL effectiveness.

5.4 Benchmarking Infrastructure

Natural's benchmarking infrastructure is implemented in `natural-benchmark`. This benchmarking infrastructure is enabling the development, verification and ultimately deployment of Natural pipeline. This section focuses on the respective benchmarking infrastructure development, performance optimizations that were required and engineering challenges encountered. Whilst Natural's benchmarking infrastructure is not part of the core pipeline, it helps to develop and test hypotheses for extensions or design changes confidently, compare performance across pipeline versions, models and understand the benchmark performance.

5.4.1 Execution-Based Evaluation System

The `natural-benchmark` CLI evolves around the concept of executions which are single benchmark runs against a specific benchmark dataset (defaulting to Spider). An execution is the set of tests that have been executed using a version of Natural against a benchmark. The interface of `natural-benchmark` offers multiple options to create executions and subsequently understand Natural's performance:

1. Running new benchmarks via:
   `natural-benchmark new`

2. Continuing a halted execution via:
   `natural-benchmark continue execution <id>`

3. Comparing performance on previous (partial) executions via:
   `natural-benchmark compare-to previous <id>`
4. Computing statistics on past executions via:
   `natural-benchmark stats execution <id>`

While benchmarking `natural-benchmark` maintains a local SQLite database maintaining a history of past performance, pipeline failures etc. for future introspection as well as comparison of approaches.

5.4.2 Cross-Dataset Validation

In order to measure the performance of NATURAL against multiple benchmarking datasets, the benchmarking and execution system in `natural-benchmark` must generalize across one dataset. The benchmarking setup achieves this through a set of Rust traits (e.g., type characteristics) to model benchmarking datasets.

The traits `Benchmark`, `BenchmarkDatabase` and `BenchmarkTest` allow to abstract the file system layout of different evaluation datasets. Thus when implementing benchmark support against SPIDER the implementation abstracts the fact that SPIDER is using SQLite for test databases, and uses a JSON file to store the questions. The storage layout of SPIDER and BIRD is largely similar but not identical.

The trait system (see Appendix A.2.2.2) allows integrating further benchmarks in the future (e.g., SPIDER2 which relies on cloud databases like Snowflake) while maintaining the same ergonomics and tooling across benchmark datasets (e.g., recording and indexing all test executions in a local database, recording pipeline logs etc.).

5.4.3 Metric Computation

The benchmarking infrastructure computes the metrics execution accuracy, exact match, candidate latency and error rate (see Section 6.1.3). The execution accuracy and exact match are found in prevalent benchmarks (e.g., SPIDER and BIRD) and allow cross study comparison. SPIDER determines the execution accuracy by comparing the result sets of two queries based on possible row and column permutations as well as optional order sensitivity. The exact Match metric is determined by exact equivalence of the ground truth query ω_{ground} and the candidate query ω.

5.5 Engineering Challenges and Lessons Learned

The implementation approach of NATURAL highlighted significant challenges regarding practicability that were independent of the algorithmic design and theoretical problems. The development process was significantly slowed down due to constrained hardware availability, technology stack decisions, and research methodology. The overall system architecture turned out positive, but the timeline was noticeably expanded. This section summarizes these challenges and analyzes how hardware limitations influenced design decisions, confidence in the performance of NATURAL and the overall impact on development velocity.

5.5.1 Hardware Constraints and Development Velocity

Likely the most significant circumstance that affected development velocity was the constrained access to high performance hardware. The hardware available during development time was the RTX 3090 with 24GB of VRAM. This limitation fundamentally shaped the development process and research methodology as the iteration speed was severely slowed down.

5.5.1.1 Benchmarking and Evaluation Bottlenecks

A full benchmark execution of NATURAL against the SPIDER test dataset requires around 12-36 hours depending on the number of candidates generated and whether the refinement mechanism is used during the benchmark. This made continuous validations during development a non trivial task, requiring extensive evaluation phases between times of active development. Testing new hypotheses, validating algorithmic changes, experimenting with the implementation approaches, comparing different configurations and sampling strategies therefore became multi-week processes rather than the rapid iteration typically desired in research environments. The prolonged evaluation cycles created a cascading effect on development velocity. Rather simple adjustments that

would normally be validated within hours required days of evaluations, effectively preventing fast experimentation. The hardware constraints forced a rather conservative approach to experimentation, where each change needed to be carefully considered before committing to multiple days of benchmarking.

5.5.1.2 Memory Management and CUDA Issues

Frequent CUDA out-of-memory (OOM) errors made the evaluation interpretation non trivial as accuracy scores became inaccurate due to pipeline failures. This persisted as a second development challenge throughout the implementation phase. These memory exhaustion issues occurred rather unpredictably during both development and benchmarking, depending on the database schema and question asked. As Natural uses a code representation prompt, the database schema is inlined into the prompt during inference. If Natural is executed against large databases, even with schema subsetting, the context window can be exceeded or the KV-Cache can exhaust the rest of the available GPU memory (typically around 2-4GB) after models have been loaded. This required extensive trial-and-error debugging to identify the root causes and implement workarounds.

5.5.2 Technology Stack Trade-offs

During the implementation of Natural, several critical technology stack decisions fundamentally impacted both development velocity and the final system characteristics. The two most consequential decisions were the choice of programming language and the model deployment strategy present trade-offs between velocity and robustness.

5.5.2.1 Programming Language: Rust vs Python

The decision to implement Natural in Rust rather than Python represents a notable architectural choice made during the implementation phase. Choosing Rust allows a trivial port of the system into production environments, but comes at the cost of developing benchmarking infrastructure and not being able to exactly reuse the inference code used by OmniSQL. This decision ultimately increased the implementation time, likely doubling it compared to Python. Using Python would have provided immediate access to existing NLP and ML infrastructure and existing implementations could have been used directly instead of developing abstractions for model inference, vector database operations, and benchmarking infrastructure.

However, the Rust implementation provides significant benefits for production-grade deployments and system integration. The resulting system can be embedded directly into DBMS such as PostgreSQL using extensions, which would be impractical when using Python. The ability to compile an entire NL2SQL system into a single, standalone binary offers better portability.

5.5.2.2 Model Deployment: Local vs Remote Inference

Using local and open-source models reemphasized the limited local hardware access as using an inference provider would have significantly speeded up the benchmarking time. Since a primary research goal of this thesis is to explore the capabilities of offline-only NL2SQL systems, choosing a proprietary inference service was deemed unviable.

5.5.2.3 Integration Complexity and Ecosystem Maturity

The NLP and ML ecosystem of Rust showed to be substantially less mature than anticipated, requiring extensive development and research for functionality that would be available off-the-shelf Python packages. The integration with Python libraries for specialized components like the WWL kernels required rather complex FFI code using `pyo3`, adding complexity and potential stability concerns. Despite these challenges, the resulting architecture demonstrates that multi language approaches can be viable when different components have distinct requirements. The Python integration was isolated to the sampling phase, preserving the runtime portability characteristics of the core Rust implementation while still leveraging existing, specialized libraries where appropriate.

5.5.3 Lessons Learned and Future Recommendations

The implementation of Natural showed multiple practical challenges associated with researching and implementing NL2SQL systems. The approach used in this thesis highlights several areas of consideration for future research projects in this field.

5.5.3.1 Hardware Infrastructure Requirements

Research of modern NL2SQL systems imposes steep hardware requirements on researchers. Priority should be given to securing access to sufficiently powerful hardware and computing infrastructure for faster iteration cycles. While the 24GB VRAM constraint of the RTX 3090 may be reasonable for most consumer grade applications, it proved insufficient for efficient iteration in this field, especially given the involvement of LLMs. Systems with 48GB+ of VRAM would have enabled the usage of larger, more capable models without the quantization compromises that may impact system accuracy. More modern hardware, even on the consumer grade level, would allow for a tremendous benchmarking speed up (e.g., using an RTX 5090 would have resulted in nearly 200% increase in speed). The availability of multiple GPUs would have enabled parallel experimentation and benchmarking, which could transform multi-day iteration cycles into more manageable timeframes. The ability to run concurrent experiments with different configurations, models, or algorithmic approaches would have dramatically accelerated the research and development process and enabled more experimental validation.

5.5.3.2 Technology Stack

The implications of choosing between Rust and Python underline the importance of aligning technology choices with research goals and time constraints. For pure research projects focused on algorithmic development and experimentation, Python's ecosystem and extensive existing tooling provide clear advantages that outweigh the benefits of compiled languages like Rust. For projects with a clear path to production deployments or those intended to bridge academic research with real-world applications, the additional development overhead of systems programming languages like Rust can be justified. Overall, the availability of hardware impacts the implementation timeline significantly more than language chosen. Implementations in this field should follow a two-staged approach: prototyping and algorithm validation in Python to leverage existing tools and speed up iteration, followed by a production port in a systems language like Rust once the algorithmic approach is validated and stable.

5.5.3.3 Development Methodology Recommendations

The challenges encountered suggest several methodological improvements for future projects. Implementing benchmarking and validation infrastructure early in the development process is crucial when doing algorithmic development to gain an understanding of the relative improvement or setback of different approaches. Establishing performance baselines and using incremental validation procedures can help ensure that development effort is focused on improvements that in fact improve performance. The long running benchmarks made it difficult to validate whether individual changes were beneficial, leading to uncertainty about the effectiveness of specific optimizations. In situations with highly constrained compute resources, a representative subset of prevalent benchmarking datasets like Spider and Bird may be defined to determine the relative improvement between approaches before executing a full benchmark run. Finally, maintaining detailed logs and metrics throughout the development process showed to be crucial when unexpected errors occur. Identifying root causes of pipeline failures was rather trivial due to having an advanced benchmarking setup with log collection.

5.5.3.4 Research Impact and Production Readiness

Despite all challenges encountered in the implementation phase, the Rust implementation of Natural demonstrates that it is possible to port research algorithms from academic environments into production-ready systems. The resulting implementation provides a strong foundation for real-world deployments in database environments, which would be significantly more challenging with a Python-based research prototype.

This production readiness ensures that all algorithmic contributions can have practical impact beyond pure academic evaluation, potentially influencing how NL2SQL systems are deployed in real database environments. A simple recommendation between research velocity and production readiness can not be given as it ultimately depends on the intended impact and longevity of the research contributions and implementation.

6 Evaluation

The evaluation of NATURAL builds utilizes the benchmarking infrastructure implemented in Section 5.4. This section is evaluating the performance metrics of different NATURAL configurations on two prevalent benchmarking datasets: SPIDER and BIRD. A representative baseline is introduced, aiming to mirror the base-model performance. Subsequently an ablation study is performed to understand the performance impact of individual pipeline components.

6.1 Methodology

The following sections document the experimentation environment, outline tradeoffs and constraints influencing the performed evaluations.

6.1.1 Experimentation Environment

All benchmarks are performed on a consumer-grade linux system, as research- or enterprise-grade hardware was not available. Only a subset of commonly used datasets and configurations have been therefore measured, as a single benchmark executions of SPIDER and BIRD take between 12-36 hours.

6.1.1.1 Hardware Configuration

The host system used for benchmarking has a NVIDIA RTX 3090 GPU with 24GB VRAM available, an AMD 9990X3D CPU with 12-cores up to 5.5GHz, 64GB of RAM at 6400mt/s and 1TB of SSD storage at a read speed of 7450 MB/s and a write speed of 6900 MB/s.

6.1.1.2 Software Stack

The software stack running on the system used for benchmarking is NixOS (25.05) with a recent linux kernel version (6.17.7). CUDA 13.0 is being used for inference on GPUs and Rust 1.91 for compiling NATURAL. The models used during benchmarking are OMNISQL-7B and Qwen3 Embedding 4B (Q8) for generation and embedding respectively.

6.1.2 Benchmark Datasets

SPIDER and BIRD were used as benchmarking datasets as they are prevalently used for the evaluation of NL2SQL systems and therefore enable cross study evaluations (see Section 2.6.1 and Section 2.6.2). The complex SPIDER 2.0 version of SPIDER was excluded from this evaluation due to the lack of time and hardware required to develop a competitive approach (see Section 2.6.3). Comparable research publishes the execution accuracy and exact match metrics on these benchmarks which enables relative and absolute performance comparisons of NATURAL with other studies.

6.1.3 Evaluation Metrics

The chosen set of evaluation metrics used is a mixture of semantic metrics ($\mathbb{EA}$, $\mathbb{EM}$) and functional metrics ($\mathbb{ER}$, $\mathbb{CL}$) to gain understanding on system behavior and real world applicability.

6.1.3.1 Execution Accuracy ($\mathbb{EA}$)

Execution Accuracy (or $\mathbb{EA}$) is the primary success metric of NL2SQL systems as it represents the semantic accuracy of the SQL queries produced by NL2SQL systems. $\mathbb{EA}$is computed by determining whether the rows in the candidate results are a permutation of the rows returned by ground truth results.

For a query q_g with candidate results R_c and ground truth results R_g, where $R_c, R_g \subseteq \mathbb{V}^{n \times m}$ (sets of n rows with m columns of values $\mathbb{V}$), semantically accurate execution is defined as:

$$semanticeq(R_c, R_g, q_g) = \nVdash \left[\exists \pi \in \Pi_m : \begin{cases} R_c = \pi(R_g) & \text{if } ordered(q_g) \\ R_c \equiv_{\text{multiset}} \pi(R_g) & \text{otherwise} \end{cases} \right] \tag{15}$$

where Π_m is the set of valid column permutations (those preserving column value sets), $\pi(R_g)$ applies permutation π to reorder columns in each row of R_g, $\equiv_{\text{multiset}}$ denotes multiset equality (allowing row reordering) and $ordered(q_g)$ is determining if the ground truth query q_g contains an `ORDER BY` clause.

In order to compute the $\mathbb{EA}$ on Spider and Bird the official evaluation suite is used respectively to ensure comparability across externally reported measurements and locally observed measurements.

6.1.3.2 Exact Match ($\mathbb{EM}$)

Exact Match (or $\mathbb{EM}$) measures the syntactic equivalence between candidate and ground truth SQL queries after syntax normalization. Compared to $\mathbb{EA}$, which validates semantic correctness through result comparison, $\mathbb{EM}$ determines whether the generated query structurally matches the ground truth query. $\mathbb{EM}$ thus acts as a lower bound for execution accuracy as queries must be identical which is therefore more restrictive as semantically equivalent queries with different syntax (e.g., using a different join order) are marked as incorrect.

6.1.3.3 Error Rate ($\mathbb{ER}$)

Error Rate (or $\mathbb{ER}$) describes the system reliability by measuring the frequency of SQL generation failures that prevent query execution. $\mathbb{ER}$ is reported in failures per hundred queries and monitors system reliability (schema violations, syntax errors, out-of-memory errors etc.).

6.1.3.4 Candidate Latency ($\mathbb{CL}$)

Candidate Latency (or $\mathbb{CL}$) measures the end-to-end execution time of a system from natural language input to SQL candidate output, reported in seconds. This metric captures the compound runtime of all pipeline components $(\sigma, \phi, \pi, \rho, \nu)$ and reflects real-world system responsiveness. Latency increases with pipeline complexity, particularly when example selection and self refinement are applied, thus representing relative computational cost.

6.1.4 Baselines

To determine the performance impact of methods and components applied in Natural a baseline helps to measure relative performance gains or losses compared to existing systems. Therefore two sets of baseline systems are introduced: Internal baselines (measuring the system without crucial components) and external baselines (e.g., proprietary system performance and raw model performance).

6.1.4.1 Internal Baselines

Introducing two internal baselines subsequently allows for a brief ablation study, where the impact of different pipeline configurations and different sampling datasets can be measured to isolate the contribution of each configuration. The *Baseline* configuration of Natural is using only the inference logic (π) without all other pipeline components, representing the performance contribution of OmniSQL as closely as possible. Additionally the *Zero-Shot* configuration of Natural is introduced and measures the performance of Natural with all pipeline components activated but without any examples available to use during ICL.

6.1.4.2 External Baselines

Using published results from other systems, Natural can be briefly compared against existing systems with similar capabilities highlighting relative performance improvements or losses. Notably external baselines largely rely on unverified data from other papers. Given that Natural is largely based on OmniSQL, GPT-4 and OmniSQL are the primary external systems are taken into account as baselines.

6.1.5 Vector Databases

Three different vector databases are introduced per benchmark. These differ in the datasets using during sampling:

6.1.5.1 Synthetic

The *Synthetic* vector databases are sampled from the SynSQL-2.5m dataset introduced by H. Li et al. in 2025. The SynSQL-2.5m dataset is largely unrelated to Spider and Bird but covers a wide array of domains through its LLM-based synthetic data generation approach.

6.1.5.2 Train

The *Train* vector databases are sampled from the respective training splits from Spider and Bird (J. Li, Hui, Qu, et al., 2023; Yu, Zhang, et al., 2018). The train splits include similar style of SQL queries and similar domains but different databases and different questions.

6.1.5.3 Ground

The *Ground* vector databases are sampled from the respective splits used during evaluation (dev and test) from Spider and Bird (J. Li, Hui, Qu, et al., 2023; Yu, Zhang, et al., 2018). These allow Natural to reference correct answers as examples during generation indicating the potential upper-limit of performance during a self-learning deployment which has access to past conversations.

6.2 Benchmark Results

This section is presenting the measured performance across all Natural configurations and benchmark datasets. Internal ablations are compared (*Baseline*, *Zero-Shot*, as well as different vector DBs) against external prior art (GPT-4, OmniSQL). Subsequently performance characteristics and improvement trends are quantitatively and qualitatively analyzed.

6.2.1 Overview

The following sections present these findings across benchmarks and configurations in detail. The system metrics are presented in Table 3 and visualized in Figures 11 and 12, which displays the performance for all system configurations across the Spider and Bird datasets, the execution accuracy ($\mathbb{EA}$), exact match ($\mathbb{EM}$), error rate ($\mathbb{ER}$) and candidate latency ($\mathbb{CL}$) metrics.

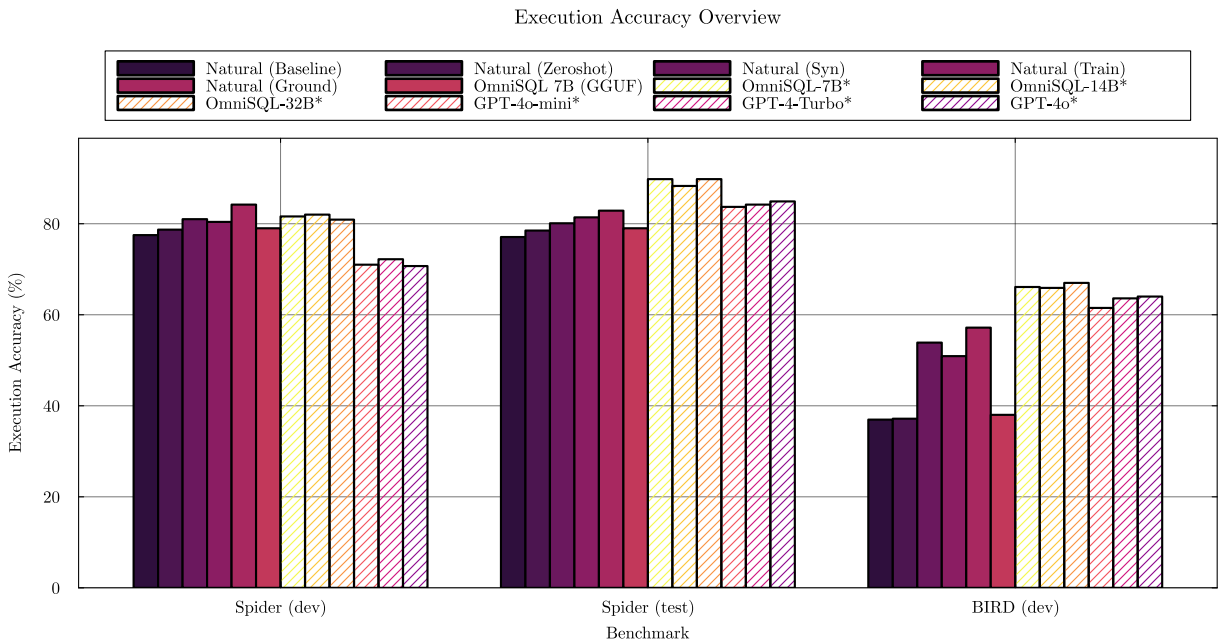

Figure 11: Execution accuracy overview across all systems and benchmarks. The *Syn* configuration achieves the highest verified performance on Spider (dev) at 81.0% and Bird (dev) at 53.8%, while *Train* leads on Spider (test) at 81.4%.

System	Spider (dev)				Spider (test)				BIRD (dev)			
	$\mathbb{EA}$	$\mathbb{EM}$	$\mathbb{ER}$	$\mathbb{CL}$	$\mathbb{EA}$	$\mathbb{EM}$	$\mathbb{ER}$	$\mathbb{CL}$	$\mathbb{EA}$	$\mathbb{EM}$	$\mathbb{ER}$	$\mathbb{CL}$
Closed-Source LLMs												
GPT-4o-mini*	71.0	-	-	-	83.7	-	-	-	61.5	-	-	-
GPT-4-Turbo*	72.2	-	-	-	84.2	-	-	-	63.6	-	-	-
GPT-4o*	70.7	-	-	-	84.9	-	-	-	64.0	-	-	-
Open-Source LLMs												
OmniSQL-7B*	81.6	-	-	-	89.8	-	-	-	66.1	-	-	-
OmniSQL-14B*	82.0	-	-	-	88.3	-	-	-	65.9	-	-	-
OmniSQL-32B*	80.9	-	-	-	89.8	-	-	-	67.0	-	-	-
OmniSQL-7B-gguf	79.0	33.0	0.1	**6.6s**	79.0	35.7	0.3	**6.7s**	38.0	0.03	0.8	**8.2s**
Pipelines												
Natural (Baseline)	77.9	30.3	**0.0**	7.3s	77.0	32.5	**0.0**	7.3s	32.4	0.03	**0.3**	9.3s
Natural (Zero-Shot)	78.7	27.1	0.2	14.0s	77.5	32.7	0.4	15.3s	34.0	0.03	0.4	18.7s
Natural (Syn)	**81.0**	41.2	0.3	16.3s	79.6	38.7	0.4	15.4s	**53.8**	**18.8**	0.2	43.7s
Natural (Train)	80.4	**42.2**	0.6	16.2s	**81.4**	**43.9**	0.3	15.8s	48.4	13.2	0.3	33.7s
Natural (Ground)**	84.2	42.7	0.7	16.1s	82.8	39.5	0.3	16.0s	55.8	19.3	0.2	43.8s

Table 3: Comprehensive benchmark results across all systems and datasets. $\mathbb{EA}$ and $\mathbb{EM}$ are reported as percentages. $\mathbb{ER}$ is reported in failures per hundred queries and $\mathbb{CL}$ is reported in seconds. Systems marked with * are external benchmarks with unverified results from published papers. Systems marked with ** had access to ground truth data during inference, illustrating the upper bound achievable. Bold values indicate best verified performance, excluding systems with access to ground truth. (H. Li et al., 2025)

6.2.2 Baseline Performance Analysis

One notable insight during benchmarking was the apparent gap between the published and the locally measured performance of OmniSQL 7B-gguf (with F16 weights) and the reported accuracy metrics results from H. Li et al. (2025). As shown in Figure 18, the Δ in performance between the local measurement and the official values ranges from 2.6% on Spider (dev) to 11.8% on Spider (test) and 28.1% on Bird (dev). The section performance gap section (6.4) analyzes this gap in detail.

The *Baseline* configuration of Natural further showed degradation of performance compared to the OmniSQL-7B-gguf system configuration. Using the *Baseline* configuration, Natural only uses the inference logic and prompt template for ICL (π) without other pipeline components, achieving 77.9% $\mathbb{EA}$ on Spider (dev), 77.0% on Spider (test), and 32.4% on Bird (dev). These results further fall below the published OmniSQL-7B performance of 81.6%, 89.8%, and 66.1% respectively, representing gaps of 3.7pp, 12.8pp, and 33.7pp. This continued degradation of performance indicates that a misaligned prompt template (i.e., divergent from training phase) harms performance if not paired with counter measures such as ICL, self-correction or majority voting. These performance numbers mark the *Baseline* configuration the worst performing variant of Natural on $\mathbb{EA}$ and $\mathbb{EM}$ across Spider (dev and test) and Bird (dev).

6.2.3 Spider Results

The results of the Spider benchmarks demonstrate consistent performance characteristics of Natural and OmniSQL across the development and test splits (detailed breakdown in Appendix A.3.1, Figures 19, 20, 21, and 22). The OmniSQL-7B-gguf system outperforms the two Natural baseline configurations *Baseline* and *Zero-Shot* in $\mathbb{EA}$ and $\mathbb{EM}$ by 2.1% and 0.3% on the development split and by 2.0% and 1.5% on the test split respectively.

The full pipeline with the *Zero-Shot* configuration (all components active but no examples) hence only yields marginal improvements (+0.8pp on dev, +0.5pp on test) over *Baseline*, suggesting that schema subsetting and query refinement contribute minimal performance gains if any. Notably, as demonstrated in Figure 13, the $\mathbb{CL}$ of *Zero-Shot* increases by 6.7s on dev and 8.0s on test over *Baseline*, doubling the system latency without a significant improvement in system performance.

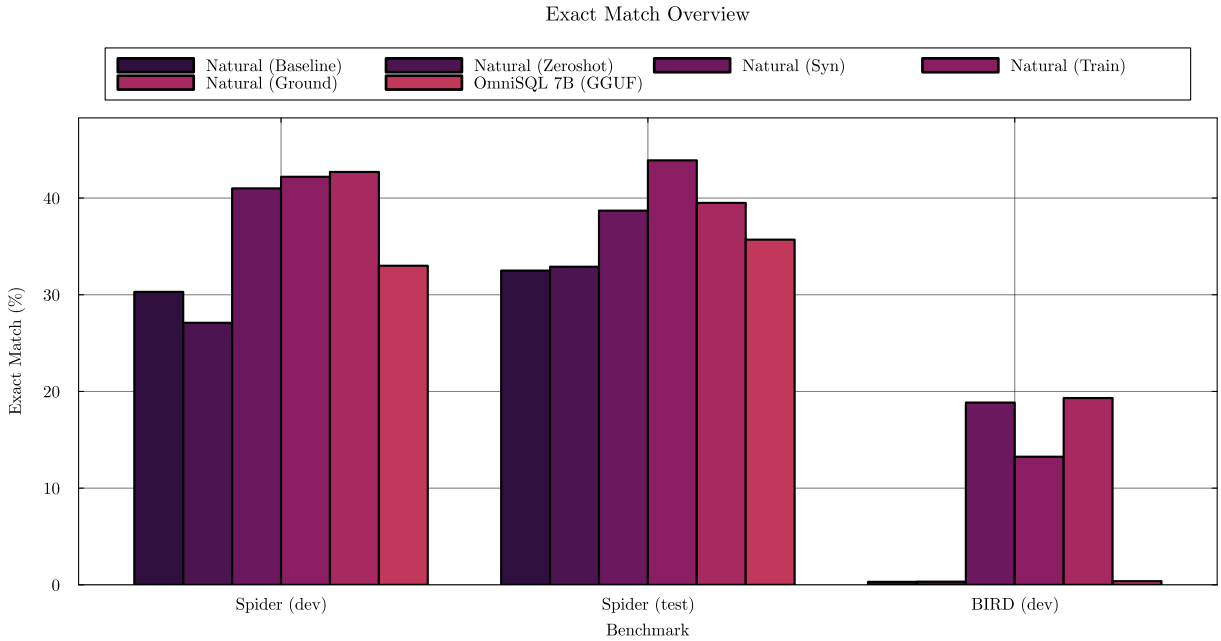

Figure 12: Exact match overview across all systems and benchmarks. ICL configurations demonstrate substantial improvements over baselines, with *Train* achieving 43.9% on SPIDER (test), the highest verified exact match rate across all benchmarks.

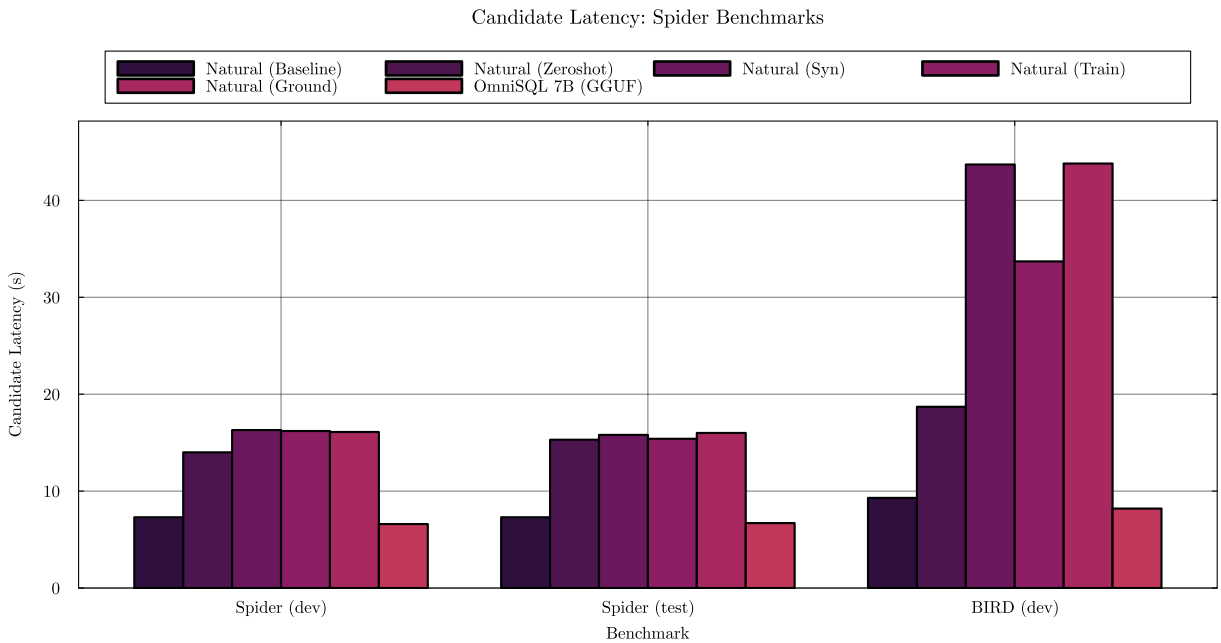

Figure 13: Candidate latency across system configurations on SPIDER benchmarks. The *Zero-Shot* configuration shows doubled latency (14.0s on dev, 15.3s on test) compared to *Baseline* (7.3s) without notable performance gains, while ICL configurations incur additional overhead from example retrieval.

Introducing vector database access to the system configurations demonstrates substantial improvement in performance. The two configurations without ground truth examples showed a significant improvement in both $\mathbb{EA}$ and $\mathbb{EM}$: *Syn* outperformed all other system variants and baselines with a accuracy improvement of 3.1% over *Baseline* and 2.0% over OmniSQL-7B-gguf on Spider (dev). On Spider (test) the *Train* configuration outperformed *Syn* by 1.8% in $\mathbb{EA}$. For $\mathbb{EM}$ both system configurations show a steep improvement over the three baselines with largest $\mathbb{EM}$ deltas being observed between *Zero-Shot* and *Train* with +15.1% on Spider (dev) and between *Baseline* and *Train* with +21.4% on Spider (test). These double digit improvements in accuracy, as shown in Figure 14, point towards the conclusion that ICL is an effective mechanism for further improving the performance of already fine-tuned models. The performance of the *Ground* configuration shows the theoretical upper limits in $\mathbb{EA}$ achievable on top of OmniSQL with example selection algorithms with 84.2% on Spider (dev) and 82.8% on Spider (test) (see Appendix A.3.1).

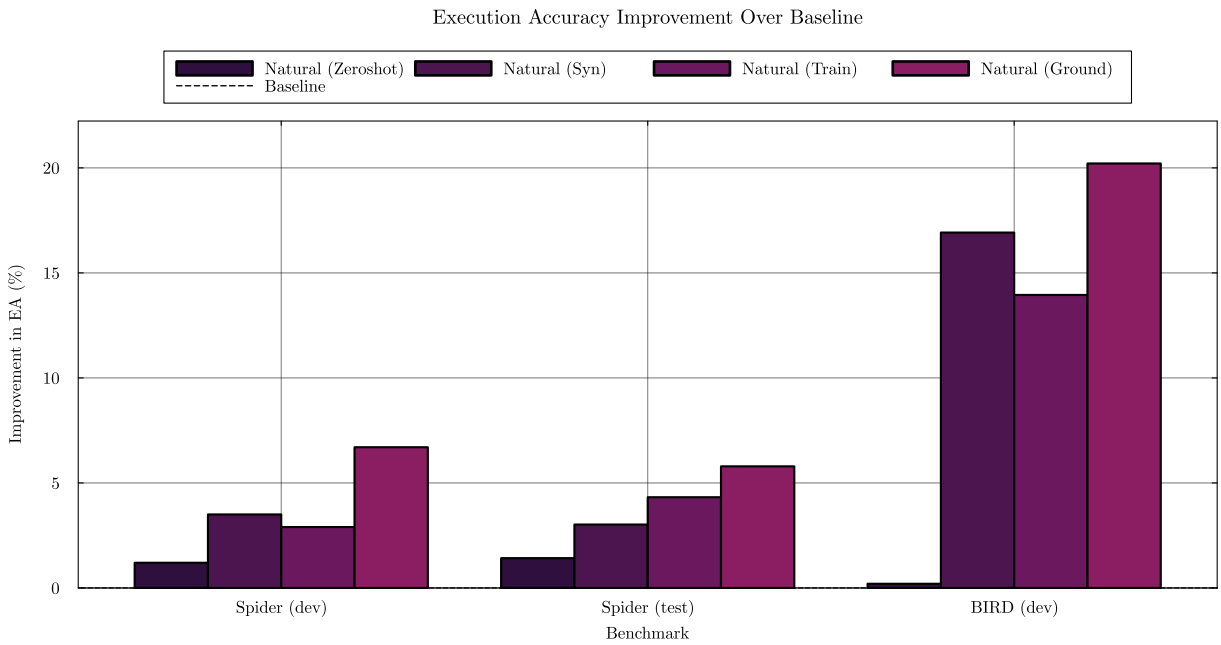

Figure 14: Execution accuracy improvement over the *Baseline* configuration across all benchmarks. The *Zero-Shot* configuration yields marginal gains ($\leq$1.6pp), while ICL-enabled configurations show substantial improvements peaking at +21.4pp (*Syn* on Bird dev). The substantial contrast between Spider and Bird gains highlights that ICL impact scales with query and schema complexity.

Error rates remain consistently low across all configurations ($<$0.7 failures per hundred queries), see Figure 15, indicating that failures are predominantly semantic rather than syntactic, which validates the robustness of OmniSQL's SQL generation capabilities.

6.2.4 BIRD Results

Measurements on Bird, which is designed to represent real-world databases with imperfect data (see Section 2.6.2), external knowledge requirements and hard to reason about database schemas yields significantly lower absolute performance numbers but shows improvements through ICL based systems. While the *Baseline* configuration of Natural only achieves 32.4% $\mathbb{EA}$ (less than half the accuracy than on Spider with 77.9% and 77.0% for dev and test splits respectively). This performance confirms that Bird is a significantly more challenging benchmark. While the *Zero-Shot* configuration shows incremental improvements over the baseline with a relative improvement of +1.6pp in $\mathbb{EA}$ (34.0% absolute), the overall performance shows to be behind the reported numbers of OmniSQL-7B and GPT-4 by H. Li et al. (2025) which were reported to have an $\mathbb{EA}$ of 66.1% and 61.5% respectively. The locally reproduced measurement of OmniSQL-7B-gguf yielded a drop in $\mathbb{EA}$ of -28.1pp compared to the officially reported $\mathbb{EA}$ yielding 38.0% in absolute accuracy.

Both the *Syn* and *Train* configurations of Natural outperform all baselines. *Syn* shows an improvement of +21.4pp over *Baseline* and +15.8pp over OmniSQL-7B-gguf, achieving 53.8% $\mathbb{EA}$, the best local measured

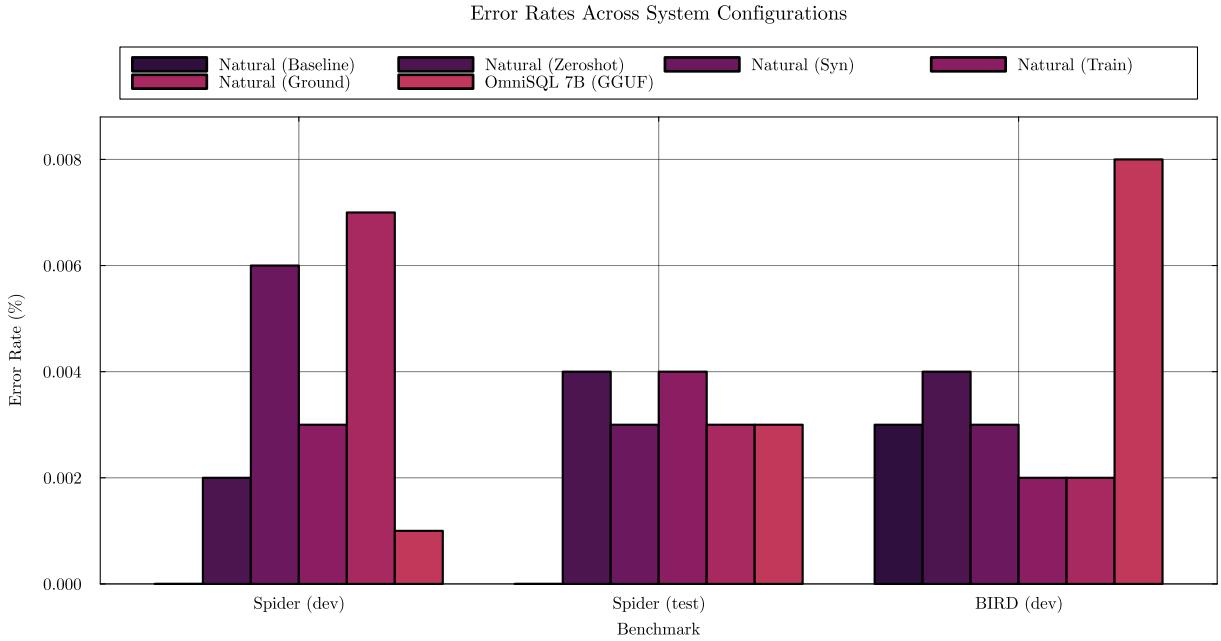

Figure 15: Error rates across system configurations. All configurations maintain error rates below 0.7 failures per hundred queries, indicating that failures are predominantly semantic rather than syntactic or runtime related.

system excluding *Ground*. *Train* yields improvements of +16.0pp over *Baseline* and +10.4pp over OmniSQL-7B-gguf.

This shows a notable contrast with Spider, where synthetic examples provided only +3.1pp improvement on dev and +2.6pp improvement on train. The effectiveness of synthetic examples on Bird suggests that the impact of ICL increases with query and schema complexity. When queries involve complex joins, nested subqueries, and aggregations, even domain-agnostic examples provide crucial scaffolding for the generation. The *Train* configuration of Natural underperformed *Syn* by 5.4pp. This may indicate that Bird's training examples, while domain-relevant, contain different structural patterns than the development set, whereas SynSQL-2.5m contains a broader coverage of different database schema structures to reference from.

The other metrics $\mathbb{EM}$, $\mathbb{ER}$ and $\mathbb{CL}$ further show a substantial difference between Spider and Bird. Systems which mostly relied on the model (like Natural *Baseline* and *Zero-Shot* and OmniSQL-7B-gguf) showed the same 0.03% in $\mathbb{EM}$ indicating a potential overfitting of OmniSQL to the SQL style used in Spider. Notably ICL based systems showed a significant improvement with +13.17pp for *Train* and +18.77pp for *Syn* (see Figure 25 for detailed $\mathbb{EM}$ improvements). The $\mathbb{CL}$ metric showed a similar pattern to the Spider benchmarks with the $\mathbb{CL}$ increasing with pipeline complexity from 9.3s for *Baseline* to 43.8s for *Ground*, as illustrated in Figure 16.

Lastly the *Ground* configuration establishes an upper bound of 55.8% $\mathbb{EA}$, representing a +23.4pp improvement over *Baseline*, a +72% relative gain. Despite these substantial relative improvements within the recorded benchmarks a significant gap of -10.3pp and -8.2pp remains when compared to the published OmniSQL-7B performance with 66.1% and GPT-4o performance with 64.0%. This notable difference between local replication and officially reported performance metrics is yet unclarified. Detailed per-metric breakdowns for Bird (dev) are provided in Appendix 23 and 24.

6.3 Performance Characteristics

To assess the performance of NL2SQL systems holistically metrics beyond semantic accuracy have to be taken into account. The two performance characteristics $\mathbb{ER}$ and $\mathbb{CL}$ provide insight into system reliability and their real-world applicability. These functional metrics complement accuracy by revealing operational constraints, such as hardware requirements or reliability, that affect deployment feasibility.

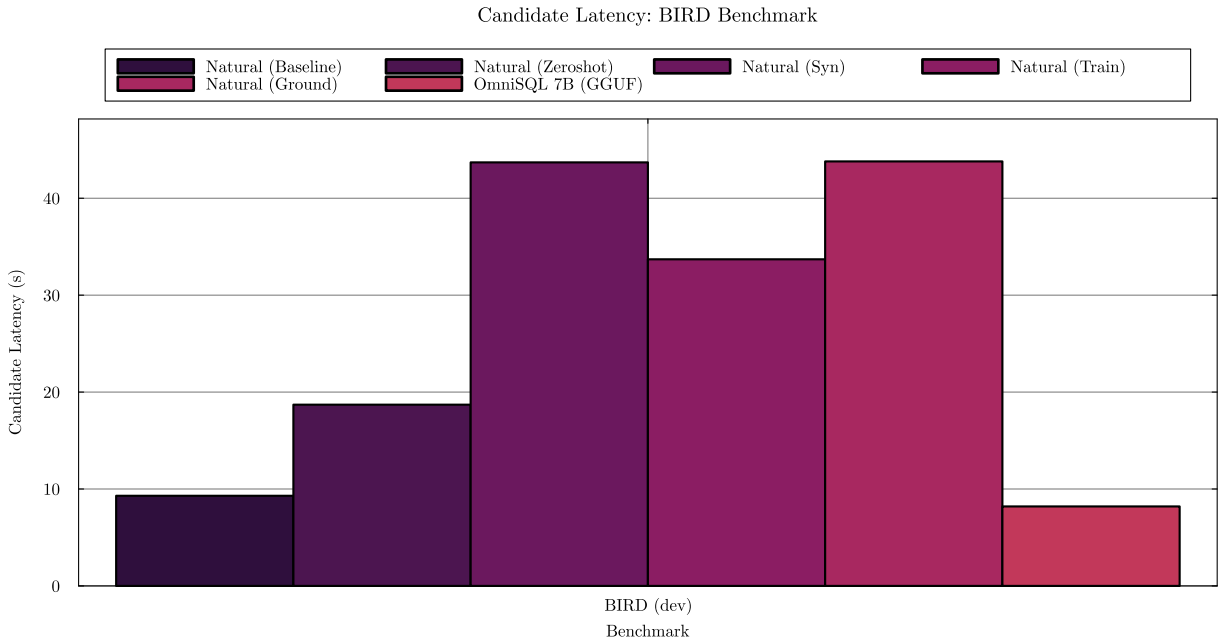

Figure 16: Candidate latency across system configurations on the Bird benchmark. Latencies are substantially higher than Spider equivalents, with *Syn* and *Ground* reaching 43.7s and 43.8s respectively, driven by the larger schemas, longer prompts and more complex query generation required by Bird.

6.3.1 Error Rate Analysis

The error rate results demonstrate great reliability across all benchmarked systems and datasets. The error rates remain below 0.7% (fewer than 7 failures per 1000 queries) across all measured systems, with the *Baseline* achieving 0.0% error rate on Spider (dev) and 0.3% on Bird (dev). Even Natural configurations with self-refinement, query parsing and ICL maintain low error rates: *Ground* shows an $\mathbb{ER}$ of 0.7% on Spider (dev), 0.3% on Spider (test) and 0.2% on Bird (dev). This consistency indicates that the OmniSQL model generates syntactically valid, executable SQL with a high reliability, and that pipeline components do not introduce significant failure modes. Furthermore these low error rates reveal an important characteristic of OmniSQL and Natural: Errors of these NL2SQL systems are predominantly semantic rather than syntactic or runtime-related. This distinction matters for production deployments, as semantic errors are significantly harder to validate and catch than hard system failures. Semantic accuracy might be further improved through user feedback which can feedback into ICL.

6.3.2 Latency Analysis

Measuring the $\mathbb{CL}$ reveals the computational trade-offs that come with the the Natural pipeline architecture. The *Baseline* configuration achieves minimal candidate latency (7.3s on Spider (dev and test) and 9.3s on Bird dev), representing the time required for schema subsetting and SQL generation without example retrieval or refinement.

The *Zero-Shot* pipeline configuration approximately doubles the end-to-end latency to (14.0s, 15.3s, 18.7s respectively) highlighting that the schema subsetting, self-refinement and voting components contribute to a significant increase in computational cost. Configurations with example selection and ICL show further latency increases. The *Train* and *Syn* configurations achieve 15.4-16.3s $\mathbb{CL}$ on Spider and 33.7-44.7s on Bird, with the *Syn* configuration yielding the highest latency (43.7s on Bird).

Overall OmniSQL-7B-gguf consistently showed the lowest $\mathbb{CL}$ values with (6.6s, 6.7s and 8.2s) for Spider (dev and test) and Bird respectively. This is unsurprising as the OmniSQL-7B-gguf system provides a realistic estimation for the π component of Natural. These characteristics inform hardware requirements and deployment decisions. As shown in Figure 17, the trade-off between latency and accuracy varies significantly across configurations and benchmarks. For interactive applications sub-second response times of these models are recommended. Natural clearly exceeds these thresholds by a factor of up to 45 on complex databases and

questions (e.g., Bird). Thus either significantly better hardware is required, a simpler configuration needs to be used or further performance optimizations need to be introduced.

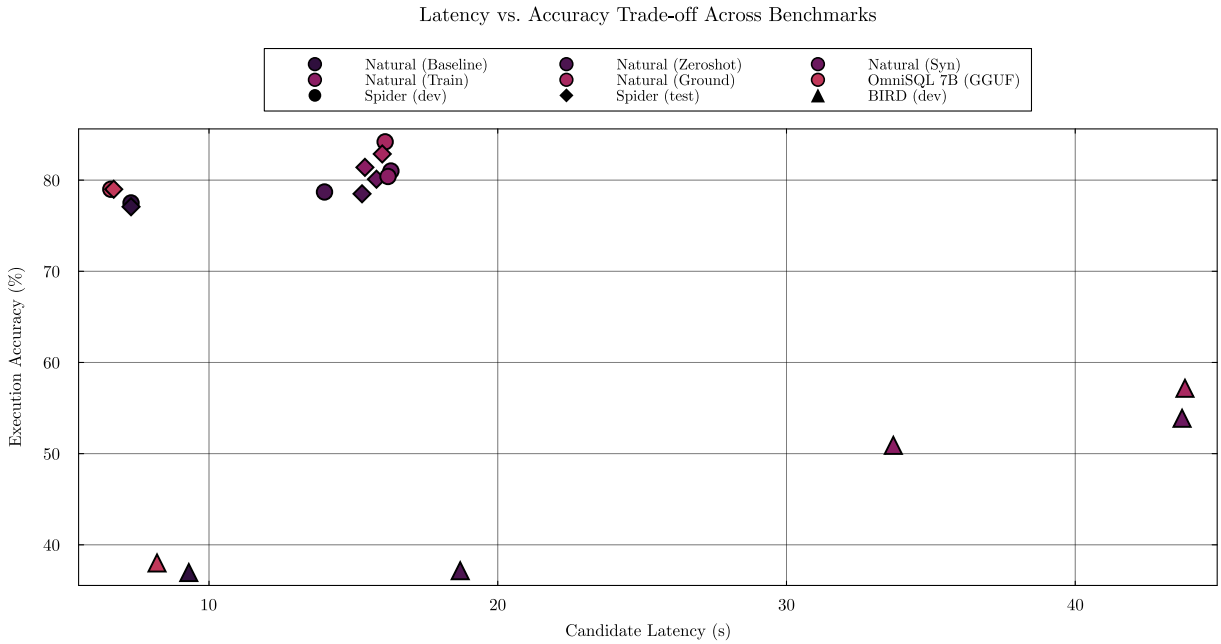

Figure 17: Latency vs. execution accuracy trade-off across all verified system configurations and benchmarks. Each point represents a (configuration, benchmark) pair. This figure highlights that moving from *Baseline* to ICL-enabled configurations increases $\mathbb{EA}$, but yields significant higher $\mathbb{CL}$, particularly on Bird with a $\mathbb{CL}$of 43s. The *Zero-Shot* configuration represents an inefficient operating point with a comparatively high latency to accuracy.

6.4 Performance Gap

During baseline validations of the foundational models a critical performance discrepancy emerged (see Figure 18) between the locally measured performance of OmniSQL 7B and the published performance by H. Li et al.. This section systematically breaks down the magnitude, potential causes and implications of this gap.

6.4.1 Magnitude and Characterization

Table 4 presents the gap between the measured performance discrepancies across all evaluated benchmarks for the OmniSQL 7B model. It is important to note that a GGUF variant was used for local measurements. The resulting gap varies significantly by dataset, ranging from only 2.6% on Spider (dev) up to 28.1% on Bird (dev).

Benchmark	**Published ($\mathbb{EA}$)**	**Measured ($\mathbb{EA}$)**	**Gap (Δ)**
Spider (dev)	81.6%	79.0%	-2.6pp
Spider (test)	89.8%	79.0%	-10.8pp
Bird (dev)	66.1%	38.0%	-28.1pp

Table 4: Performance gap between published OmniSQL 7B results (H. Li et al., 2025) and local measurements using OmniSQL-7B-gguf. Negative values indicate underperformance relative to published baselines.

The uneven distribution of the measure performance gaps across the three benchmarks suggests a systematic cause rather than random variation. The gap correlates strongly with benchmark complexity: Spider (dev), the simplest of the three benchmarks with well-structured schemas and clean data is showing only a minimal performance degradation of -2.6pp. Spider (test) which is not included in any training dataset, using more

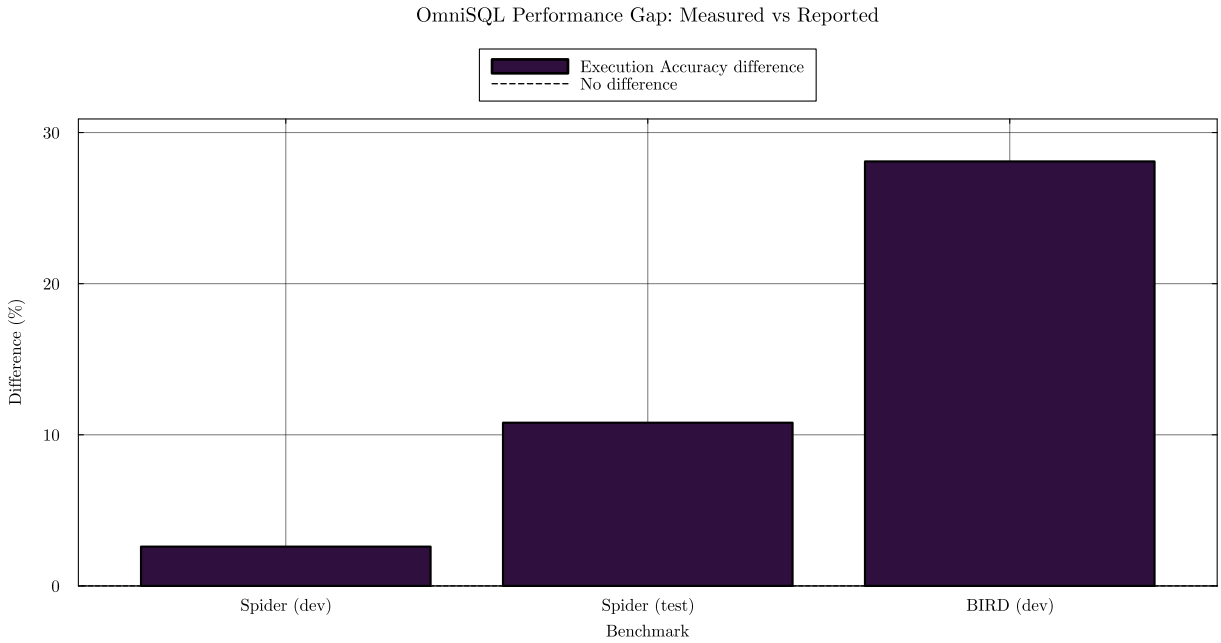

Figure 18: Performance gap between measured OmniSQL-7B-gguf and reported OmniSQL-7B results across benchmarks. The Δ ranges from 2.6% on Spider (dev) to 28.1% on Bird (dev), highlighting differences between local reproduction and published results.

diverse domains and more complex queries is showing a moderate degradation of -10.8pp. Bird (dev) which is the most complex benchmark of the three, featuring larger schemas, dirty data and external knowledge is showing an extreme degradation of -28.1pp. This is indicating a performance drop correlating with task difficulty, suggesting that the locally measured model had impaired reasoning capabilities, difficulties handling larger schemas or handling external knowledge requirements.

6.4.2 Potential Contributing Factors

Several factors may contribute to this notable gap between the two systems. This subsection is analyzing potential causes using available evidence.

6.4.2.1 Model Format Differences

One apparent difference between the published performance metrics and the local measurements for OmniSQL 7B is the model format and precision used. The OmniSQL-7B-gguf system uses the F16 weights but transformed into the GGUF model format while published results presumably use full-precision models in the safetensors formats. Prior research has shown that bit-wise quantization can degrade model accuracy, particularly on complex reasoning tasks on multi-billion parameter models (Dettmers, Lewis, Belkada, & Zettlemoyer, 2022) and it is yet unclear whether the mere transformation into GGUF harms performance for this model series. Second, the exact hyperparameters used during evaluation likely differed between H. Li et al. (2025) and local benchmarks which may influence the model accuracy in unforeseen ways. Unless the exact same inference environment, quantization and hyperparameters the performance difference might be caused by a multitude of subtle differences. Finally, dataset version differences or preprocessing variations could contribute to divergent results. The Δ of 2.6% on Spider (dev) is well within the expected format-induced degradation ranges, which initially suggests that the model format alone could explain the simple benchmark discrepancy. However, the widening performance gaps with increasing benchmark complexity, up to 28.1pp on Bird, substantially exceed the typical effects of model format conversions indicating that other factors might contribute to the observed performance loss.

6.4.2.2 Inference Implementation Differences

H. Li et al. likely used vLLM as their inference engine, while local measurements of OmniSQL 7B used Rust FFI bindings to llama.cpp. These engines implement the same mathematical operations but may differ in the implementation of matrix operations, attention, sampling algorithms and KV-cache management. To keep further inference environment divergent as small as possible, similar (where possible) sampling configurations, prompts and hyperparameters where used.

6.4.2.3 Hyperparameter and Configuration Uncertainty

While the published OmniSQL evaluation does not exactly describe all its inference hyperparameters, some were noted in example code but uncertainty remains about the concrete configuration. Critical parameters that can widely influence model performance include temperature, top-p sampling, maximum generation length, context window size, repetition penalties and stop sequences.

Local measurements used a context size of 32768 and only greedy sampling with standard stop sequences. Inferring from the results published by H. Li et al. greedy sampling was the primary sampling method during evaluation alongside majority voting. Overall a difference in hyperparameters and inference engine configuration is possible, but is concluded to be unlikely given that greedy sampling as a standard and very straightforward algorithm.

6.4.2.4 Prompt Template Alignment

A prompt drift could cause significant performance differences the exact prompt as supplied by H. Li et al. was used. Subtle differences in phrasing, layout or presentation could yield widely diverging results. It is likely that OmniSQL-7B was fine-tuned using a specific prompt format, and deviation from this format can significantly degrade performance. The local benchmarking implementation used the published prompt template but likely had a slightly diverging code representation implementation than the authors of OmniSQL. The code representation implementation used by the benchmarking harness is based around the `sqlparser` crate in Rust which includes an implementation for human readable formatting of SQL queries. While these modifications are subtle, they may inadvertently misalign with the model's learned expectations on the code representation format, confusing the generation process.

The *Baseline* configuration results are further evidence for a possible prompt drift: Even when using only the inference logic (π) without additional pipeline components, Natural achieved 77.9% EA on Spider (dev) compared to OmniSQL-7B-gguf's 79.0%, a 1.1pp degradation despite using the same model and quantization. This suggests that even OmniSQL is noticeably sensitive to prompt template differences affect performance. The prompt template used in the local benchmarks for Natural is shown in Appendix A.1.1 and for OmniSQL in Appendix A.1.3. Natural's prompt deviated from the presumed original format through the addition of similarity scores for ICL examples, more explicit instructions and the likely modified schema representation.

6.4.3 Implications for Evaluation Validity

Due to time and resource constraints no exhaustive systematic evaluation of different quantization schemes, inference engines and prompt templates was performed. This significantly limits the ability to draw definite conclusions on the exact root cause for the performance drift between the published performance numbers and the locally measured ones.

6.4.3.1 Impact on Absolute Performance Claims

The substantial gap between the published vs reproduced baselines undermines the confidence in *absolute* performance gains. The *relative* performance gains are excellent (e.g., *Syn* achieving 53.8% vs. OmniSQL-7B-gguf achieving 38.0% on Bird) but given that no clear absolute improvement could be measured (e.g., *Syn* achieving 53.8% vs OmniSQL-7B achieving 66.1% on Bird). Furthermore the comparisons with other external systems become unclear if local measurements face a significant gap in compared to the published performance numbers.

Thus this thesis avoids strong claims about absolute performance levels and instead focuses on the relative improvements achieved on local NL2SQL systems which is consistent with the evaluation environment and results.

6.4.3.2 Reliability of Relative Comparisons

Most notably, the performance gap between OmniSQL-7B and OmniSQL-7B-gguf does not infect the validity results measured between different variants of Natural and between Natural and OmniSQL. All Natural configurations and OmniSQL-7B-gguf have been evaluated using the same evaluation framework and are therefore strictly comparable. Natural showed strong performance improvements across increasingly complex pipeline configurations. *Syn* showed an improvement of +21.4pp over *Baseline* on Bird which was reliably measurable regardless of the absolute baseline uncertainty.

Furthermore component ablations in the evaluation provide valid evidence for understanding the relative contributions of different pipeline components (minimal gain without ICL vs. substantial gain with ICL). Therefore the conclusion holds true that example selection is the most impactful component and that components exhibit synergistic rather than additive effects remain valid although the conclusion is only backed by same-environment evaluations instead of cross-study evaluations.

6.4.3.3 Broader Implications for LLM-based Research

The challenges encountered in this thesis reflect broader, systemic issues in LLM-based NL2SQL research. Without accessibility to the same hardware, models, specifications and evaluation frameworks a reliable cross-study comparison becomes close to unachievable. Intra-study comparisons remain valid but require significant time and hardware resources to achieve. This suggests that the NL2SQL research community would benefit from standardization of inference frameworks, comprehensive documentation of all hyperparameters and configurations used as well as a release of exact prompts, preprocessing code and scripts used during evaluation. Furthermore could multiple independent reproductions increase the confidence in claimed results.

6.4.4 Summary

Overall the gap of 28.1pp on Bird between the published OmniSQL-7B results (66.1% $\mathbb{EA}$) and local measurements (38.0% $\mathbb{EA}$) represents a challenge for deriving absolute conclusions. While strong incremental performance gains could be observed (e.g., *Syn* achieving 53.8% vs. OmniSQL-7B-gguf achieving 38.0% on Bird) by applying the techniques outlined in this thesis (schema aware example selection, self-refinement, majority voting) no clear statement can be made on the absolute performance of Natural. A multitude of differences cn influence the observed gap, including the model format, inference implementation, hyperparameter, prompt template, and benchmark version used. Nonetheless, the relative comparisons within the local evaluation environments remain reliable, therefore enabling valid, but thesis-local conclusions.

7 Discussion

This section determines the applicability of NATURAL, answers the research questions based on the evaluation and derives recommendations for future research.

7.1 Results

The evaluation of NATURAL revealed five primary insights from the results:

1. **Relative Improvements** – NATURAL *Full* achieves higher scores than the *Baseline* configuration and OMNISQL-7B-gguf throughout. NATURAL achieves 81.0% $\mathbb{EA}$ on SPIDER (dev), 81.4% on SPIDER (test) and 53.8% on BIRD (dev), compared to OMNISQL-7B-gguf with 79.0% on SPIDER (dev and test) and 38.0% on BIRD.

2. **Examples Structure** – Synthetic examples outperformed domain-specific training data on BIRD (dev) by +5.4pp (53.8% vs 48.4%), and +0.6pp on SPIDER (dev) (81.0% vs 80.4%), suggesting structural similarity matters more than domain relevance in ICL.

3. **Suboptimal Latency** – Candidate latency of NATURAL ranges from 7.3s (*Baseline*) to 43.8s (*Full*) on BIRD, making it primarily suitable for analytical but not for interactive workloads.

4. **Semantic Errors** – 99%+ of failures are semantic (i.e., wrong results) rather than syntactic (i.e., malformed SQL), indicating strong SQL generation and validation but weak semantic understanding.

5. **Limited Validity** – A Δ of 28.1pp exists between published OMNISQL-7B results (66.1%) and the local reproduction (38.0%) on BIRD, limiting absolute performance comparisons.

On the complex BIRD benchmark, the *Syn* configuration reaches a 53.8% $\mathbb{EA}$ which represents a +21.4pp improvement over the 38.0% achieved by the *Baseline* configuration — representing a 66% relative gain in execution accuracy. These results position NATURAL as competitive with the locally-reproduced OMNISQL-7B-gguf baseline (79.0% on SPIDER (dev), 79.0% on SPIDER (test), 38.0% on BIRD) while remaining below published accuracy scores of closed-source model results (GPT-4o: 84.9% SPIDER, 64.0% BIRD).

7.1.1 Ablation

The ablation study showed that example selection was the differentiating component for performance, other pipeline components have not been ablated individually but showed to add unclear performance gains. The *Zero-Shot* configuration which uses schema subsetting, self-refinement, and majority voting but does not apply example selection achieves only a +0.8pp improvement on SPIDER despite doubling the candidate latency from 7.3s to 14.0s. NATURAL configurations with active ICL and example selection yield up to +2.3pp on SPIDER (dev), +3.9pp on SPIDER (test) and +19.8pp on BIRD compared to *Zero-Shot*. This shows that both schema subsetting and self-refinement techniques provide minimal benefit without ICL and example selection. Further ablation studies would help to understand the potential cross effects between components and the implications of disabling pipeline components like schema-subsetting, self-refinement and majority voting. Furthermore increasing the pipeline iterations from 1 to k would yield insights into self balancing and self correction across iterations of NATURAL.

7.1.2 Example Structure

A comparison of different example sources shows that structurally diverse synthetic examples are not necessarily performing worse than examples from the same dataset using NATURAL's example selection algorithm. Synthetic examples outperformed domain-similar training examples on BIRD by 5.4pp (53.8% vs. 48.4%), while perfect example selection yielded 55.8% accuracy and thus adds +2.0pp over schema-aware selection. This indicates diminishing returns of ICL from increasingly sophisticated example strategies.

Performance characteristics reveal a complex tradeoff for real-world applications. All NL2SQL systems that were evaluated maintain remarkably low error rates of less than 0.7 failures per 1000 queries (see Figure 15) errors are therefore becoming predominantly semantic (i.e., incorrect results). Candidate latency ranges from 7.3s for NATURAL (*Baseline*) to 16.3s for ICL configurations on SPIDER and 43.8s on BIRD. NATURAL is viable for experimental, analytical workflows and asynchronous report generation but unviable for interactive systems

like chatbots. While performance optimizations could be applied the hardware used is the primary bottleneck for the speed that is achievable as the majority of $\mathbb{CL}$ is attributable to inference.

7.1.3 Limited Validity

The $\mathbb{EA}$ gap of 28.1pp between the published OmniSQL-7B results (66.1% on Bird) and local measurements of OmniSQL-7B-gguf (38.0%) raises concerns about the reliability of measurements and highlights systemic challenges in LLM-based research. The analysis of this gap concludes that it is likely attributable to format differences, inference engine variations and prompt template and code formatting sensitivity. Despite the remaining uncertainty regarding absolute performance, clear relative improvements from pipeline composition are observable and reproducible within a controlled environment.

7.2 Research Questions

The research questions that were outlined in Section 1.3 are subsequently discussed and answered using the evaluation results.

7.2.1 RQ1: To what extent can open-source LLMs achieve competitive NL2SQL performance through pipeline composition and optimization?

Open-source NL2SQL models can achieve partially competitive performance on prevalent NL2SQL benchmarks in controlled environments. Through the composition of different algorithms the performance can be significantly improved. The evaluation demonstrates that a combination of ICL, schema subsetting, self-refinement and majority voting can boost the competitiveness of fine-tuned base models. Significant gaps in execution accuracy remain for enterprise-level real-world deployments and further ablation studies are needed to understand the impact of individual algorithms.

Natural achieves up to 81.0% $\mathbb{EA}$ on the Spider (dev) benchmark using the *Syn* configuration (examples selected from the SynSQL-2.5m dataset, Table 3). This represents a +3.1pp improvement over *Baseline* configuration and a +2.0pp increase over the baseline model performance of OmniSQL-7B-gguf. On the test set of Spider the *Train* configuration outperforms the *Syn* configuration by +1.8pp with an $\mathbb{EA}$ of 81.4%. *Train* shows an +2.4pp increase over OmniSQL-7B-gguf. On the more complex Bird benchmark *Syn* outperforms *Train* by +5.4pp in $\mathbb{EA}$ and OmniSQL-7B-gguf by +15.8pp. On all three benchmarks a performance gap ranging from 2.6pp to 28.1pp was observed which reduces the trust in the measured results.

This yields an unclear picture, with Natural showing strong performance gains over the locally measured baseline model performance of OmniSQL but no absolute gain over the published results for OmniSQL and other external systems by H. Li et al.. For definite conclusions on the competitiveness of local LLM-based NL2SQL systems more research is needed.

7.2.1.1 Pipeline Composition Effectiveness

The pipeline composition had notable impact on the performance on NL2SQL benchmarks. The effectiveness of more complex pipeline configurations scaled with database and benchmark complexity. On simpler benchmarks like Spider only marginal, single digit accuracy gains (+2.0pp and +2.4pp) could be observed while approximately doubling the candidate latency (from 6.6s and 6.7s to 16.3 and 15.4s).

On Bird, the difference between the evaluated configurations of Natural is significant. *Baseline* achieved only about 32.4% $\mathbb{EA}$ while *Syn* achieved 53.8% $\mathbb{EA}$, representing a 66% relative gain over baseline performance through an altered pipeline composition.

The ablation study shows that the effectiveness of a pipeline configuration heavily depends on ICL and example selection, as the *Zero-Shot* configuration showed only marginal gains (+0.8pp on Spider (dev), +0.5pp on Spider (test) and +1.6pp on Bird) over the *Baseline* configuration while effectively doubling the candidate latency from 7.3s to 14.0s on Spider (dev), 7.3s to 15.3s on Spider (test) and 9.3s to 18.7s on Bird. A naive composition strategy of simply enabling all available algorithms is not cost effective and might produce counter productive returns on user experience as the latency increases are significant.

The *Ground* configuration of Natural establishes a theoretical performance ceiling using perfect data during example selection and consistently performed best-in class during local measurements. *Ground* achieved 84.2% on Spider (dev), 82.8% on Spider (test) and 55.8% on Bird, representing a relative gain to the next-best performing configuration of Natural of +3.4pp, 1.4pp and +2.0pp respectively. In-context learning and

example selection strategies have limits and are already performing close to them. Further improvements would likely require better foundation models or a different architecture altogether. A visual comparison across the different configurations (see Figure 11) confirms that ICL-based approaches (*Syn*, *Train*, *Ground*) outperform other configurations with diminishing returns from increasingly sophisticated example selection strategies.

7.2.1.2 Comparison to Closed-Source Baselines

As no direct local comparison of NATURAL against closed-source models has been performed, no definite answer can be given to the performance gap between local NL2SQL systems and external, closed-source systems. The published results by H. Li et al. in 2025 report GPT-4o achieving 84.9% execution accuracy on SPIDER test and 64.0% on BIRD development (H. Li et al., 2025). The best-in class configuration of NATURAL compared to GPT-4o shows a performance Δ of +10.3pp on SPIDER (dev), -3.5pp on SPIDER (test) and -10.2pp on BIRD. This suggests competitive but not superior performance, although it must be interpreted cautiously as the performance gap between the results reported by H. Li et al. (2025) and local measurements are non trivial. Given the uncertainties in measurements this thesis can't claim a definite competitiveness compared to closed-source models. Further evaluations are needed to clarify the performance characteristics of all measured systems. Nonetheless, NATURAL presents a portable, at least partially competitive, alternative to closed-source systems.

7.2.1.3 Accuracy Ceiling on Consumer Hardware

As NATURAL was developed and evaluated on consumer hardware, the hardware constraints induced by it imply a ceiling of possible model size and speed characteristics. The RTX 3090 has 24GB VRAM and can theoretically accommodate OMNISQL-7B and OMNISQL-14B. However, NATURAL uses additional embedding models and leaves 2-4GB headroom for KV-caches etc. to prevent out-of-memory crashes. Therefore only the OMNISQL-7B model (15GB without quantization) is supported unless heavier quantization formats are used.

This makes NATURAL accessible to consumers with high-end consumer-grade hardware ($1,500 as of 2026) without requiring enterprise-level hardware. The performance impact of using larger foundation models such as 14B and 32B was not evaluated. For enterprise real-world scenarios larger base models could be used if the available hardware allows for it. The evaluation suggests that with the selected model sizes and the available hardware NATURAL has a theoretical execution accuracy ceiling of 84.2% and 82.8% on SPIDER (dev and test) and 55.8% on BIRD. This marks an upper bound achievable with the 7B models and the algorithms used. The foundation model capabilities emerge as the limiting factor unless an entirely different pipeline architecture such as agentic usage of LLMs is used. Scaling the hardware and parameter sizes to 14B or 32B would likely yield significant gains in execution accuracy.

The latency characteristics of NATURAL remain in the practical range for non-interactive workflows but pose a significant challenge for local, interactive applications. While the *Baseline* configuration achieves 7.3s $\mathbb{CL}$, the optimal configurations of NATURAL (*Syn* and *Train*) only achieve a $\mathbb{CL}$ of 15.4-16.3s (see Table 3). This makes NATURAL viable for exploratory analytics and report generation but not for interactive environments such as chatbots or interactive natural language driven data analysis platforms which would require sub-second latency.

7.2.1.4 Conclusion

Overall, open-source LLMs *can* achieve competitive NL2SQL performance as fine-tuning and ICL showed to be effective and efficient mechanisms to achieve high accuracy values in constrained hardware environments. The measured performance yields significant challenges in enterprise environments as an execution accuracy of 53.8% on BIRD with a latency of 43.7s is unacceptable for end users. While promising, a performance gap remains to be closed. These insights must be interpreted with caution due to the limited evaluation.

7.2.2 RQ2: How do NL2SQL pipeline components interact, and which configurations optimize the accuracy-latency tradeoff?

Combining all pipeline components (σ, ϕ, π, ρ and ν) outperformed configurations with only selective components enabled. The ablation study reveals σ as the primary performance driver through a systematic evaluation of the three types of configurations: *Baseline*, *Zero-Shot* and *Full*. The evaluation results reveal the behaviour of the individual pipeline components in synergy, where *Zero-Shot* only provides a marginal improvement over *Baseline* (e.g., +0.8pp on SPIDER) while nearly doubling the latency. All configurations with active ICL demonstrated significant improvement over their non-ICL counterparts; The worst results produced by ICL

configurations compared to the best-performing non-ICL configurations resulted in an improvement through ICL by +1.7pp on SPIDER (dev), +2.1pp on SPIDER (test) and +14.4pp on BIRD. These results reveal that the other pipeline components bring only a minimal benefit (+0.8pp on SPIDER (dev), +0.5pp on SPIDER (test) and +1.6pp on BIRD) to the $\mathbb{EA}$ performance of a pipeline while significantly impacting (i.e., approximately doubling) the $\mathbb{CL}$.

The component interactions become apparent when examining NATURAL's failure modes. Without examples NATURAL (and thus OMNISQL) generate semantically invalid SQL candidates more often; usually incorrect column or table selection, misunderstanding of column semantics and value ranges or wrong join patterns result in syntactically correct queries which result in wrong result sets. For example the schema subsetting component is unable to work correctly if the initial query candidate is using the wrong set of tables altogether, the majority voting algorithm fails to select the best candidate when all SQL candidates correspond to different result sets and self-refinement can only rarely repair semantically mislead initial queries. In-context learning helps NATURAL to guide the foundational model by supplying a set of highly relevant, ranked examples which steers the model to put its attention on specific parts of the SQL schema. The schema-aware example selection algorithm works particularly well and was shown to be effective even on data that did not originate from the training splits of the corresponding benchmarks (3).

7.2.2.1 Component Contribution Ranking

These insights allow for a ranking of components by their estimated impact based on the ablation study evidence from Table 3 and Figure 11. The pipeline components rank by impact as follows:

1. **Query Projection (π)** – The query projection component itself remains the largest single contributor in performance and the foundation models used predominantly drives the performance of the system. All other components are orchestrated around query projection. This was shown through the *Baseline* configuration which achieved 77.9% and 77.0% on SPIDER (dev and test) and 32.4% on BIRD. The substantial gap between the two benchmarks highlights how severely the model capabilities degrade with increasing task and schema complexity.

2. **Example Selection (σ)** – Example selection provides a non-negligible improvement in performance to the overall system while introducing relatively little candidate latency (e.g., only +0.1-+2.3s on SPIDER). The impact that ICL showed on BIRD demonstrates that example selection is not only helpful for LLMs generating SQL queries but a *highly impactful* measure for enabling them to handle complex query generation tasks on real-world data.

3. **Consensus Voting (ν)** – The consensus voting algorithm selects the most prevalent result set amongst multiple generation attempts and validates that queries execute without any errors prior to returning them to the user. This self-consensus mechanism ensures low syntactic error rates across all configurations (see Figure 15). The voting strategy only gains in performance impact with an increasing number of internal candidates. The evaluation was performed with 1-2 internal candidates due to computational complexity of increasing the pipeline iterations. This prevents definite conclusions on the impact.

4. **Self-Refinement (ρ)** – Self-refinement is effectively prompting the underlying model for repairing or improving its initially produced query candidate. This results in an overall marginal improvement of $\mathbb{EA}$, but comes at a significant computational cost. Adding self-refinement nearly doubles the $\mathbb{CL}$ on average and provides diminishing returns. The results of the *Zero-Shot* configuration, especially on the BIRD benchmark, suggest that self-refinement is unable to correct significant semantic errors on its own without ICL.

5. **Schema-Subsetting (ϕ)** – Schema subsetting aims to reduce the noise for the model by filtering out unused tables from the SQL schema before running query projection, thus improving token efficiency and accuracy. The exact impact of schema-subsetting is difficult to isolate due to the constrained ablation study. It is questionable whether schema-subsetting itself provides a statistically significant improvement in performance or even harms performance as the *Baseline* configuration underperformed the model baseline.

7.2.2.2 Accuracy-Latency Tradeoff

The accuracy-latency measurements visualized in Figure 13 and Table 3 reveal three efficiency profiles across the benchmarked systems.

1. **Baseline** — The baseline profile includes both the *Baseline* configuration of NATURAL and the raw OMNISQL models. The baseline profile generally yields the lowest possible candidate latency, but also yields the lowest scores for both $\mathbb{EA}$ and $\mathbb{EM}$ and thus prioritises latency over accuracy.

2. **Zero-Shot** — The zero shot profile is denoted by a suboptimal accuracy latency tradeoff where there are marginal performance gains over the baseline results but this is accompanied by a strong average increase in latency (effectively doubling on average).

3. **Full** — Lastly the full profile is a balanced profile which weights accuracy over latency. Systems in this class include the *Syn*, *Train* and *Ground* configurations of NATURAL. These systems generally achieve best in class results but also a consistently and significantly higher candidate latency.

These three performance profiles highlight different tradeoffs that can be consciously made depending on the deployment scenario and the target usage pattern

7.2.2.3 Optimal Configurations for Different Deployment Scenarios

Given the tradeoffs, a recommendation by deployment scenarios can be made. The recommended profile depends on the reliability requirements, the user interaction model and the complexity of the domain and its terminology:

1. **Exploratory queries against simple databases** — For interaction heavy systems like query interfaces, data exploration and well-structured and smaller databases the *Baseline* configuration provides the overall best tradeoff given the lower latency which will significantly impact user experience. On simpler databases, the *Baseline* configuration provides the best accuracy to latency ratio.

2. **Mixed complexity use cases** — For general purpose, mixed complexity use cases like internal analytical tools, asynchronous report generation and potentially complex databases the **Full** profile excels. If latency is generally less important than accuracy the latency problem can be eased by provisioning more powerful hardware, especially in enterprise or professional scenarios.

3. **Enterprise databases** — For large, complex databases with tens or even hundreds of tables, dirty data and potential external knowledge requirements (e.g., resembling BIRD) the **Full** profile is recommended. It provides meaningful assistance and paired with synthetic, historic and potentially domain-specialized examples it can achieve meaningful accuracy values in complex environments. Especially when manual querying would take significant amounts of time NATURAL can speed up the workflow.

4. **Research deployments** — For research deployments and user experience evaluations of NL2SQL databases the **Full** profile with potential ground truth samples is deemed optimal. NATURAL could serve for evaluations of the potential of NL2SQL systems and their upper bound performance. Such a research deployment could reference past conversations and historical examples to form a self-learning system. This configuration is not sensible for real world deployments as it would require a ground truth entry for all possibly asked questions.

Concluding, for highly interactive systems like chat applications and interactive learning tools, the **Baseline** profile will yield better latency characteristics while maintaining a significant portion of accuracy, especially on simple tasks. For more complex use cases where accuracy is generally more critical than latency (e.g., asynchronous report generation), the **Full** profile represents a more desirable tradeoff, which renders the **Zero-Shot** profile mostly undesirable.

7.2.2.4 Implications for NL2SQL Pipeline Design

Implications for future NL2SQL pipeline and system design can be derived from this work. First, using example selection and self-learning capabilities for NL2SQL systems using LLMs will notably drive up their accuracy. Focusing on example quality, example selection and example presentation will result in a good return on investment. Second, detailed component ablations will help to evaluate the concrete contribution of individual

components, making iterations simpler and faster. Components should be evaluated against realistic benchmarks (e.g., BIRD or SPIDER 2.0) to measure their real-world effects. Lastly, it is desirable to enable the system to be fine-tunable to a specific deployment scenario. Reference data used, fine-tuned models or specialized example generation can help to improve the accuracy scores on a concrete deployment case beyond the theoretical benchmark results that are cross-domain.

7.2.2.5 Conclusion

NATURAL is a portable NL2SQL system which achieves largely competitive performance for a range of NL2SQL tasks. The **Full** profile of NATURAL delivers significant accuracy gains over the **Baseline** profile on complex use cases. NATURAL has poor latency characteristics compared to the foundation models making it not always the better choice. Therefore, the optimal NL2SQL system depends on the use case at hand, for enterprise level complexity using the advanced pipeline components can be beneficial, especially if continuously learning systems are desirable. For exploratory and low latency systems, such as chatbots, NATURAL provides only marginal benefit over the underlying models.

7.3 Practical Implications

This section discusses the practical implications of the evaluation results ranging from deployment viability to system integration patterns and accessibility concerns.

7.3.1 Deployment Viability and User Experience

NATURAL's deployment viability and user experience closely rely on the latency and portability characteristics of the system. Steeper hardware requirements, suboptimal candidate latency or non trivial dependencies make it harder and less appealing to employ NL2SQL systems.

7.3.1.1 System Integration Approaches

The portability of NATURAL is predominantly defined by its self- contained architecture which enables two recommended deployment architectures offering distinct trade-offs:

1. **Database Extension** — By directly integrating NATURAL into a DBMS a tight coupling between the underlying database and the NL2SQL system can be achieved which allow for potentially drastically reduced latency if NATURAL is used for expensive Online Analytical Processing (OLAP) queries. This is largely implied by the pipeline architecture loading data from the database during inference. Furthermore the embedded architecture allows for enhanced data privacy as no data needs to leave the DBMS during inference. To enable access control patterns NATURAL can hook into row level access control and PostgreSQLs access control system. This architecture comes with two significant downsides. First, the DBMS nodes need to have access to GPUs in order to load the model and run inference directly. Second, this architecture does not scale well, as a linear number of equally sized nodes with GPUs would be required and would thus likely yield a resource under utilization most of the time.

2. **Inference Service** — The alternative architecture is the service oriented architecture which is trivially supported by embedding the Rust crate of NATURAL into a gRPC or REST API. This architecture would primarily excel in efficient resource utilization and deployment flexibility but introduce a latency increase through the large amounts of bidirectional network traffic between the inference service and the DBMS. Lastly the network traffic might result in non trivial load on the overall system and induce scalability challenges.

The portability and flexibility of NATURAL's architecture allows for new hybrid approaches optimizing for specific deployment scenarios.

7.3.1.2 Latency and Interactivity

As shown in the evaluation results (see Table 3) the $\mathbb{CL}$ consistently increases with the pipeline accuracy. This means that more powerful profiles of NATURAL require more powerful hardware in order to maintain stable latency characteristics. As described in 7.2.2.3 the latency of a NL2SQL system is a primary denominator of the use cases it is viable for. High latency systems are primarily sensible for asynchronous tasks whereas low

latency systems are viable for fast and iterative use cases like query interfaces (e.g., snowflake) and chatbot integrations. The currently achieved latency of all NL2SQL systems that were benchmarked on consumer hardware is not viable for interactive user interfaces. Significantly more powerful hardware (e.g., factor 10) would be required to make fully interactive systems viable for users.

7.3.1.3 Trust, Transparency, and Error Communication

As shown in Figure 15 and Figure 11 the errors of Natural and other NL2SQL systems are primarily semantic errors, rather than syntactic errors. This makes error detection and communication significantly harder, as the pipeline does not return information on potential semantic mistakes. A lightweight approach for improving the trust of users in downstream interfaces could be as simple as showing the generated SQL statement, generating multiple viable candidates and interpretations of the users natural language query at once and offer switching between them. A second, more advanced solution for semantic error detection would employ a new pipeline component to interpret the users natural language query in different ways and return a set of different interpretations.

7.3.2 Use Cases and Accessibility

To evaluate the real world impact of Natural, potential use cases and accessibility implications are discussed.

7.3.2.1 Optimal Use Cases

NLIDBs are most viable when the query complexity is moderate, e.g., simple joins, filtering and aggregations. The schema is well-documented, with clear table and column names that closely match domain language used during querying. Lastly, users need to have domain knowledge in order gauge whether returned results are accurate and iterations must be acceptable, e.g., no bulk deletion is automatically performed. Traditional SQL interfaces will remain superior for use cases where production systems are altered, complex analytical queries are asked which require window functions, advanced aggregations and many joins as well as the exact behavior of a query must be user controllable. In an ideal scenario, natural language would be combined with SQL, enabling drafting and iterating on queries with natural languages but eventually transitioning to SQL if precise control is required (e.g., before deleting data). This approach would leverage Natural's strengths (e.g., rapid drafting of reasonable queries) while mitigating its primary weakness when it comes to definite semantic accuracy. Traditional SQL tooling can be further employed for query validation and optimization before executing them. Lower level database interfaces could surface both options simultaneously in the future, allowing users to generate queries in natural language and refine them in SQL within the same workflow, removing the gap.

7.3.2.2 Accessibility and Democratization

The hypothesis of NL2SQL systems to significantly lower the entry barrier to data science and analytics holds true, but the evaluation of Natural and similar NL2SQL systems showed significant caveats due to the limited semantic accuracy (see Figure 11).

The primary user groups benefitting from NL2SQL systems are domain experts with limited SQL knowledge, occasional database users and users wanting to learn SQL. Due to the need to verify the output candidates of Natural for semantic errors it is currently unfeasible for users to have no awareness of SQL as this would risk frequent semantic inaccuracy. The evaluation results show that the accuracy of Natural and OmniSQL is severely constrained on Bird which contains complex schemas and external knowledge requirements. This indicates that the accuracy of NL2SQL systems on complex, real-world relevant databases and questions is still substantially too low in order to sunset the need for SQL.

NL2SQL can speed up various workflows, easing the need to explore and understand the schema manually, allows users to draft sensible queries from natural language problem statements and helps users learn how different natural language questions map to SQL statements. The democratization achieved through Natural & co is therefore mostly *vertical* (speeding up existing users and easing workflows) instead of *horizontal* (allowing entirely unfamiliar users to use databases). With increasing NL2SQL capabilities this trend will likely change.

7.4 Threats to Validity

Given the substantial differences between published and locally reproduced result, the constrained benchmark selection, excluding more advanced benchmarks like Spider 2.0, and the constrained hardware that was available during the evaluation the results of this thesis need to be treated as rather unreliable.

7.4.1 Experimental Limitations

The evaluation had severe limitations with regards to the model sizes, reproducibility and the constrained benchmark selection.

7.4.1.1 Benchmark Representativeness

The evaluation relies on only two benchmarks, Spider and Bird, which likely do not accurately represent the real-world query distributions and databases schemas that NL2SQL systems will encounter. The results are not indicative of guaranteed real-world performance, but highlight patterns where Natural performs well (e.g., structured schemas, simpler questions) and problematic patterns that need improvement (e.g., external knowledge requirements and dirty schemas).

This results in limitations this thesis does not address; First, the evaluation has constrained domain coverage, second the query complexity and distributions are fairly constrained with advanced complexity benchmarks like Spider 2.0 not being used. Lastly, the schema design patterns are limited to the patterns used by Spider and Birdand the SQLite dialect.

7.4.1.2 Model and Hardware Specificity

All results shown in Table 3 are subject to the configurations and models used due to the hardware constraints during benchmarking. More powerful hardware would allow for larger models (e.g., OmniSQL-32B and Qwen Embedding 8B) as well as likely substantially improve the candidate latency measurements. Additionally there exists a risk of OmniSQL being fine-tuned to perform well on Spider training and development sets which would further limit the ability to transfer the performance into real-world deployments.

7.4.1.3 Reproducibility Challenges

The most significant threat to validity is the observed performance gap between the published OmniSQL results and locally achieved results. This gap is substantial on Bird, where published results report 66.1% $\mathbb{EA}$ for OmniSQL-7B while locally only 38.0% $\mathbb{EA}$ could be achieved. This represents a 28.1pp difference in accuracy. As discussed in Section 6.4, this might be caused by factors such as model format differences, diverging hyperparameters, prompt drifting and dataset versioning. The existence of this gap severely undermines the confidence in the absolute performance of Natural compared to OmniSQL and other systems, resulting in uncertainty on whether the steep, relative performance gains shown by Natural are transferable into environments where this gap does not exist. It highlights reproducibility challenges prevalent in LLM-based research.

The implications for this thesis are twofold; First, no absolute performance assessment can be made and any claims of absolute performance superiority of Natural over OmniSQL or GPT-4 would be premature. Further research and extensive evaluations are required to solve this gap and make a reliable, absolute assessment possible. Second, while absolute performance is unreliable, relative performance increases can be treated with more confidence. The algorithms introduced in Natural offer a reliable improvement of up to +21.4pp over the *Baseline* configuration. It is unclear whether this improvement in accuracy would persist or substantially shrink with increasingly better *Baseline* performance. Relative improvements achieved through pipeline composition can be substantial, but must be verified through independent research before any deployment decision is made.

7.4.2 Methodological Constraints

Beyond representativeness and reliability of measurements the chosen evaluation metrics, comparisons with external systems and internal validity are constrained.

7.4.2.1 Evaluation Metric Limitations

While the two prevalent metrics $\mathbb{EA}$ and $\mathbb{EM}$ are indicative of the general pipeline performance and ability to generate sound SQL queries, a series of metrics is missing to get a full understanding of NL2SQL performance that is relevant in real-world deployments.

The metrics lack further dimensions, such as the performance of a query candidate (e.g., inefficient queries should be penalized), user satisfaction and perceived usefulness are not evaluated and learning effectiveness and productivity gains are not measured. This makes the evaluation primarily machine learning bound, rather than impact bound which is acceptable given the rather early stage of NL2SQL systems but limit the ability to forecast the actual benefit of using it. The $\mathbb{EM}$ metric is overly strict, limiting the solution space to one exact style of writing SQL and risks that optimizing for $\mathbb{EM}$ needs fine-tuned models to perform well. These limitations indicate that a system with good metric performance might still bring questionable real world benefits to users (e.g., through a poor user experience, inefficient SQL etc.). Future evaluations should define reasonable metrics that look beyond the functional performance of a system.

7.4.2.2 Comparison Validity

The comparisons with published results face additional validity challenges beyond the observed performance gaps. Variables that influence the validity of the performed comparisons include the potentially diverging hardware platforms, model configurations and evaluation environments that were used. The results from OmniSQL would benefit from an independent reproduction which would substantially strengthen their reliability and enable a conclusion to be drawn on whether the local measurements are faulty and what could have caused them to diverge.

7.4.2.3 Internal Validity

Beyond the comparisons validity with other systems, the internal validity is questionable as there were several significant limitations during the evaluation. Hyperparameters and example selection weights were experimentally estimated, not empirically optimized. The configurations were only a small subset of possible configurations due to the extreme length of benchmark runtimes (12-36h). No statistical testing and observation of performance characteristics was performed. Further evaluations should include detailed case studies, fine-grained performance metrics by query and schema complexity and more benchmark datasets.

8 Summary

This thesis introduced NATURAL, a sovereign and portable NL2SQL pipeline using multiple techniques such as schema-aware example selection (σ), schema subsetting (ϕ), query projection (π), self-refinement (ρ) and consensus voting (ν) while remaining executable on consumer-grade systems. It was evaluated against multiple variants of OMNISQL and GPT-4 on SPIDER and BIRD and a brief ablation study was performed to identify the impact of pipeline components and ICL example sources. The results are presented in the evaluation and discussion sections respectively. The key contributions of this thesis are:

1. **Graph-Based Example Selection** — A new schema-aware example selection mechanism using Wasserstein-Weisfeiler-Lehman graph kernels to measure the structural similarity between two database schemas.
2. **Portable Pipeline Design** — A five-stage pipeline design combining example selection (σ), schema subsetting (ϕ), few-shot generation (π), self-refinement (ρ), and consensus voting (ν) that demonstrated substantial accuracy improvements in a local, controlled environment.
3. **Production Implementation** — A complete, production-grade Rust implementation optimized for consumer-grade hardware (RTX 3090, 24GB VRAM), achieving a latency of 7.3-16.0s on SPIDER and 9.3-43.8s on BIRD depending on the configuration used.
4. **Highlighted Reproducibility Gap** — Documentation and discussion of the 28.1pp performance gap between published and reproduced results of OMNISQL, highlighting critical challenges in LLM-based research reproducibility and transferability of models, prompting future research.

8.1 Research Objectives and Achievement

The primary research objective of this thesis was to design and implement a production-ready, portable and sovereign NL2SQL system which avoids the dependency on proprietary APIs for inference. This goal was achieved by introducing NATURAL, which showed that small and portable LLMs, paired with schema-aware example selection can achieve meaningful performance on imperfect, real-world databases. In local evaluations NATURAL performed +2.0-15.8pp better than the OMNISQL-7B-gguf model, but a gap remains to the published OMNISQL results. This is subject to further evaluations as a full reproduction of the reported baseline model performance was not possible and a significant gap of up to 28.1pp was encountered. Using techniques such as ICL combined with schema-aware example selection can partially close the performance gap to proprietary solutions while maintaining full sovereignty and portability.

8.2 Research Questions

The research questions formulated in Section 1.3 have been addressed in the discussion section. The systematic evaluation gathered and analyzed the performance characteristics and indicates that open-source models can achieve competitive NL2SQL performance in controlled environments. During local evaluations, systematic pipeline composition with graph-based example selection showed to be the critical performance driver, especially on complex databases. Pipeline components show synergistic rather than additive tendencies, but further evaluations, especially better ablations are required to make an absolute assessment.

Notable tradeoff between accuracy and latency was observed across different configurations of NATURAL, which informs deployment decisions and viability for different use cases.

8.3 Future Work

The work on NATURAL revealed several important areas that subsequent research should focus on. These areas include system improvements, reproducibility research, and user-oriented research.

8.3.1 Reproducibility Research

Based on the performance gap analysis in Section 6.4, several recommendations for further research in this area can be made:

8.3.1.1 Systematic Quantization Study

A dedicated investigation of the impact of model quantization on NL2SQL performance could provide guidance for better understanding the quantization-to-performance tradeoffs. This would allow for empirical decision-making when choosing model sizes and quantization for resource-constrained environments. Prior research exists on the general impact of quantization on model performance, but having concrete datapoints specific to NL2SQL would clarify the observed behavior (Dettmers et al., 2022). This study should evaluate SOTA NL2SQL systems across multiple quantization levels (F16, Q8, Q6, Q4, and Q3) on all major NL2SQL benchmarks such as Spider, Spider 2.0, and Bird measuring both the accuracy and latency implications of different configurations. Furthermore, an analysis could reveal correlation in performance behavior between quantization levels and query and schema complexity.

8.3.1.2 Cross-Framework Validation

To isolate inference engine effects and subtle differences introduced by different model formats, the same model should be evaluated across multiple frameworks. This study could include PyTorch and Transformers, llama.cpp, vLLM, and ONNX and evaluate SOTA NL2SQL models in varying sizes against prevalent benchmarks. Consistent results across frameworks would further increase confidence in reproducibility of results, while discrepancies would highlight implementation-specific details of algorithms and prompting.

8.3.1.3 Schema and Example Presentation

While research on schema and example presentation exists within the NL2SQL community, it is not exhaustive and includes evaluations of different classes of schema and query presentation (D. Gao et al., 2023; Y. Gao et al., 2025). Further research in this field could highlight differences introduced through subtle changes in presentation of examples or schemas and recommend mitigation techniques such as identifying the most performant presentation formats or fine-tuning base models to a specific presentation format.

8.3.2 System Improvements

Significant opportunities for future research revolve around further system and performance improvements to enable deployments on less powerful hardware. The overall candidate latency of Natural emerged as a critical blocker for real-world adoption, as requiring users to procure expensive research or enterprise-grade GPUs is suboptimal. Substantial performance optimization is needed to achieve a democratization effect.

8.3.2.1 Model Distillation

Through further research on NL2SQL models and the potential of model distillation, significantly smaller LLMs could yield a noticeable increase in performance and drop the hardware requirements. For users with existing high-end hardware, this could yield a significant candidate latency reduction and unlock interactive use cases that are not viable with the latency values measured in this thesis.

8.3.2.2 Component Refinement

Additional component refinement and removing components that add minimal or no additional performance would both increase the semantic accuracy of Natural and reduce potentially needless computational overheads. Especially the schema subsetting component has shown little to no notable performance improvement, and could potentially even harm performance.

8.3.2.3 Multi-Model Architectures

Lastly, this thesis did not utilize a multi-model or agentic architecture as proposed by Y. Gao et al. (2025). Future research could integrate the learnings of this thesis with a multi-model or agentic architecture to see whether a reimplementation of Natural's components as tool calls provides a performance gain over agentic architectures without example selection algorithms and self-refinement.

8.4 Closing Remarks

This thesis demonstrates that portable, sovereign NL2SQL systems based on open-source models represent a promising alternative to systems relying on proprietary APIs, making NL2SQL deployments in constrained domains such as government, healthcare, and finance feasible. Significant challenges remain, particularly regarding latency, semantic accuracy on complex databases, and reproducibility of research results. The algorithms introduced in this thesis, especially the schema-aware example selection, show that additional algorithmic support systems can partially close the performance gap between open-source and closed-source models in controlled environments. Contributions were made both through a practical implementation of Natural and methodological insights for future NL2SQL research, particularly by highlighting the importance of example sources and selection algorithms. As foundational models continue to improve and hardware becomes more accessible, NL2SQL systems will play an increasingly important role in democratizing data access while maintaining data sovereignty.

A Appendix

A.1 Prompts

A.1.1 Natural Inference Prompt

The following prompt is used by the ICL generator stage (π) when relevant examples are available. Template variables are denoted `{variable}`: `{schema}` is the (optionally subsetted) database schema, `{examples}` is the rendered example block (see below), and `{query}` is the natural language question.

```
### Task Overview
1. You are a text to sql model that is meant to answer the question
   using valid sql.
2. Below, you are provided with a database schema, a set of examples
   and a natural language question.
3. Your task is to understand the schema, the examples and generate a
   valid SQL query to answer the question.

### SQLite Schema
‘‘‘
{schema}
‘‘‘
This schema describes the database’s structure, including tables,
columns, primary keys, foreign keys, and any relevant relationships
or constraints.

### Examples
{examples}

### Instructions
- Make sure you only output the information that is asked in the
  question. If the question asks for a specific column, make sure to
  only include that column in the SELECT clause, nothing more.
- The generated query should return all of the information asked in
  the question without any missing or extra information.
- Before generating the final SQL query, please think through the
  steps of how to write the query.

### Question
‘{query}‘

### Output Format
In your answer, please enclose the generated SQL query in a code block:

‘‘‘sql
-- Your SQL Query
‘‘‘

Take a deep breath and think step by step to find the correct SQL query.
```

The `{examples}` block is rendered from the retrieved sample candidates (filtered to distance < 0.5) as follows. Each example is annotated with its cosine similarity score:

```
Some example questions and corresponding SQL queries are provided
based on similar problems:

‘‘‘sql
-- Question: {question_masked} [similarity={score}]
```

```
{sql}

-- Question: {question_masked} [similarity={score}]
{sql}
‘‘‘

These examples can help you find the solution to the following
question, you must choose the most relevant example!
```

When no examples pass the relevance threshold, `{examples}` is replaced with:

```
No relevant examples found, *carefully* think through the given
question and schema to arrive at a valid query
```

A.1.2 Natural Refinement Prompt

The following prompt is used by the self-refinement stage (ρ). It provides the candidate query generated in the projection stage alongside the schema and original question for correction. Template variables: `{schema}`, `{query}` (the natural language question), and `{candidate}` (the SQL candidate to be revised).

```
### Task Overview:
1. You are a text to sql model that is meant to answer the question
   using valid sql.
2. Below, you are provided with a database schema, a set of examples
   and a natural language question.
3. Your task is to understand the schema, revisit the given SQL query
   to answer the question.
4. If you spot any errors in the SQL query, correct them!

### Database Engine:
SQLite

### Database Schema:
‘‘‘
{schema}
‘‘‘
This schema describes the database’s structure, including tables,
columns, primary keys, foreign keys, and any relevant relationships
or constraints.

### Question
‘‘‘sql
-- Question: {query}
{candidate}
‘‘‘

### Instructions
- Make sure you only output the information that is asked in the
  question. If the question asks for a specific column, make sure to
  only include that column in the SELECT clause, nothing more.
- The generated query should return all of the information asked in
  the question without any missing or extra information.
- Before generating the final SQL query, please think through the
  steps of how to write the query.

### Output Format
In your answer, please enclose the generated SQL query in a code block:
```

```
‘‘‘sql
-- Your SQL Query
‘‘‘

Take a deep breath and think step by step to find the correct SQL query.
```

A.1.3 OmniSQL Inference Prompt

The following simplified prompt is used for the zero-shot *Baseline* configuration and as the base prompt for the OmniSQL model directly (without in-context examples). Template variables: `{schema}` and `{query}`.

```
Task Overview:
You are a data science expert. Below, you are provided with a database
schema and a natural language question. Your task is to understand the
schema and generate a valid SQL query to answer the question.

Database Engine:
SQLite

Database Schema:
{schema}
This schema describes the database’s structure, including tables,
columns, primary keys, foreign keys, and any relevant relationships
or constraints.

Question:
{query}

Instructions:
- Make sure you only output the information that is asked in the
  question. If the question asks for a specific column, make sure to
  only include that column in the SELECT clause, nothing more.
- The generated query should return all of the information asked in
  the question without any missing or extra information.
- Before generating the final SQL query, please think through the
  steps of how to write the query.

Output Format:
In your answer, please enclose the generated SQL query in a code block:
‘‘‘
-- Your SQL query
‘‘‘

Take a deep breath and think step by step to find the correct SQL query.
```

A.2 Code

A.2.1 Source Code Repository

The source code for Natural, including all pipeline components, benchmarking infrastructure, and libraries, is available under the AGPL license:

`https://github.com/mara-schulke/natural`

The repository contains the implementation of all five software components (`natural-models`, `natural-graphs`, `natural-inference`, `natural-sampling`, and `natural-benchmark`) as described in Section ??.

The WWL kernel bindings using `pyo3` are maintained in a separate repository:

https://github.com/hemisphere-systems/wwl

A.2.2 Vector Database API

```
let question = "How many ..";

// Initialize vector database and embedding model
let vector = Vector::infer();
let model = EmbeddingModel::from_env().expect("failed to load embedding model");

// Load schema distance index for the target database
let index: DistanceIndex = DistanceIndexRow::get("test_db", &vector)?.into();

// Embed the question and run the multi-dimensional selection algorithm
let selector = Selector::new(question.as_ref(), None, &model)?;
let samples = selection::<8>(&selector, &index, &vector)?;
```

A.2.2.1 Vector Database Schema

```
CREATE TABLE IF NOT EXISTS databases (
  id               text not null,
  name             text not null,
  graph            text not null
);

CREATE TABLE IF NOT EXISTS distance_indices (
  db_id            text not null,
  distances        text not null
);

CREATE VIRTUAL TABLE IF NOT EXISTS samples USING vec0 (
  id                 text not null,
  db_id              text not null,

  question           text not null,
  question_embedding float[4096],

  sql                text not null,
  sql_embedding      float[4096]
);
```

A.2.2.2 Benchmark Traits

```
pub trait Benchmark {
    type Error: From<eyre::Report> + Debug;
    type Database: BenchmarkDatabase<Error = Self::Error>;

    fn name() -> &'static str;

    fn databases(&self) -> impl Iterator<Item = &Self::Database>;
    fn database(&self, id: impl AsRef<str>) -> Option<&Self::Database>;

    fn tests(&self) -> impl Iterator<
      Item = &<Self::Database as BenchmarkDatabase>::Test
    >;

    fn test(&self, id: impl AsRef<str>) -> Option<
```

```
      &<Self::Database as BenchmarkDatabase>::Test
    >;
}

pub trait BenchmarkDatabase {
    type Error: From<eyre::Report> + Debug;
    type Test: BenchmarkTest;

    fn id(&self) -> &str;
    fn connect(&self) -> Result<Database, Self::Error>;
    fn tests(&self) -> impl Iterator<Item = &Self::Test>;
    fn test(&self, id: impl AsRef<str>) -> Option<&Self::Test>;
}

pub trait BenchmarkTest {
    fn id(&self) -> &str;
    fn db_id(&self) -> &str;

    fn name(&self) -> &str;
    fn question(&self) -> &str;
    fn query(&self) -> &str;
}
```

A.2.3 System Configuration

The following table documents all hyperparameters and configuration parameters used in the NATURAL system.

Component	Parameter	Value
	Example Selection (σ)	
Similarity Weights	Question semantic similarity	70%
	SQL structural similarity	30%
Schema Weighting	Sample-level similarity	70%
	Schema-level compatibility	30%
Retrieval	TOP_K candidates	256
	Relevance threshold	0.5
	Models	
SQL Generation	Model	OmniSQL-7B
	Quantization	GGUF (F16)
	VRAM usage	∼14 GB
Embedding	Model	Qwen3-Embedding-4B
	Quantization	Q8_0
	VRAM usage	∼2 GB
	Embedding dimension	4096
	WWL Kernel ($\mathcal{D}$)	
Propagation	Mode	Categorical
	Iterations	3
Distance	Metric	Wasserstein
	Approximation	Sinkhorn

Table 5: Complete system configuration and hyperparameters for NATURAL.

A.3 Benchmark Results

Detailed measurements and visualizations for the evaluation section.

A.3.1 Spider

This section provides detailed breakdowns of the SPIDER benchmark results for both development and test splits, showing execution accuracy and exact match metrics across all system configurations.

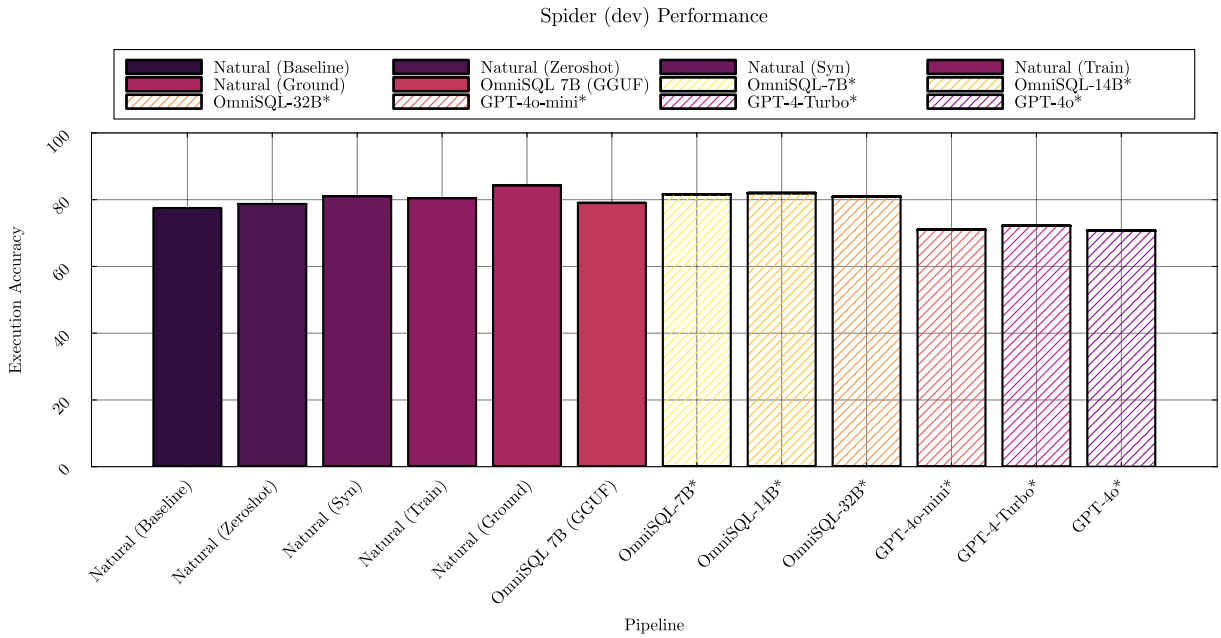

Figure 19: Execution accuracy on SPIDER (dev). The *Syn* configuration achieves the highest execution accuracy among all tested systems at 81.0%, outperforming OmniSQL-7B-gguf by 2.0pp.

A.3.2 Bird

This section provides detailed breakdowns of the BIRD benchmark results for its development split, showing execution accuracy and exact match metrics across all system configurations.

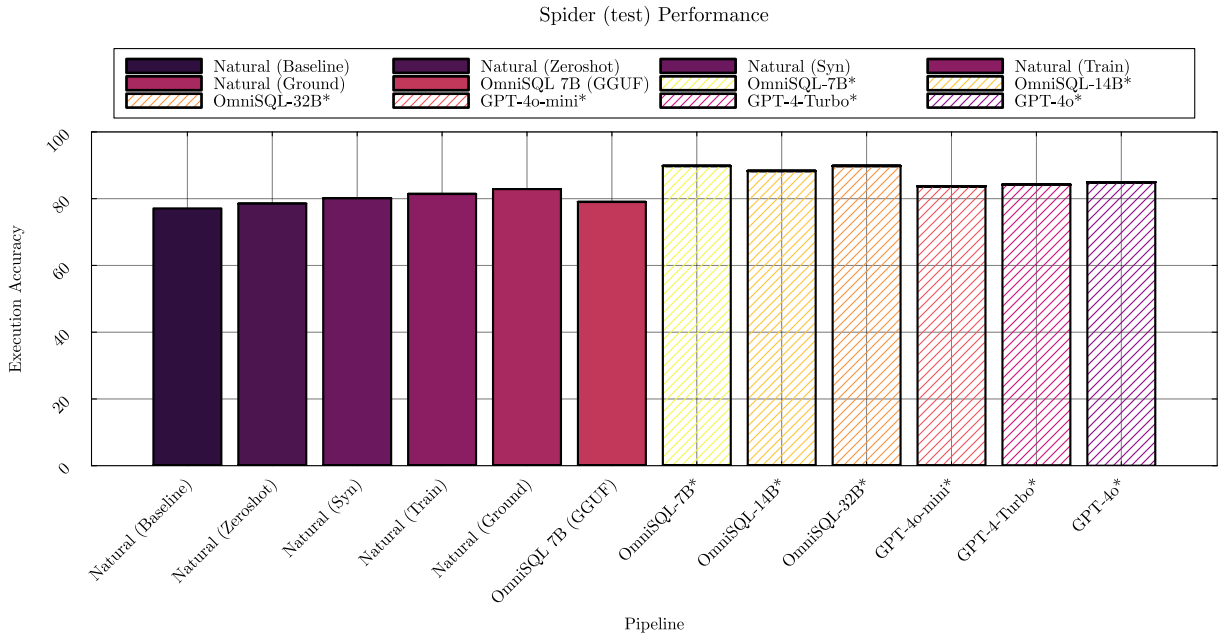

Figure 20: Execution accuracy on SPIDER (test). The *Train* configuration demonstrates the strongest performance among tested configurations, achieving 81.4% execution accuracy.

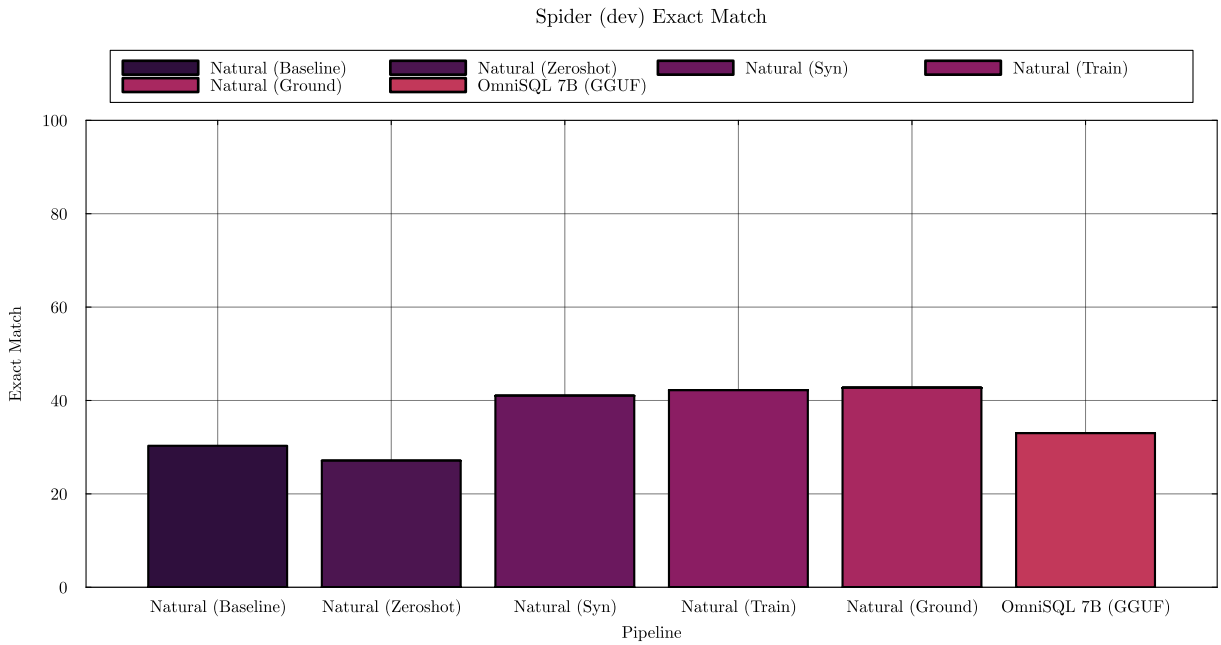

Figure 21: Exact match performance on SPIDER (dev). ICL configurations demonstrate substantial improvements over baseline, with *Ground* achieving 42.7% exact match compared to 30.3% for *Baseline*.

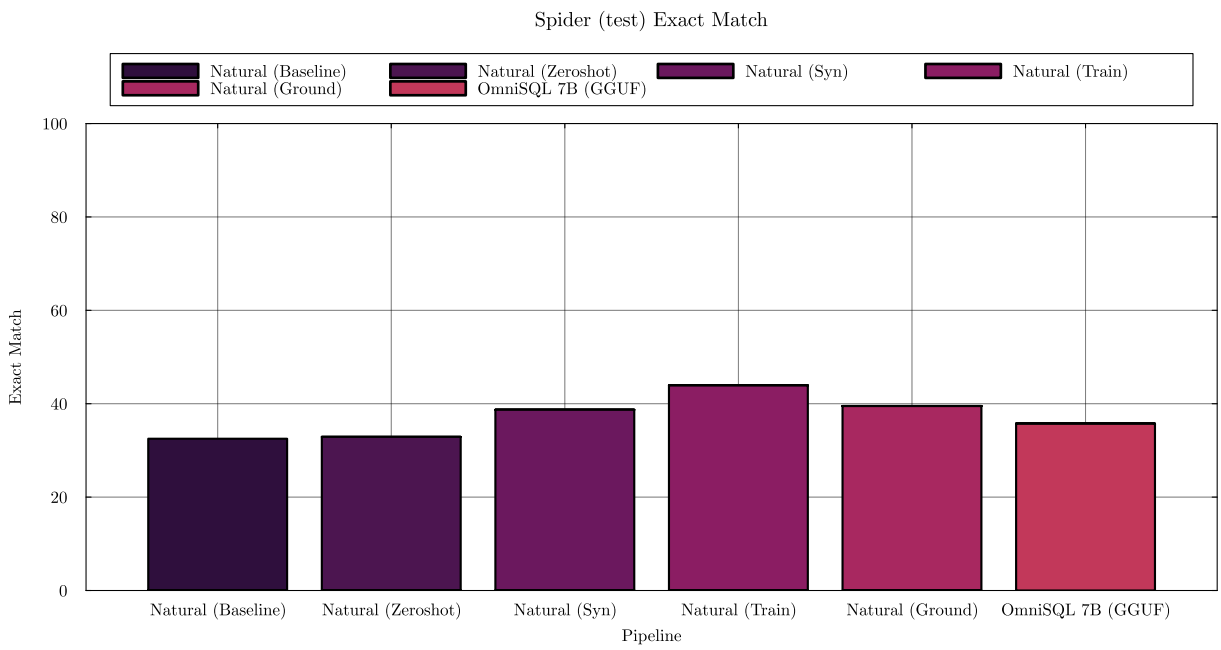

Figure 22: Exact match performance on SPIDER (test). The *Train* configuration achieves 43.9% exact match, representing a +11.4pp improvement over *Baseline*.

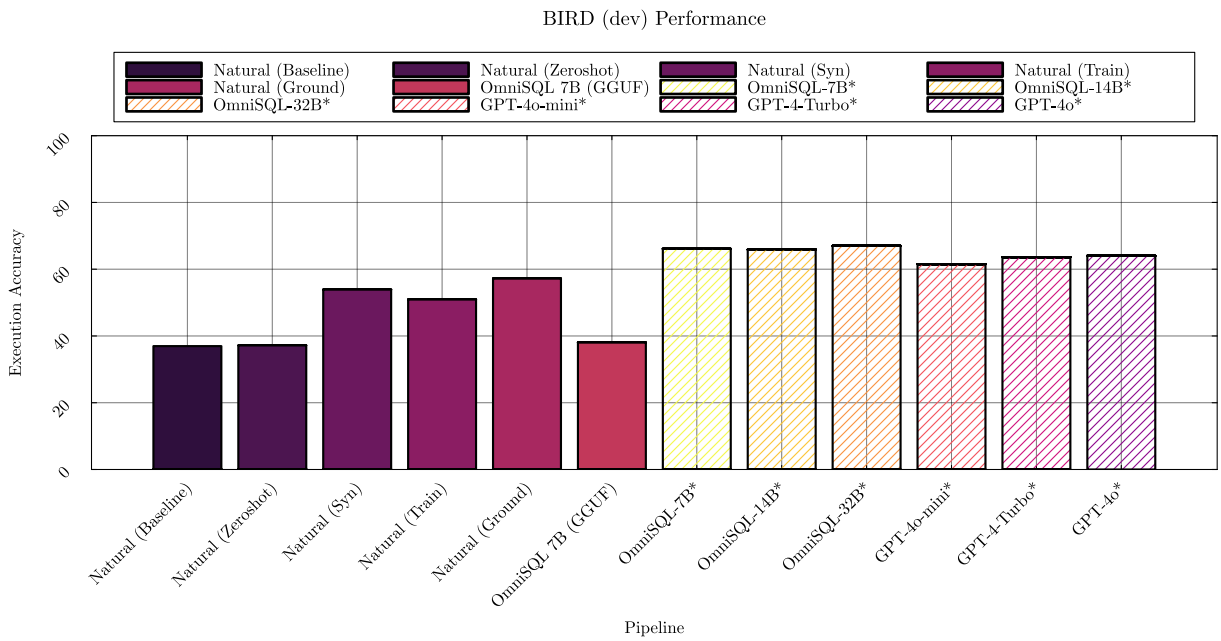

Figure 23: Execution accuracy on BIRD (dev). The *Syn* configuration achieves the highest verified execution accuracy at 53.8%, representing a +21.4pp improvement over the *Baseline* configuration.

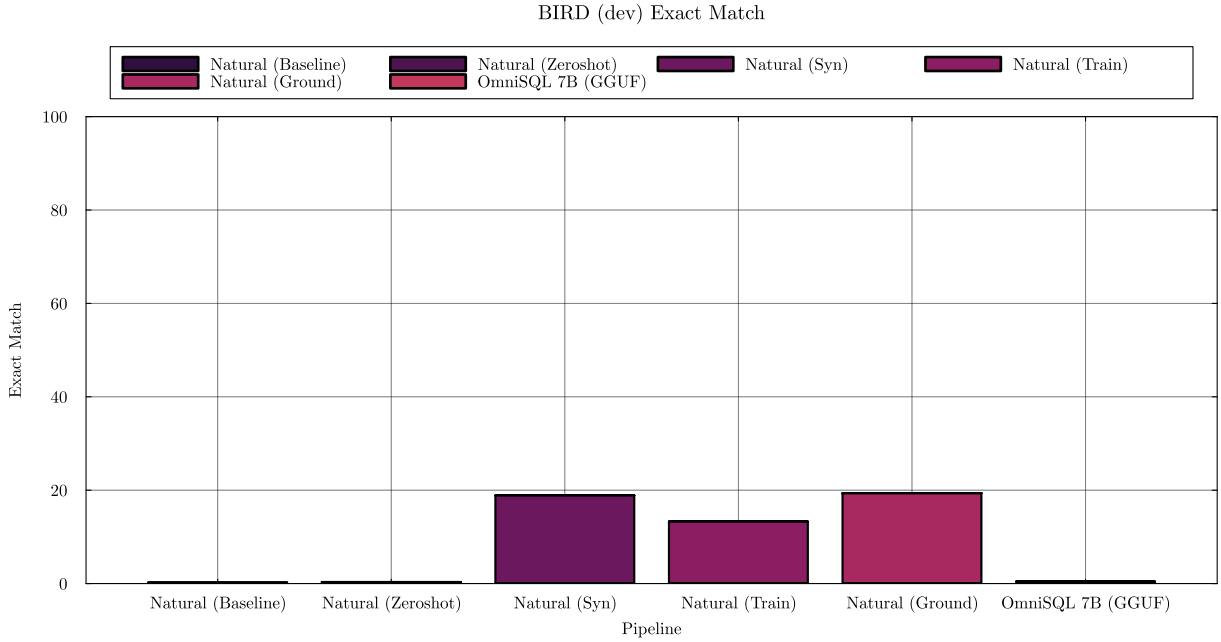

Figure 24: Exact match performance on Bird (dev). ICL-based configurations show dramatic improvements, with *Syn* achieving 18.8% compared to near-zero rates (0.03%) for systems without example retrieval, highlighting the strong impact of in-context learning on syntactic alignment for complex queries.

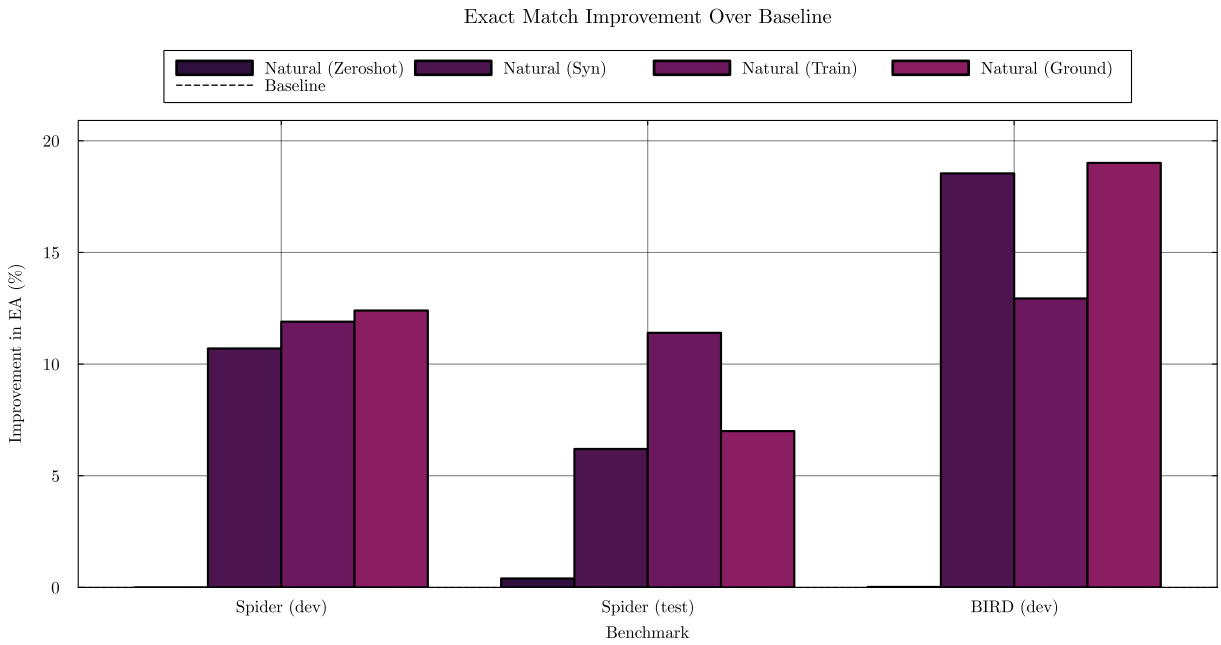

Figure 25: Exact match improvement over the *Baseline* configuration across all benchmarks. The *Train* configuration yields the largest $\mathbb{EM}$ improvement on Spider (+12.0pp on dev, +11.4pp on test), while *Syn* exceeds on Bird (dev) with +18.5pp, confirming that diverse synthetic examples provide stronger syntactic alignment for complex queries.

References

Androutsopoulos, I., Ritchie, G. D., & Thanisch, P. (1995). Natural language interfaces to databases - an introduction. *CoRR*, *cmp-lg/9503016*. Retrieved from `http://arxiv.org/abs/cmp-lg/9503016`

Askari, A., Poelitz, C., & Tang, X. (2024). *Magic: Generating self-correction guideline for in-context text-to-sql.* Retrieved from `https://arxiv.org/abs/2406.12692`

Bogin, B., Berant, J., & Gardner, M. (2019, July). Representing schema structure with graph neural networks for text-to-SQL parsing. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 4560–4565). Florence, Italy: Association for Computational Linguistics. Retrieved from `https://aclanthology.org/P19-1448/` doi: 10.18653/v1/P19-1448

Choi, D., Shin, M. C., Kim, E., & Shin, D. R. (2020). *Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases.* Retrieved from `https://arxiv.org/abs/2004.03125`

Codd, E. F. (1970, June). A relational model of data for large shared data banks. *Commun. ACM*, *13*(6), 377–387. Retrieved from `https://doi.org/10.1145/362384.362685` doi: 10.1145/362384.362685

Codd, E. F. (1974, January). Seven steps to rendezvous with the casual user. In J. W. Klimbie & K. L. Koffeman (Eds.), *Ifip working conference data base management* (p. 179-200). North-Holland. Retrieved from `http://dblp.uni-trier.de/db/conf/ds/dbm74.html#Codd74` (IBM Research Report RJ 1333, San Jose, California)

Date, C. (2003). *An introduction to database systems* (8th ed.). USA: Addison-Wesley Longman Publishing Co., Inc.

Deng, X., Awadallah, A. H., Meek, C., Polozov, O., Sun, H., & Richardson, M. (2020). Structure-grounded pre-training for text-to-sql. *CoRR*, *abs/2010.12773*. Retrieved from `https://arxiv.org/abs/2010.12773`

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). *Llm.int8(): 8-bit matrix multiplication for transformers at scale.* Retrieved from `https://arxiv.org/abs/2208.07339`

Floratou, A., Psallidas, F., Zhao, F., Deep, S., Hagleither, G., Tan, W., ... Curino, C. (2024). Nl2sql is a solved problem... not! In *Cidr*. Retrieved from `https://www.cidrdb.org/cidr2024/papers/p74-floratou.pdf`

Gan, Y., Chen, X., Xie, J., Purver, M., Woodward, J. R., Drake, J. H., & Zhang, Q. (2021). Natural SQL: making SQL easier to infer from natural language specifications. *CoRR*, *abs/2109.05153*. Retrieved from `https://arxiv.org/abs/2109.05153`

Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2023). *Text-to-sql empowered by large language models: A benchmark evaluation.* Retrieved from `https://arxiv.org/abs/2308.15363`

Gao, Y., Liu, Y., Li, X., Shi, X., Zhu, Y., Wang, Y., ... Li, Y. (2025). *A preview of xiyan-sql: A multi-generator ensemble framework for text-to-sql.* Retrieved from `https://arxiv.org/abs/2411.08599`

Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., & Zhang, D. (2019, July). Towards complex text-to-SQL in cross-domain database with intermediate representation. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 4524–4535). Florence, Italy: Association for Computational Linguistics. Retrieved from `https://aclanthology.org/P19-1444/` doi: 10.18653/v1/P19-1444

Hendrix, G. G., Sacerdoti, E. D., Sagalowicz, D., & Slocum, J. (1978, June). Developing a natural language interface to complex data. *ACM Trans. Database Syst.*, *3*(2), 105–147. Retrieved from `https://doi.org/10.1145/320251.320253` doi: 10.1145/320251.320253

Kate, R. J., & Mooney, R. J. (2006, July). Using string-kernels for learning semantic parsers. In N. Calzolari, C. Cardie, & P. Isabelle (Eds.), *Proceedings of the 21st international conference on computational linguistics and 44th annual meeting of the association for computational linguistics* (pp. 913–920). Sydney, Australia: Association for Computational Linguistics. Retrieved from `https://aclanthology.org/P06-1115/` doi: 10.3115/1220175.1220290

Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., ... Yu, T. (2025). *Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows.* Retrieved from `https://arxiv.org/abs/2411.07763`

Li, F., & Jagadish, H. V. (2014). Nalir: an interactive natural language interface for querying relational databases. In *Proceedings of the 2014 acm sigmod international conference on management of data* (p. 709–712). New York, NY, USA: Association for Computing Machinery. Retrieved from `https://doi.org/10.1145/2588555.2594519` doi: 10.1145/2588555.2594519

Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., ... Li, C. (2025). *Omnisql: Synthesizing high-quality text-to-sql data at scale.* Retrieved from `https://arxiv.org/abs/2503.02240`

Li, H., Zhang, J., Li, C., & Chen, H. (2023). *Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql.* Retrieved from https://arxiv.org/abs/2302.05965

Li, H., Zhang, J., Liu, H., Fan, J., Zhang, X., Zhu, J., ... Chen, H. (2024). *Codes: Towards building open-source language models for text-to-sql.* Retrieved from https://arxiv.org/abs/2402.16347

Li, J., Hui, B., Cheng, R., Qin, B., Ma, C., Huo, N., ... Li, Y. (2023). *Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing.* Retrieved from https://arxiv.org/abs/2301.07507

Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., ... Li, Y. (2023). *Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls.* Retrieved from https://arxiv.org/abs/2305.03111

Montgomery, C. A. (1972). Is natural language an unnatural query language? In *Proceedings of the acm annual conference - volume 2* (p. 1075–1078). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/800194.805902 doi: 10.1145/800194.805902

Popescu, A.-M., Etzioni, O., & Kautz, H. (2003). Towards a theory of natural language interfaces to databases. In *Proceedings of the 8th international conference on intelligent user interfaces* (p. 149–157). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/604045.604070 doi: 10.1145/604045.604070

Pourreza, M., Li, H., Sun, R., Chung, Y., Talaei, S., Kakkar, G. T., ... Arik, S. O. (2024). *Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql.* Retrieved from https://arxiv.org/abs/2410.01943

Pourreza, M., & Rafiei, D. (2023). Din-sql: decomposed in-context learning of text-to-sql with self-correction. In *Proceedings of the 37th international conference on neural information processing systems.* Red Hook, NY, USA: Curran Associates Inc.

Rahaman, A., Zheng, A., Milani, M., Chiang, F., & Pottinger, R. (2024). *Evaluating sql understanding in large language models.* Retrieved from https://arxiv.org/abs/2410.10680

Rajkumar, N., Li, R., & Bahdanau, D. (2022). *Evaluating the text-to-sql capabilities of large language models.* Retrieved from https://arxiv.org/abs/2204.00498

Reddy, S., Lapata, M., & Steedman, M. (2014). Large-scale semantic parsing without question-answer pairs. *Transactions of the Association for Computational Linguistics*, *2*, 377–392. Retrieved from https://aclanthology.org/Q14-1030/ doi: 10.1162/tacl_a_00190

Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: parsing incrementally for constrained auto-regressive decoding from language models. *CoRR*, *abs/2109.05093*. Retrieved from https://arxiv.org/abs/2109.05093

Shen, Z., Vougiouklis, P., Diao, C., Vyas, K., Ji, Y., & Pan, J. Z. (2024). *Improving retrieval-augmented text-to-sql with ast-based ranking and schema pruning.* Retrieved from https://arxiv.org/abs/2407.03227

Tang, L. R., & Mooney, R. J. (2001). Using multiple clause constructors in inductive logic programming for semantic parsing. In *Proceedings of the 12th european conference on machine learning* (p. 466–477). Berlin, Heidelberg: Springer-Verlag.

Togninalli, M., Ghisu, M. E., Llinares-López, F., Rieck, B., & Borgwardt, K. M. (2019). Wasserstein weisfeiler-lehman graph kernels. *CoRR*, *abs/1906.01277*. Retrieved from http://arxiv.org/abs/1906.01277

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. *CoRR*, *abs/1706.03762*. Retrieved from http://arxiv.org/abs/1706.03762

Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020, July). RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 7567–7578). Online: Association for Computational Linguistics. Retrieved from https://aclanthology.org/2020.acl-main.677/ doi: 10.18653/v1/2020.acl-main.677

Woods, W. A., Kaplan, R., & Nash-Webber, B. (1972). *The lunar sciences natural language information system: Final report.* Cambridge, Massachusetts: Bolt, Beranek and Newman, Inc.

Xu, X., Liu, C., & Song, D. (2017). Sqlnet: Generating structured queries from natural language without reinforcement learning. *CoRR*, *abs/1711.04436*. Retrieved from http://arxiv.org/abs/1711.04436

Yaghmazadeh, N., Wang, Y., Dillig, I., & Dillig, T. (2017, October). Sqlizer: query synthesis from natural language. *Proc. ACM Program. Lang.*, *1*(OOPSLA). Retrieved from https://doi.org/10.1145/3133887 doi: 10.1145/3133887

Yu, T., Li, Z., Zhang, Z., Zhang, R., & Radev, D. (2018, June). TypeSQL: Knowledge-based type-aware neural text-to-SQL generation. In M. Walker, H. Ji, & A. Stent (Eds.), *Proceedings of the 2018 conference of*

the north American chapter of the association for computational linguistics: Human language technologies, volume 2 (short papers) (pp. 588–594). New Orleans, Louisiana: Association for Computational Linguistics. Retrieved from `https://aclanthology.org/N18-2093/` doi: 10.18653/v1/N18-2093

Yu, T., Wu, C., Lin, X. V., Wang, B., Tan, Y. C., Yang, X., … Xiong, C. (2020). Grappa: Grammar-augmented pre-training for table semantic parsing. *CoRR*, *abs/2009.13845*. Retrieved from `https://arxiv.org/abs/2009.13845`

Yu, T., Yasunaga, M., Yang, K., Zhang, R., Wang, D., Li, Z., & Radev, D. (2018, October-November). SyntaxSQLNet: Syntax tree networks for complex and cross-domain text-to-SQL task. In E. Riloff, D. Chiang, J. Hockenmaier, & J. Tsujii (Eds.), *Proceedings of the 2018 conference on empirical methods in natural language processing* (pp. 1653–1663). Brussels, Belgium: Association for Computational Linguistics. Retrieved from `https://aclanthology.org/D18-1193/` doi: 10.18653/v1/D18-1193

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., … Radev, D. R. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *CoRR*, *abs/1809.08887*. Retrieved from `http://arxiv.org/abs/1809.08887`

Zelle, J. M., & Mooney, R. J. (1996). Learning to parse database queries using inductive logic programming. In *Proceedings of the thirteenth national conference on artificial intelligence - volume 2* (p. 1050–1055). AAAI Press.

Zhang, B., Ye, Y., Du, G., Hu, X., Li, Z., Yang, S., … Mao, H. (2024). *Benchmarking the text-to-sql capability of large language models: A comprehensive evaluation.* Retrieved from `https://arxiv.org/abs/2403.02951`

Zhang, H., Cao, R., Xu, H., Chen, L., & Yu, K. (2024). *Coe-sql: In-context learning for multi-turn text-to-sql with chain-of-editions.* Retrieved from `https://arxiv.org/abs/2405.02712`

Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., … Zhou, J. (2025). Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, *abs/1709.00103*. Retrieved from `http://arxiv.org/abs/1709.00103`